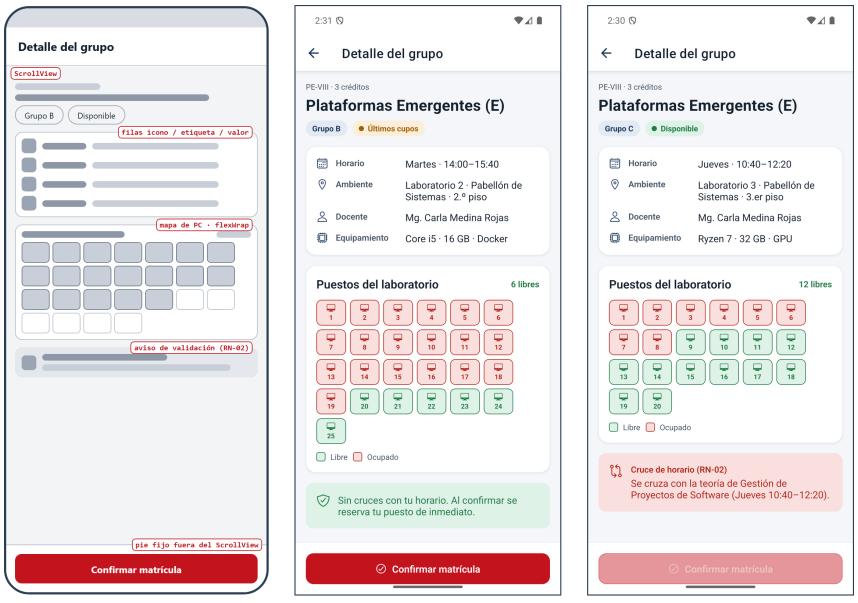
INFORME DE LABORATORIO

Plataformas Emergentes (E) – EPIS UNSA

I. INFORMACIÓN BÁSICA

ASIGNATURA:	**PLATAFORMAS EMERGENTES (E)**		
TÍTULO DE PRÁCTICA:	Framework multiplataforma: interfaces de MatriLab EPIS con React Native		
NRO. PRÁCTICA:	04	**AÑO LECTIVO:**	2026-B
SEMESTRE ACADÉMICO:	VIII	**FECHA DE ENTREGA:**	20/10/2026
MODALIDAD / TIPO:	Grupal (3 integrantes)	**CURSO:**	LABORATORIO
ESTUDIANTE(S):	**1.** Lopez Arela, Ower Frank **2.** Zegarra Puma, Mauricio Eduardo **3.** Huamani Chañi, Diego Oswaldo		
DOCENTE:	M. Sc. Ing. R. Fabrizio Calienes Rodriguez		
REPOSITORIO:	`https://github.com/OwerLopez/matrilab-epis-react-native`		



(a) Mockup (b) Android (c) Variante

MatriLab EPIS ejecutándose en Android: mockup, grupo disponible y bloqueo por cruce de horario.

Índice General de Contenidos

1. Descripción general del programa **4**
1.1 Contexto y propósito . . . 4
1.2 Árbol de problemas . . . 4
1.3 Árbol de objetivos y alcance . . . 5
1.4 Actores y roles . . . 6
1.5 Requisitos funcionales atendidos . . . 7
1.6 Elección del framework . . . 7

2. Diseño de interfaces **9**
2.1 Funcionalidades derivadas del árbol de objetivos . . . 9
2.2 Mapa de navegación . . . 9
2.3 Sistema visual . . . 10
2.4 Mockups de baja fidelidad . . . 11
2.5 Criterios de usabilidad y accesibilidad . . . 14

3. Componentes de React Native para la organización de la interfaz **15**
3.1 Cómo dibuja React Native una interfaz . . . 15
3.2 Flexbox en React Native . . . 15
3.3 Contenedores de organización . . . 16
3.4 Listas virtualizadas . . . 17
3.5 Posicionamiento, dimensiones y adaptación . . . 17
3.6 Componentes de interacción y retroalimentación . . . 18
3.7 Navegación nativa . . . 18
3.8 Catálogo de componentes por interfaz . . . 19

4. Descripción del proceso de desarrollo **20**
4.1 Entorno de trabajo . . . 20
4.2 Creación del proyecto y estructura . . . 20
4.3 Arquitectura . . . 21
4.4 Modelo de dominio y reglas de negocio . . . 21
4.5 Estado global, tema y componentes propios . . . 22
4.6 Desarrollo de las interfaces . . . 23
4.7 Adaptación a la orientación y a otras plataformas . . . 35
4.8 Pruebas automatizadas y verificación . . . 37
4.9 Ejecución en Android y recorrido automatizado de capturas . . . 38
4.10 Registro de cambios . . . 40
4.11 Repositorio, ejecución y versión web . . . 40

5. Apreciación de la herramienta **42**
5.1 Características favorables . . . 42
5.2 Características desfavorables . . . 42
5.3 Valoración general . . . 42

6. Conclusiones **44**

7. Anexos: código de la aplicación **45**

8. Referencias **95**

Índice de figuras

1. Árbol de problemas del proceso de matrícula de laboratorios de la EPIS-UNSA 5
2. Árbol de objetivos: los medios fundamentales se traducen en interfaces de MatriLab 6
3. Mapa de navegación de MatriLab: pila nativa raíz y pestañas por rol 10
4. Mockups de baja fidelidad de las interfaces del estudiante (I1 a I6) 12
5. Mockups de baja fidelidad de las interfaces del docente y de administración (I7 a I12) 13
6. Patrones de layout con Flexbox utilizados en MatriLab 16
7. Arquitectura por capas de MatriLab EPIS 21
8. Inicio de sesión: (a) mockup, (b) pantalla inicial, (c) validación del dominio institucional . . . 24
9. Panel del estudiante: (a) mockup, (b) tema claro, (c) tema oscuro después de matricularse en Plataformas Emergentes 25
10. Catálogo: (a) mockup, (b) todos los grupos, (c) filtro Sin cruce de horario 26
11. Detalle del grupo: (a) mockup, (b) grupo B disponible, (c) grupo C bloqueado por cruce con la teoría (RN-02) 27
12. Pase de laboratorio: (a) mockup, (b) pase emitido tras confirmar la matrícula 28
13. Horario: (a) mockup, (b) vista semanal, (c) vista de lista por día 29
14. Panel del docente: (a) mockup, (b) panel con los grupos a cargo 30
15. Nómina del grupo: (a) mockup, (b) dos asistencias registradas con su hora 31
16. Escáner: (a) mockup, (b) cámara activa en el emulador, (c) pase validado con el código de verificación 32
17. Gestión de laboratorios: (a) mockup, (b) lista de ambientes, (c) edición en la hoja modal . . . 33
18. Configuración de la matrícula: (a) mockup, (b) ventana abierta y reglas 34
19. Perfil y ajustes: (a) mockup, (b) perfil del estudiante 35
20. Gestión de laboratorios en orientación horizontal: la FlatList pasa a dos columnas 36
21. La misma aplicación ejecutándose en un navegador web: inicio de sesión, panel del estudiante y catálogo 37

Índice de tablas

1. Actores del sistema y lo que cada uno puede hacer en la aplicación 7
2. Requisitos funcionales del LAB 01 y cómo se resuelven en la aplicación React Native 7
3. Versiones de la pila tecnológica utilizada 8
4. Trazabilidad entre medios del árbol de objetivos, funcionalidades e interfaces 9
5. Tokens de diseño de la aplicación 11
6. Componentes del núcleo y la vista nativa que los representa en cada plataforma (según [2]) . . 15
7. Propiedades de layout más usadas y dónde se aplican 16
8. Configuración de las listas virtualizadas de la aplicación 17
9. Componentes de interacción utilizados 18
10. Componentes de React Native utilizados en cada interfaz 19
11. Entorno de desarrollo y ejecución 20
12. Pruebas unitarias agrupadas por regla verificada 38
13. Cambios realizados durante el desarrollo y su motivo 40
14. Resumen de la apreciación de React Native con Expo 43

1. DESCRIPCIÓN GENERAL DEL PROGRAMA

1.1 Contexto y propósito

MatriLab EPIS es una aplicación móvil para gestionar la matrícula en los grupos de laboratorio de la Escuela Profesional de Ingeniería de Sistemas de la UNSA. Es el mismo dominio que el equipo trabajó en el LAB 01 (selección de herramienta y requisitos) y en el LAB 02 (prototipo RMAD en FlutterFlow). En esta práctica la aplicación se reconstruye desde cero con un **framework multiplataforma**, **React Native**, usando únicamente los componentes propios de la plataforma para organizar cada interfaz.

El programa atiende a tres tipos de usuario. El **estudiante** consulta la oferta de grupos con un semáforo de vacantes, se matricula sin cruces de horario y obtiene un pase digital con código QR. El **docente o jefe de prácticas** revisa la nómina de sus grupos y registra la asistencia leyendo el QR con la cámara del teléfono. La **coordinación de laboratorios** define la capacidad de cada ambiente y la ventana de matrícula. Todas las pantallas comparten un mismo código TypeScript que se ejecuta como aplicación nativa en Android y como aplicación web.

Los datos de esta versión son de demostración y viven en memoria: el objetivo de la práctica es el diseño y la organización de las interfaces con los componentes del framework, no la persistencia. Aun así, las reglas de negocio (cruce de horarios, aforo, ventana de matrícula y verificación del pase) son reales, están aisladas en un módulo de dominio y se verifican con pruebas automatizadas.

1.2 Árbol de problemas

El diseño parte del diagnóstico realizado en el LAB 01 sobre el proceso de matrícula de laboratorios. Para ordenarlo se aplicó la técnica del árbol de problemas de la metodología del marco lógico [17]: se identificó un problema central, sus causas directas e indirectas y los efectos que produce. La Figura 1 muestra el resultado.

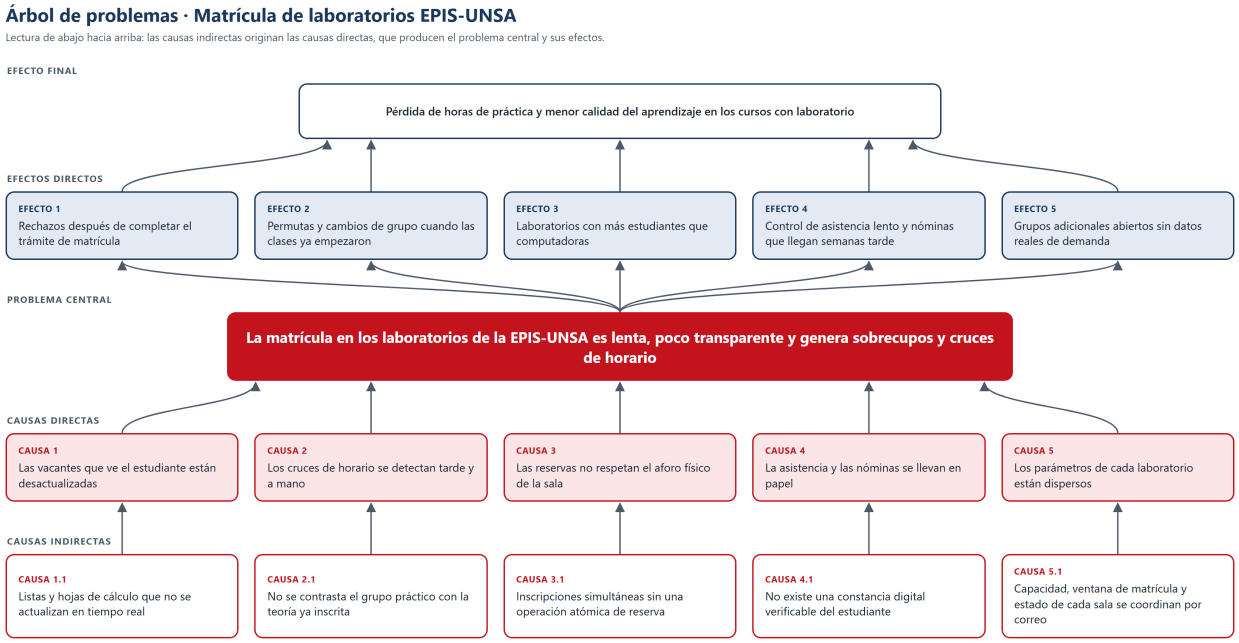


Figura 1. Árbol de problemas del proceso de matrícula de laboratorios de la EPIS-UNSA

El problema central es que **la matrícula en los laboratorios es lenta, poco transparente y genera sobrecupos y cruces de horario**. Las cinco causas directas se explican por causas más profundas: las vacantes se publican en listas que no se actualizan en tiempo real, nadie contrasta el grupo práctico con la teoría ya inscrita, las inscripciones simultáneas no se procesan como una operación atómica, no existe una constancia digital verificable y la información de cada sala se coordina por correo. Los efectos llegan hasta el aula: estudiantes rechazados al final del trámite, permutas cuando las clases ya empezaron, salas con más estudiantes que computadoras y nóminas que el docente recibe semanas tarde.

1.3 Árbol de objetivos y alcance

Cada condición negativa se convirtió en la situación deseada correspondiente (Figura 2). Los **medios fundamentales**, en la base del árbol, son las acciones concretas que el software puede ejecutar; por eso se tomaron como origen de las funcionalidades. En la misma figura, debajo de cada medio, se indican las interfaces de MatriLab que lo implementan.

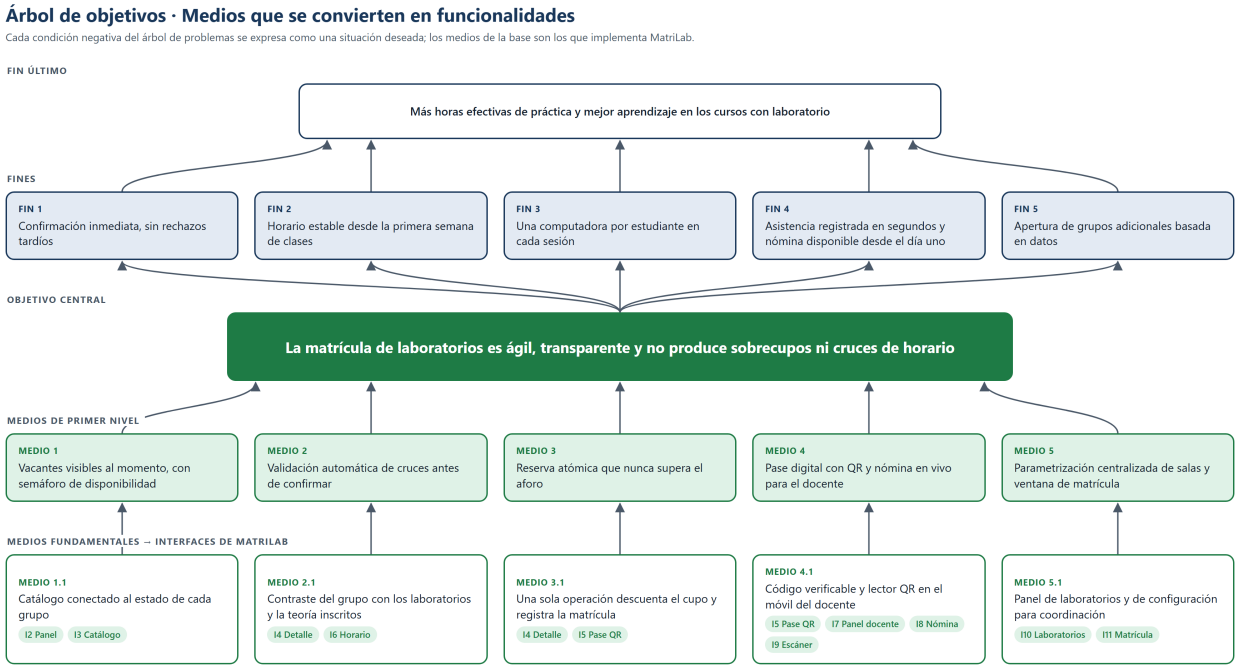


Figura 2. Árbol de objetivos: los medios fundamentales se traducen en interfaces de MatriLab

El alcance de esta práctica cubre los cinco medios con **doce interfaces** (I1 a I12). Quedan fuera la autenticación real contra Google Workspace y la persistencia en la nube, que en el LAB 02 se resolvieron con Firebase; aquí se simulan con un inicio de sesión de demostración y un almacén de estado local, de modo que la aplicación funciona sin conexión y cualquier evaluador puede probarla.

1.4 Actores y roles

Tabla 1. Actores del sistema y lo que cada uno puede hacer en la aplicación

Rol	Cuenta de demostración	Responsabilidades en MatriLab	Interfaces
Estudiante	Lopez Arela, Ower Frank · CUI 20213412	Consultar la oferta, matricularse sin cruces, ver su horario, mostrar su pase QR y anular una matrícula dentro de la ventana.	I1–I6, I12
Docente / jefe de prácticas	Mg. Carla Medina Rojas (ficticia)	Revisar la ocupación de sus grupos, consultar la nómina, registrar asistencia con QR o código manual y exportar la nómina.	I1, I7–I9, I12
Coordinación de laboratorios	`laboratorios.epis@unsa.edu.pe`	Ajustar capacidad y estado de cada ambiente, definir la ventana de matrícula y revisar los grupos agotados.	I1, I10–I12

1.5 Requisitos funcionales atendidos

La Tabla 2 relaciona los requisitos funcionales formulados en el LAB 01 con la interfaz y el mecanismo que los resuelve en esta versión.

Tabla 2. Requisitos funcionales del LAB 01 y cómo se resuelven en la aplicación React Native

Código	Requisito	Cómo se resuelve	Interfaz
RF-01	Autenticación institucional	Validación del dominio `@unsa.edu.pe` y de la contraseña; acceso de demostración por rol.	I1
RF-02	Catálogo de cursos y grupos	`SectionList` agrupada por curso con buscador y filtros.	I3
RF-03	Vacantes en vivo	Semáforo y barra de ocupación derivados del estado global; se recalculan al matricular o anular.	I2, I3, I4
RF-04	Validación anti-cruces	`validarMatricula` contrasta el grupo con los laboratorios y la teoría inscritos (RN-02).	I4
RF-05	Reserva atómica	`reservarCupo` dentro del reductor: el cupo se descuenta y la matrícula se registra en una sola transición.	I4
RF-06	Ficha digital con QR	`react-native-qrcode-svg` con contenido firmado por una huella FNV-1a.	I5
RF-07	Horario consolidado y nómina	Cuadrícula semanal con bloques posicionados y nómina virtualizada.	I6, I7, I8
RF-08	Asistencia por escaneo QR	`CameraView` de `expo-camera` con lector de códigos QR y verificación del pase.	I9
RF-09	Parametrización de laboratorios	Capacidad con *stepper*, estado con `Switch` y edición en `Modal`; ventana de matrícula validada.	I10, I11
RF-10	Exportación de nóminas	CSV con separador punto y coma compartido con la API `Share`.	I8

1.6 Elección del framework

La guía permite Flutter, React Native o Xamarin. El equipo eligió **React Native con Expo** por tres razones verificables. Primero, React Native dibuja la interfaz con **vistas nativas** de cada sistema operativo: un `<View>` es un `ViewGroup` en Android y un `UIView` en iOS [2], lo que responde de forma directa al requisito de usar los componentes propios de la plataforma. Segundo, el LAB 02 ya se había construido sobre Flutter mediante FlutterFlow; usar otro framework permite comparar ambos enfoques con evidencia propia. Tercero, Expo [14] permite ejecutar la aplicación en un teléfono o emulador con Expo Go sin compilar proyectos nativos, y exportarla a la web con el mismo código.

Tabla 3. Versiones de la pila tecnológica utilizada

Componente	Versión	Uso en el proyecto
React Native	0.88.0	Framework multiplataforma; Nueva Arquitectura (Fabric, JSI) y motor Hermes.
React	19.3.0	Modelo de componentes, hooks y contexto.
Expo SDK	58 (CLI 58.1.13)	Herramientas, `expo-camera`, `expo-status-bar`, exportación web.
TypeScript	6.0	Tipado estático de modelo, navegación y componentes.
React Navigation	7.x	`native-stack` y `bottom-tabs`.
react-native-safe-area-context	5.10	Márgenes seguros (reemplazo del `SafeAreaView` del núcleo).
react-native-svg / qrcode-svg	15.16 / 6.3	Dibujo vectorial del código QR.
react-native-web	0.21	Traducción de los mismos componentes al DOM.
Jest + jest-expo	30 / 58	Pruebas unitarias del dominio y del reductor.
Node.js / npm	22.14 / 10.9	Entorno de ejecución de las herramientas.

2. DISEÑO DE INTERFACES

2.1 Funcionalidades derivadas del árbol de objetivos

Cada medio fundamental del árbol de objetivos se descompuso en funcionalidades concretas. La Tabla 4 muestra esa trazabilidad, que es la base del diseño: ninguna pantalla se agregó sin un medio que la justifique.

Tabla 4. Trazabilidad entre medios del árbol de objetivos, funcionalidades e interfaces

Medio	Funcionalidad	Interfaz
1.1 Catálogo conectado	F-01 Iniciar sesión con correo institucional y elegir rol	I1 Inicio de sesión
1.1 Catálogo conectado	F-02 Ver resumen personal: ventana, créditos, vacantes y laboratorios matriculados	I2 Panel del estudiante
1.1 Catálogo conectado	F-03 Explorar grupos por curso con semáforo, buscador y filtros	I3 Catálogo
2.1 Contraste de horario	F-04 Revisar el detalle del grupo y el mapa de puestos; validar cruces antes de confirmar	I4 Detalle del grupo
2.1 Contraste de horario	F-05 Consultar el horario semanal con teoría y laboratorios	I6 Horario
3.1 Operación única	F-06 Confirmar la matrícula con reserva atómica del cupo	I4 Detalle del grupo
4.1 Código verificable	F-07 Mostrar y compartir el pase con QR y código de verificación	I5 Pase QR
4.1 Código verificable	F-08 Monitorear la ocupación de los grupos del docente	I7 Panel docente
4.1 Código verificable	F-09 Consultar la nómina, marcar asistencia y exportar CSV	I8 Nómina
4.1 Código verificable	F-10 Leer el QR con la cámara o digitar el código	I9 Escáner
5.1 Parametrización	F-11 Ajustar capacidad, estado y datos de cada laboratorio	I10 Laboratorios
5.1 Parametrización	F-12 Configurar la ventana de matrícula y las reglas; revisar grupos agotados	I11 Matrícula
Transversal	F-13 Elegir tema claro u oscuro, ajustar avisos y cerrar sesión	I12 Perfil

2.2 Mapa de navegación

La estructura de navegación se definió antes de dibujar las pantallas (Figura 3). Una **pila nativa** raíz contiene el inicio de sesión, tres navegadores de **pestañas inferiores** (uno por rol) y tres pantallas que se apilan sobre las pestañas: el detalle del grupo, que se presenta como hoja modal, el pase QR y la nómina. Así cada rol ve solo sus opciones y el botón Atrás del sistema se comporta como el usuario espera.

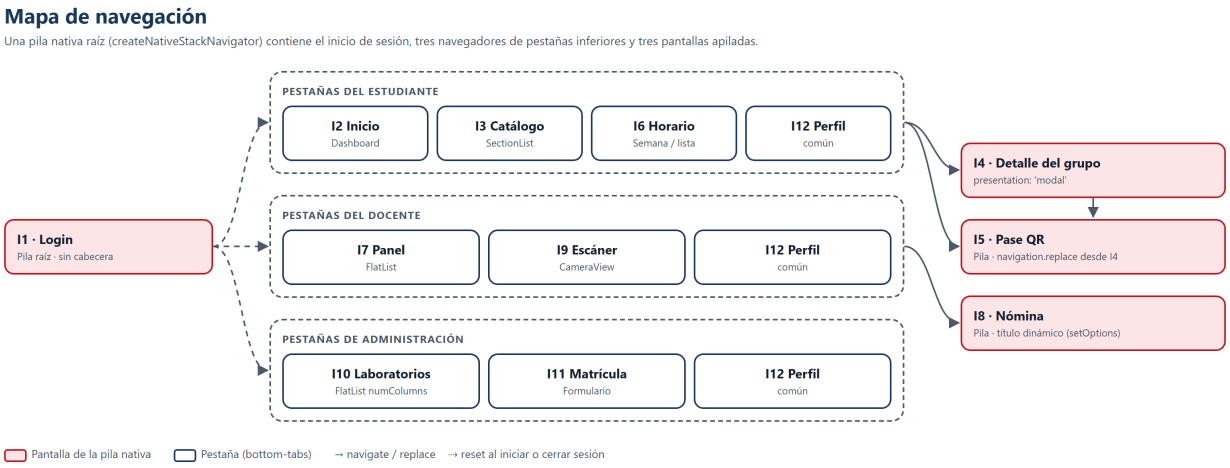


Figura 3. Mapa de navegación de MatriLab: pila nativa raíz y pestañas por rol

2.3 Sistema visual

Para que las doce interfaces se vean consistentes se definieron *tokens* de diseño en `src/theme/tokens.ts`. Se conservaron los colores institucionales usados desde el LAB 01 y se definió una variante oscura de cada color, ya que la aplicación sigue el tema del sistema operativo.

Tabla 5. Tokens de diseño de la aplicación

Token	Claro	Oscuro	Uso
primario	#19375A	#7FA7D9	Botones principales, cabeceras de tarjetas, grupo seleccionado.
marca	#C3141E	#FF6B6B	Acciones decisivas (Ingresar, Confirmar matrícula) y pestaña activa.
éxito / alerta / peligro	#1E7B45 / #A35F00 / #B3261E	variantes claras	Semáforo de vacantes: verde, ámbar y rojo.
fondo / superficie	#F4F6F9 / #FFFFFF	#0F1720 / #17212C	Fondo de pantalla y de tarjetas.
espacio	4 · 8 · 12 · 16 · 24 · 32	igual	Escala de 4 pt para márgenes, rellenos y `gap`.
radio	8 · 12 · 16 · 999	igual	Esquinas de controles, tarjetas y píldoras.
tipo	26 · 18 · 15 · 13 · 12	igual	Título, subtítulo, cuerpo, etiqueta y pie (en dp).

2.4 Mockups de baja fidelidad

Con las funcionalidades y la navegación definidas se dibujaron mockups de baja fidelidad para las doce interfaces (Figuras 4 y 5). Se usaron solo grises, sin textos reales, para discutir la **organización** de cada pantalla antes que su apariencia. Cada región lleva una etiqueta roja con el componente de React Native previsto y los recuadros discontinuos marcan contenedores de layout (filas, columnas o listas). Ese mismo boceto guió luego la implementación: en la sección 4.6 cada mockup aparece junto a la captura real de la pantalla.

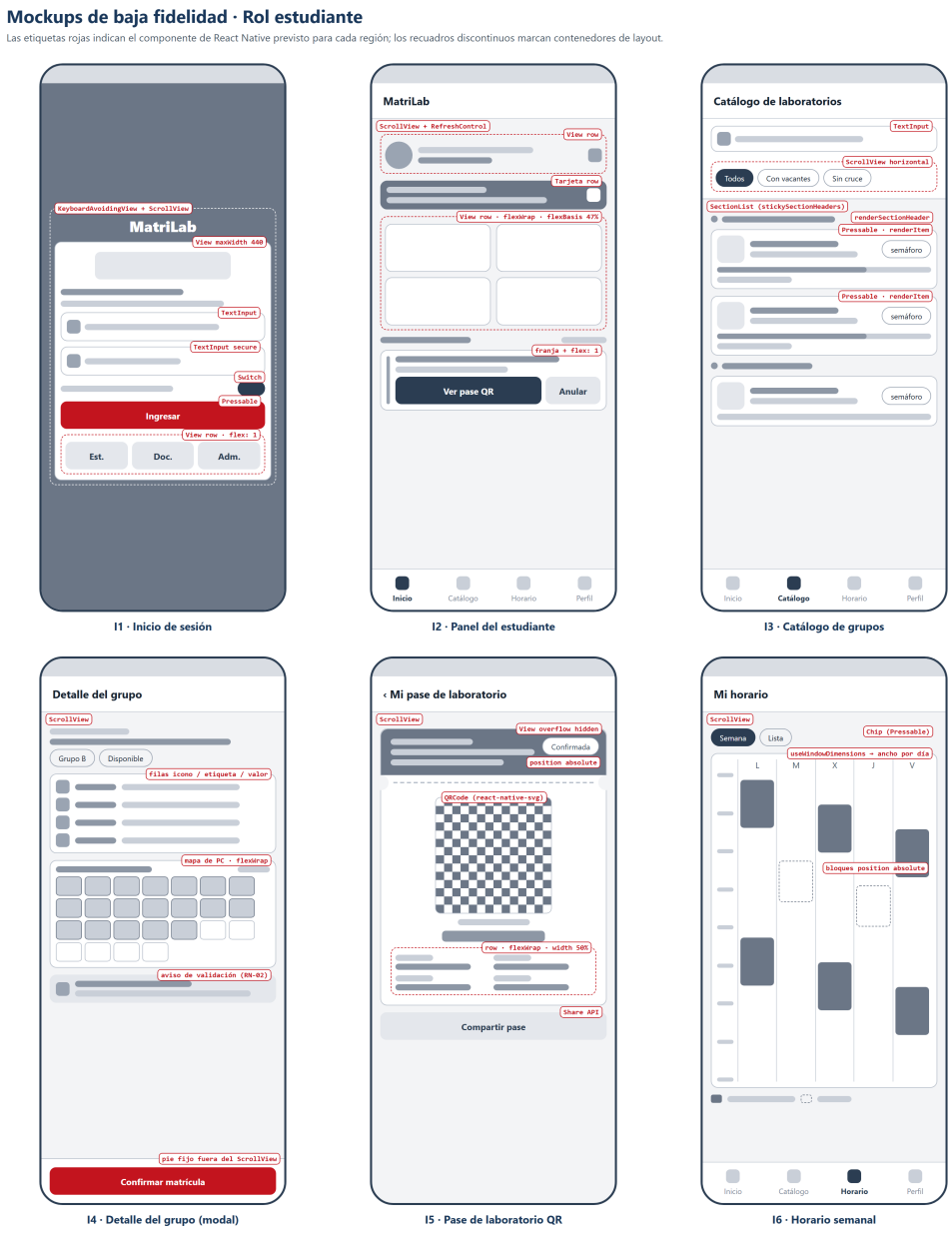


Figura 4. Mockups de baja fidelidad de las interfaces del estudiante (I1 a I6)

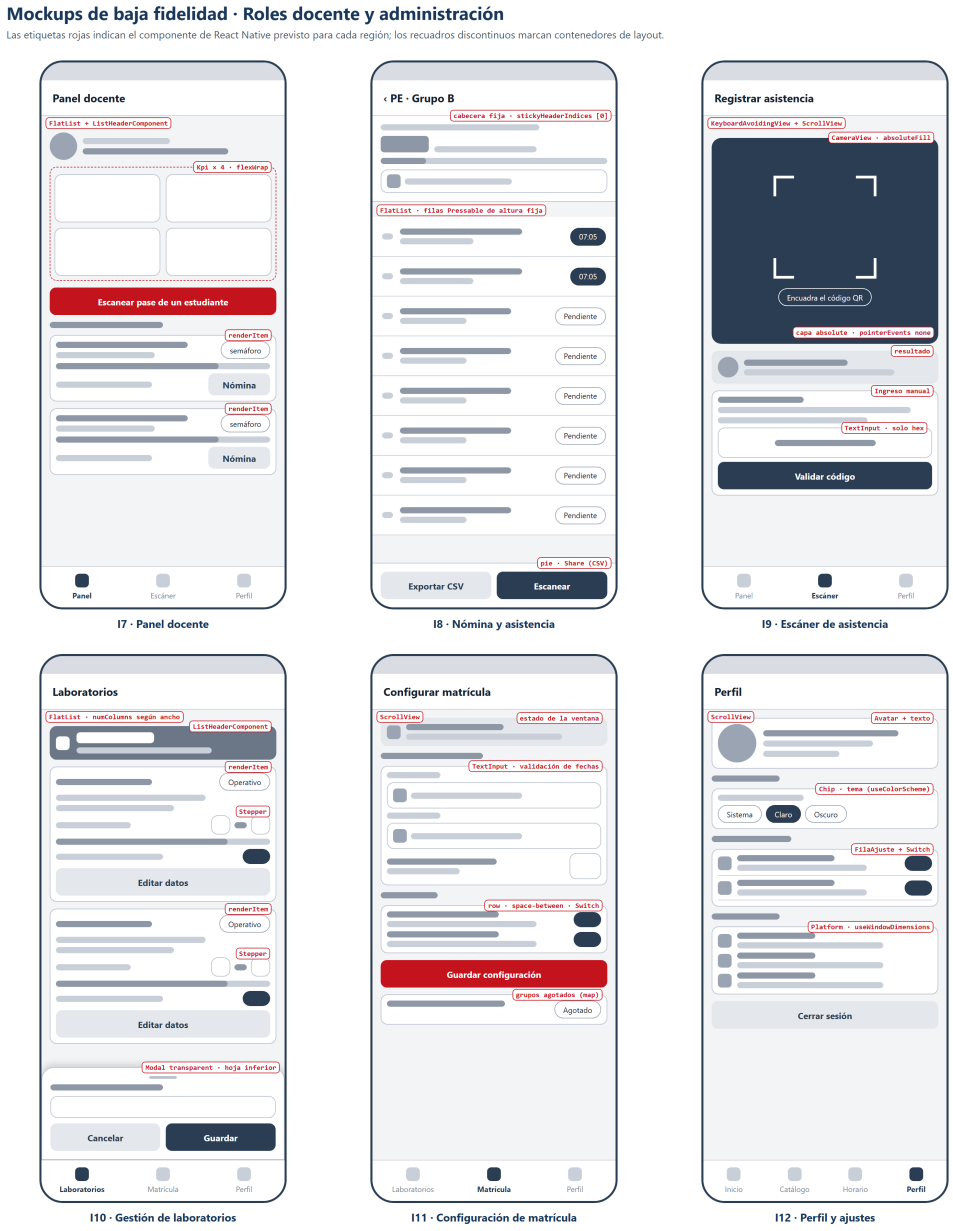


Figura 5. Mockups de baja fidelidad de las interfaces del docente y de administración (I7 a I12)

Las decisiones de organización más importantes que quedaron fijadas en los mockups fueron las siguientes:

- **Listas largas virtualizadas.** El catálogo, el panel docente, la nómina y los laboratorios se planificaron como `SectionList` o `FlatList`, nunca como un `ScrollView` con muchos hijos.
- **Acción principal fija.** En el detalle del grupo y en la nómina el botón principal queda en un pie fuera del área desplazable, siempre visible.
- **Superposiciones con posición absoluta.** El sello del pase, la guía de encuadre del escáner y los bloques del horario se colocan con `position: 'absolute'` sobre su contenedor.

- **Cuadrículas que se reacomodan.** Los indicadores (KPI) y el mapa de puestos usan `flexWrap`, de modo que el número de columnas depende del ancho disponible y no de valores fijos.

2.5 Criterios de usabilidad y accesibilidad

- Área táctil mínima de **48 dp** en botones y campos (`minHeight: 48`), y `hitSlop` en iconos pequeños.
- Todos los controles tienen `accessibilityLabel` y `accessibilityRole`; las filas de la nómina se anuncian como casillas (`accessibilityRole="checkbox"`) con su estado.
- Los mensajes de error y de validación usan `accessibilityLiveRegion` para que el lector de pantalla los lea al aparecer.
- El semáforo nunca depende solo del color: siempre va acompañado del texto *Disponible*, *Últimos cupos* o *Agotado*.
- Tema claro y oscuro con contraste suficiente; la escala de texto del sistema (`fontScale`) se respeta y se muestra en el perfil.

3. COMPONENTES DE REACT NATIVE PARA LA ORGANIZACIÓN DE LA INTERFAZ

3.1 Cómo dibuja React Native una interfaz

En React Native la interfaz se describe con componentes de React escritos en JavaScript o TypeScript, pero lo que aparece en pantalla no es HTML: el renderizador **Fabric** traduce cada componente del núcleo a una vista nativa del sistema operativo [2], [13]. Las posiciones y tamaños se calculan con **Yoga**, un motor de layout multiplataforma que implementa el modelo Flexbox [15]. La Tabla 6 resume la correspondencia documentada para los componentes básicos.

Tabla 6. Componentes del núcleo y la vista nativa que los representa en cada plataforma (según [2])

Componente	Android	iOS	Web (react-native-web)	Función
`<View>`	ViewGroup	UIView	`<div>`	Contenedor con flexbox, estilo, toques y accesibilidad.
`<Text>`	TextView	UITextView	`<p>`	Muestra y anida texto con estilos.
`<Image>`	ImageView	UIImageView	`<img>`	Muestra imágenes locales o remotas.
`<Scroll View>`	ScrollView	UIScrollView	`<div>` desplazable	Contenedor genérico con desplazamiento.
`<TextInput>`	EditText	UITextField	`<input>`	Entrada de texto con teclado del sistema.

Esta correspondencia explica por qué una aplicación React Native se siente propia de cada sistema: el `Switch` de la configuración es el interruptor de Android, el teclado es el del teléfono y la pila de navegación usa las transiciones nativas. A la vez, todo se describe con un solo árbol de componentes, que es lo que permite compartir el código.

3.2 Flexbox en React Native

Toda la organización espacial se hace con Flexbox. Un contenedor define un **eje principal** (`flexDirection`) y distribuye a sus hijos a lo largo de él (`justifyContent`) y del eje transversal (`alignItems`). La documentación oficial advierte que, aunque el modelo es el de CSS, **los valores por defecto cambian** [1]:

- `flexDirection` vale `'column'` (en la web vale `'row'`): por eso, sin indicar nada, los hijos se apilan.
- `alignContent` vale `'flex-start'` en lugar de `'stretch'`.
- `flexShrink` vale `0` en lugar de `1`: un hijo no se encoge salvo que se indique.
- `flex` acepta un solo número, no la forma abreviada de tres valores de CSS.

La Figura 6 reproduce, con las mismas propiedades que usa la app, los ocho patrones de layout que más se repiten en MatriLab.

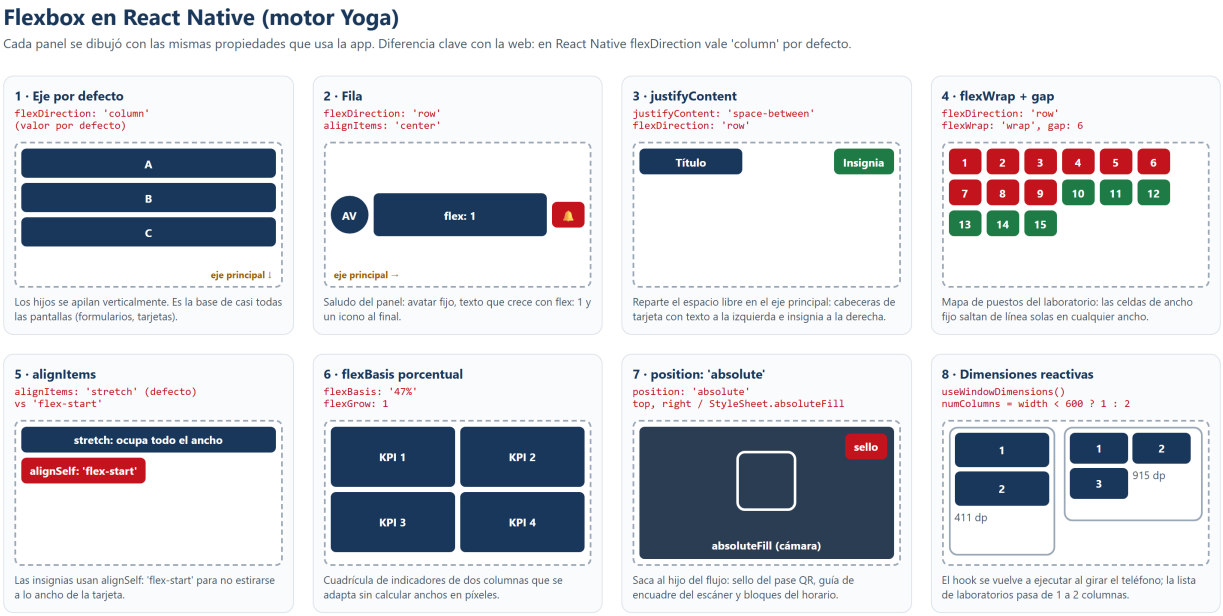


Figura 6. Patrones de layout con Flexbox utilizados en MatriLab

Tabla 7. Propiedades de layout más usadas y dónde se aplican

Propiedad	Valores usados	Ejemplo en MatriLab
`flexDirection`	`'row'`, `'column'`	Filas icono/texto/acción; formularios en columna.
`flex`	`1`	El texto de una fila ocupa el espacio que dejan avatar e insignia.
`justifyContent`	`'space-between'`, `'center'`	Cabecera de tarjeta con título e insignia; botones centrados.
`alignItems` / `alignSelf`	`'center'`, `'flex-start'`, `'stretch'`	Insignias que no se estiran; columna de datos del pase.
`flexWrap` + `gap`	`'wrap'`, 6–12 dp	Mapa de puestos del laboratorio y KPI en dos columnas.
`flexBasis` / `flexGrow`	`'47%'`, `1`	Cada KPI ocupa media fila y absorbe el sobrante.
`position`	`'absolute'` con `top/left/right/bottom`	Sello del pase, guía del escáner, bloques del horario.
`width` en porcentaje	`'50%'` y porcentajes calculados	Datos del pase en dos columnas; relleno de la barra de ocupación.

3.3 Contenedores de organización

- **View.** Es el bloque de construcción de toda pantalla. En MatriLab no se usa ninguna librería de componentes visuales: tarjetas, botones, chips, insignias y barras de progreso se construyeron sobre `View`, `Text` y `Pressable` (archivo `src/components/ui.tsx`).
- **SafeAreaView.** Evita que el contenido quede bajo la barra de estado o la muesca. La documentación de React Native marca el `SafeAreaView` del núcleo como obsoleto y recomienda `react-native-safe-area-context` [16], que funciona también en Android; la aplicación usa este último en el inicio de sesión y con `SafeAreaProvider` en la raíz.
- **ScrollView.** Contenedor desplazable que monta todos sus hijos a la vez [3]. Se reserva para pantallas de contenido acotado: panel del estudiante, detalle del grupo, pase QR, horario, configuración y perfil. Con `horizontal` se usa para la fila de filtros del catálogo y para el horario cuando la pantalla es angosta.
- **KeyboardAvoidingView.** Ajusta el espacio cuando aparece el teclado [8]. Su comportamiento difiere por plataforma, por eso se usa `behavior={Platform.OS === 'ios' ? 'padding' : 'height'}` en el inicio de sesión y en el escáner.
- **Modal.** Presenta contenido sobre la pantalla actual [9]. Con `transparent` y `animationType="slide"` se construyó una hoja inferior para editar los datos de un laboratorio; `onRequestClose` atiende el botón Atrás de Android.

3.4 Listas virtualizadas

Cuando la cantidad de elementos crece, un `ScrollView` desperdicia memoria porque dibuja todo aunque no se vea. `FlatList` y `SectionList` solo montan los elementos cercanos a la zona visible y reciclan el resto [4], [5], [18]. La Tabla 8 resume cómo se configuró cada lista.

Tabla 8. Configuración de las listas virtualizadas de la aplicación

Interfaz	Componente	Propiedades clave	Motivo
I3 Catálogo	`SectionList`	`sections`, `renderSectionHeader`, `stickySectionHeadersEnabled`, `ListEmptyComponent`	Agrupar los grupos por curso con el nombre del curso fijo arriba al desplazarse.
I7 Panel docente	`FlatList`	`ListHeaderComponent`, `ItemSeparatorComponent`	Saludo y KPI dentro de la misma lista, sin anidar listas en un `ScrollView`.
I8 Nómina	`FlatList`	`stickyHeaderIndices={[0]}`, filas de alto fijo	Resumen de asistencia y buscador siempre visibles sobre la lista de estudiantes.
I10 Laboratorios	`FlatList`	`numColumns`, `columnWrapperStyle`, `key={columnas}`	Rejilla de 1, 2 o 3 columnas según el ancho de la ventana.

> `numColumns` no puede cambiar mientras la lista está montada. Por eso en I10 la propiedad `key` depende del número de columnas: al girar el teléfono React desmonta la lista y la vuelve a crear con la nueva rejilla.

3.5 Posicionamiento, dimensiones y adaptación

- **`position: 'absolute'`** saca a un elemento del flujo y lo ubica respecto de su contenedor. `StyleSheet.absoluteFill` es el atajo para cubrirlo por completo, como hace la vista de cámara del escáner.
- **Porcentajes.** `width` y `height` aceptan porcentajes del contenedor [12]; el relleno de la barra de ocupación recibe como ancho el porcentaje calculado de puestos ocupados.
- **useWindowDimensions** devuelve el ancho y alto de la ventana y se actualiza solo al rotar o redimensionar [10]. Con él se calculan el ancho de cada día del horario, el tamaño del QR y las columnas de la lista de laboratorios.
- **Platform** permite pequeñas diferencias por sistema (`Platform.OS`, `Platform.Version`) [11].
- **StyleSheet** agrupa los estilos de cada archivo y ofrece `hairlineWidth`, la línea más fina que puede dibujar la pantalla, usada en bordes y separadores [6].
- **useColorScheme** informa si el sistema está en modo claro u oscuro; el `ThemeContext` lo usa para elegir la paleta.

3.6 Componentes de interacción y retroalimentación

Tabla 9. Componentes de interacción utilizados

Componente o API	Uso en MatriLab
`Pressable`	Base de botones, chips, tarjetas tocables y filas de la nómina. Su `style` recibe `pressed` para el efecto visual y en Android se agrega `android_ripple` [7].
`TextInput`	Correo y contraseña (`secureTextEntry`), buscadores, fechas de la ventana y código de verificación filtrado a hexadecimal.
`Switch`	Mantener sesión, habilitar laboratorio, reglas de matrícula y avisos del perfil.
`ActivityIndicator`	Indicador mientras se confirma la reserva del puesto.
`RefreshControl`	Gesto de deslizar hacia abajo para actualizar el panel del estudiante.
`Alert`	Diálogo nativo para confirmar la anulación de una matrícula y avisar cambios guardados.
`Share`	Hoja nativa de compartir para el pase y para la nómina en CSV.
`Image`	Logotipo de la EPIS en el inicio de sesión con `resizeMode=çontain"`.
`StatusBar` (expo-status-bar)	Texto claro u oscuro de la barra de estado según el fondo de la pantalla.
`CameraView` (expo-camera)	Vista previa de la cámara con lectura de códigos QR (`barcodeScannerSettings`) [19].

3.7 Navegación nativa

React Native no incluye un navegador en su núcleo; se usó **React Navigation** [20], [21]. `createNativeStackNavigator` delega la pila en los componentes nativos (fragmentos en Android y `UINavigationController` en iOS), por lo que las transiciones y el gesto de volver son los del sistema. `presentation: 'modal'` presenta el detalle del grupo como una hoja. `createBottomTabNavigator` genera la barra de pestañas de cada rol; sus íconos cambian entre la versión rellena y la de contorno según la pestaña activa.

3.8 Catálogo de componentes por interfaz

Tabla 10. Componentes de React Native utilizados en cada interfaz

Interfaz	Componentes de organización y layout	Componentes de interacción
I1 Inicio de sesión	SafeAreaView, KeyboardAvoidingView, ScrollView, View (maxWidth)	TextInput, Pressable, Switch, Image, StatusBar
I2 Panel del estudiante	ScrollView, View con flexWrap y flexBasis	RefreshControl, Pressable, Alert
I3 Catálogo	SectionList, ScrollView horizontal	TextInput, Pressable (chips y tarjetas)
I4 Detalle del grupo	ScrollView, View con flexWrap, pie fijo	Pressable, ActivityIndicator
I5 Pase QR	ScrollView, position absolute, width 50 %	QRCode (SVG), Share
I6 Horario	ScrollView (vertical y horizontal), position absolute, useWindowDimensions	Pressable (chips)
I7 Panel docente	FlatList con ListHeaderComponent	Pressable
I8 Nómina	FlatList con stickyHeaderIndices	TextInput, Pressable (checkbox), Share
I9 Escáner	KeyboardAvoidingView, ScrollView, absoluteFill	CameraView, TextInput
I10 Laboratorios	FlatList con numColumns, Modal	Pressable (stepper), Switch, TextInput
I11 Matrícula	ScrollView, filas space-between	TextInput, Switch, Alert
I12 Perfil	ScrollView, filas de ajustes	Pressable, Switch, Platform, useWindowDimensions

4. DESCRIPCIÓN DEL PROCESO DE DESARROLLO

4.1 Entorno de trabajo

Tabla 11. Entorno de desarrollo y ejecución

Elemento	Detalle
Sistema operativo	Windows 11 Pro
Editor	Visual Studio Code
Node.js / npm	22.14.0 / 10.9.2
Expo CLI	58.1.13 (proyecto con Expo SDK 58)
Emulador	Android Studio · Pixel 7a · Android 16 (API 36) · 1080 × 2400 px, 420 dpi (411 × 914 dp)
Cliente de ejecución	Expo Go 58.0.5 en el emulador
Control de versiones	Git y GitHub

4.2 Creación del proyecto y estructura

El proyecto se creó con la plantilla en blanco de TypeScript de Expo y se agregaron las dependencias con `npx expo install`, que elige versiones compatibles con el SDK instalado:

Creación del proyecto e instalación de dependencias

```
npx create-expo-app@latest matrilab-rn --template blank-typescript
cd matrilab-rn
npx expo install @react-navigation/native @react-navigation/native-stack \
  @react-navigation/bottom-tabs react-native-screens react-native-safe-area-context \
  react-native-svg react-native-qrcode-svg expo-camera @expo/vector-icons
npx expo install react-native-web react-dom @expo/metro-runtime
npx expo install jest-expo jest @types/jest -- --save-dev
```

El código se organizó por responsabilidad: dominio sin dependencias de React, estado, tema, componentes reutilizables, navegación y una carpeta de pantallas por rol.

Estructura de carpetas del proyecto

```
matrilab-rn/
|-- App.tsx                  # proveedores: área segura, tema, estado y navegación
|-- app.json                 # configuración de Expo (nombre, orientación, permisos)
|-- src/
|   |-- domain/              # types.ts, reglas.ts (reglas puras), datos.ts (2026-B)
|   |-- state/AppStore.tsx   # Context + useReducer y selectores
|   |-- theme/               # tokens.ts y ThemeContext.tsx
|   |-- components/ui.tsx    # Tarjeta, Boton, Chip, Insignia, BarraProgreso, Kpi...
|   |-- navigation/          # RootNavigator.tsx y tipos.ts
|   `-- screens/             # LoginScreen, PerfilScreen y carpetas estudiante/,
|                            # docente/ y admin/ con las demás interfaces
|-- __tests__/reglas.test.ts # 31 pruebas unitarias
|-- herramientas/capturas-android.mjs  # recorrido automático de capturas
`-- informe/                 # este informe: LaTeX, DOCX, figuras y generadores
```

4.3 Arquitectura

La aplicación se dividió en capas (Figura 7). La capa de presentación solo conoce el estado a través de hooks; el estado aplica las reglas del dominio, y el dominio está formado por funciones puras que no importan nada de React ni de la plataforma. Esa separación permitió probar las reglas de negocio con Jest en un segundo, sin emulador.

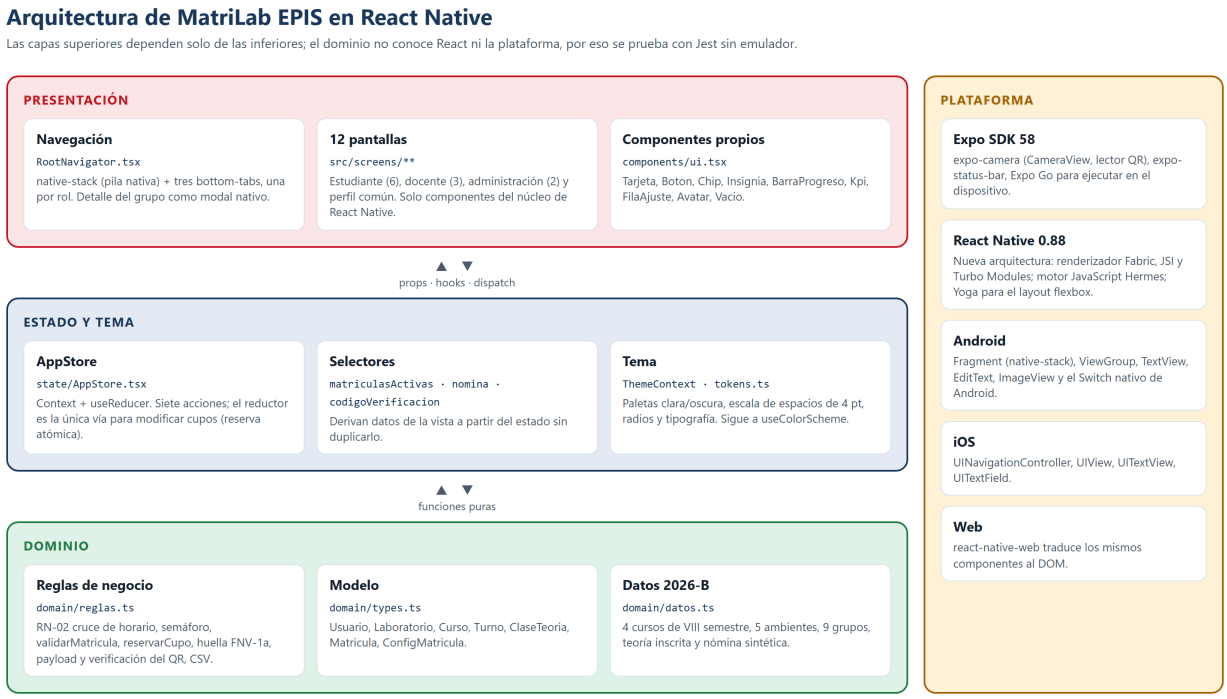


Figura 7. Arquitectura por capas de MatriLab EPIS

4.4 Modelo de dominio y reglas de negocio

Los horarios se guardan en minutos desde la medianoche (por ejemplo, 07:00 es 420). Con esa representación la regla RN-02 de cruce de horario se reduce a comparar intervalos; los extremos no cuentan, de modo que un bloque que termina a las 08:40 no choca con otro que empieza a las 08:40.

src/domain/reglas.ts · cruce de horario y semáforo

```
/** RN-02: dos bloques se cruzan si comparten día y sus intervalos se solapan (los extremos no cuentan).
     */
export function haySolape(a: Bloque, b: Bloque): boolean {
  return a.dia === b.dia && a.inicio < b.fin && b.inicio < a.fin;
}

/** Semáforo de vacantes: rojo sin cupo, ámbar con menos del 30 % libre, verde en otro caso. */
export function semaforo(t: Turno): Semaforo {
  const libres = cuposLibres(t);
  if (libres === 0) return 'rojo';
  return libres / t.cuposTotales < 0.3 ? 'ambar' : 'verde';
}
```

`validarMatricula` aplica los controles en el mismo orden del proceso BPMN del LAB 01 y devuelve el primer motivo de rechazo con un mensaje legible: ventana cerrada, grupo ya matriculado, otro grupo del mismo curso, límite de laboratorios, cruce con un laboratorio, cruce con la teoría y grupo sin cupo. La reserva se hace con una función inmutable: si entre la consulta y la confirmación el cupo se agotó, no modifica nada.

src/domain/reglas.ts · reserva atómica del cupo (RF-05)

```
export function reservarCupo(turnos: Turno[], turnoId: string): { turnos: Turno[]; reservado: boolean }
    {
  const objetivo = turnos.find((t) => t.id === turnoId);
  if (!objetivo || cuposLibres(objetivo) === 0) return { turnos, reservado: false };
  return {
    reservado: true,
    turnos: turnos.map((t) => (t.id === turnoId ? { ...t, cuposOcupados: t.cuposOcupados + 1 } : t)),
  };
}
```

El pase QR contiene la cadena `MATRILAB|matrícula|grupo|estudiante|huella`. La huella es un FNV-1a de 32 bits sobre los cuatro primeros campos: si alguien altera el contenido, la huella deja de coincidir y el escáner rechaza el pase. No pretende ser un mecanismo criptográfico; en producción la verificación debe hacerse en el servidor.

4.5 Estado global, tema y componentes propios

El estado se maneja con `useReducer` dentro de un contexto. El reductor es la única puerta para cambiar los cupos: la acción `MATRICULAR` reserva el cupo y registra la matrícula en la misma transición, lo que en la interfaz garantiza que dos toques seguidos no ocupen dos puestos.

src/state/AppStore.tsx · acción MATRICULAR

```
case 'MATRICULAR': {
  if (!e.usuario) return e;
  const { turnos, reservado } = reservarCupo(e.turnos, a.turnoId);
  if (!reservado) return e;
  const id = `M-${String(e.matriculas.length + 1).padStart(4, '0')}`;
  return {
    ...e,
    turnos,
    matriculas: [...e.matriculas, { id, estudianteId: e.usuario.id, turnoId: a.turnoId, fecha: a.ahora,
     codigo: id, estado: 'confirmada' }],
  };
}
```

Los componentes visuales propios se construyeron solo con primitivas del núcleo. El botón, por ejemplo, es un `Pressable` cuyo estilo depende de la variante y del estado `pressed`, con ripple nativo en Android y estados de accesibilidad:

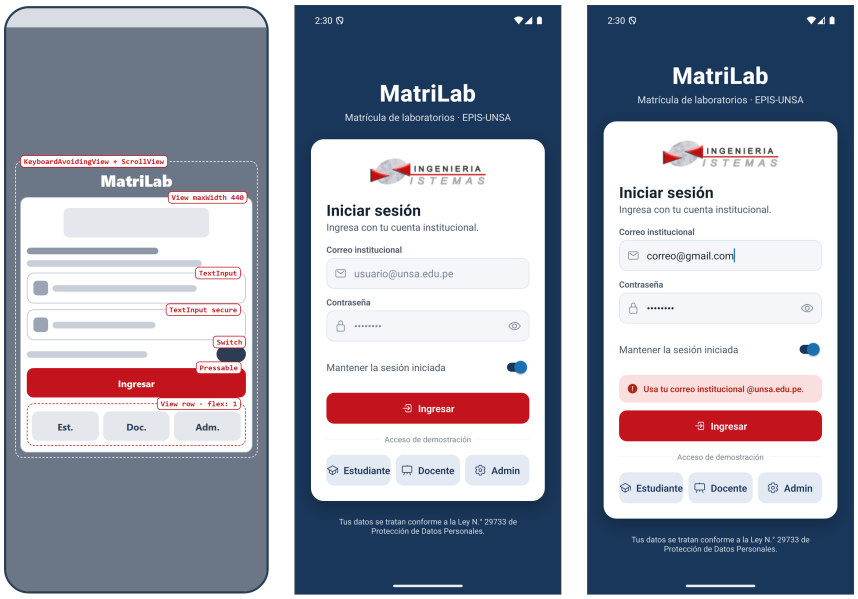
src/components/ui.tsx · componente Boton

```
<Pressable
  onPress={onPress}
  disabled={deshabilitado || cargando}
  accessibilityRole="button"
  accessibilityLabel={accessibilityLabel ?? titulo}
  accessibilityState={{ disabled: Boolean(deshabilitado), busy: Boolean(cargando) }}
  android_ripple={{ color: 'rgba(255,255,255,0.25)' }}
  style={({ pressed }) => [
    estilos.boton,
    { backgroundColor: fondo, borderColor: variante === 'fantasma' ? c.borde : fondo, opacity:
     deshabilitado ? 0.45 : pressed ? 0.85 : 1 },
    style,
  ]}
>
```

4.6 Desarrollo de las interfaces

A continuación se describe cada interfaz. En cada figura, el panel (a) es el mockup de la sección 2.4, el panel (b) es la captura real en el emulador Android y, cuando existe, el panel (c) muestra un estado alternativo de la misma pantalla.

I1 · Inicio de sesión institucional



(a) Mockup (b) Android (c) Variante

Figura 8. Inicio de sesión: (a) mockup, (b) pantalla inicial, (c) validación del dominio institucional

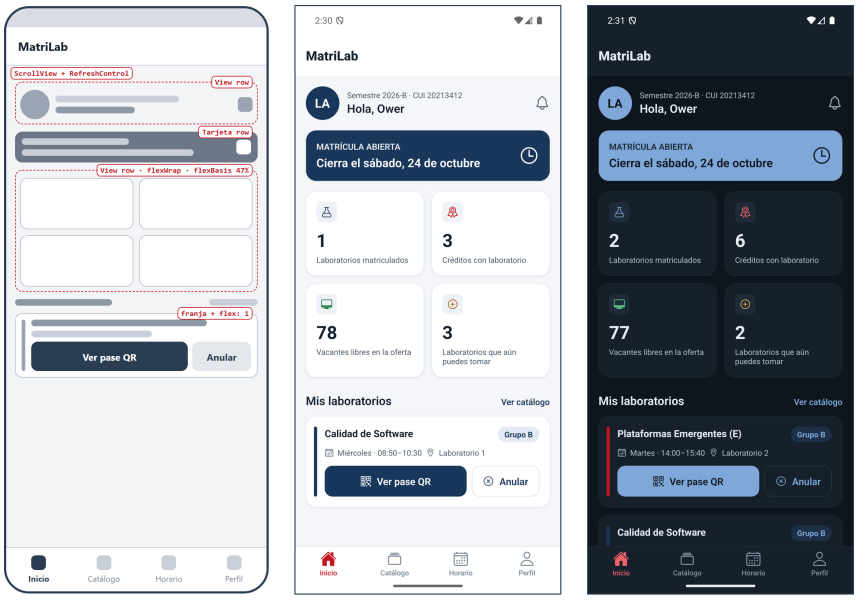
Elementos. Pantalla de entrada con la marca sobre el color primario y una tarjeta blanca centrada. Contiene el logotipo de la EPIS, los campos de correo y contraseña con iconos, el botón para mostrar u ocultar la contraseña, el interruptor *Mantener la sesión iniciada*, el botón **Ingresar** y tres accesos de demostración por rol. Al ingresar un correo que no termina en `@unsa.edu.pe` aparece el mensaje de error del panel (c).

Componentes y layout. La pantalla combina `SafeAreaView` (márgenes seguros), `KeyboardAvoidingView` y un `ScrollView` con `flexGrow: 1` y `justifyContent: 'center'`, de modo que el formulario queda centrado pero sube cuando aparece el teclado. La tarjeta tiene `width: '100%'` con `maxWidth: 440` para no estirarse en tabletas ni en la web. Los tres accesos de demostración son una fila (`flexDirection: 'row'`) de botones con `flex: 1`, que se reparten el ancho por igual. Como el fondo es oscuro, la pantalla fija `<StatusBar style="light/>`.

LoginScreen.tsx · validación del formulario

```
const validar = () => {
  const limpio = correo.trim().toLowerCase();
  if (!/^[a-z0-9._-]+@unsa\.edu\.pe$/.test(limpio)) return setError('Usa tu correo institucional @unsa.
    edu.pe.');
  if (clave.length < 6) return setError('La contraseña debe tener al menos 6 caracteres.');
  const u = USUARIOS.find((x) => x.correo === limpio);
  if (!u || clave !== CLAVE_DEMO) return setError('Correo o contraseña incorrectos.');
  setError(null);
  entrar(u);
};
```

I2 · Panel del estudiante



(a) Mockup (b) Android (c) Variante

Figura 9. Panel del estudiante: (a) mockup, (b) tema claro, (c) tema oscuro después de matricularse en Plataformas Emergentes

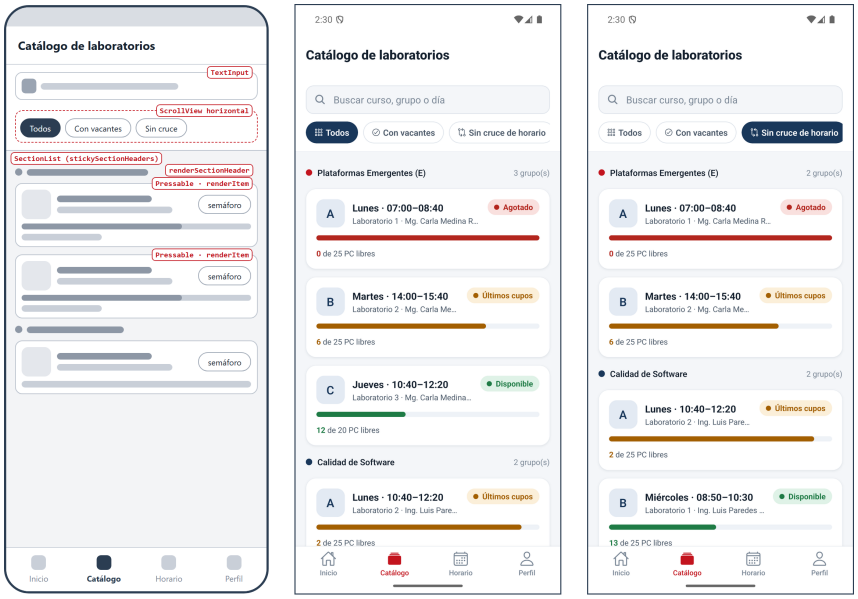
Elementos. Muestra el saludo con avatar e iniciales, una tarjeta con el estado de la ventana de matrícula y su fecha de cierre, cuatro indicadores (laboratorios matriculados, créditos, vacantes libres en la oferta y laboratorios que aún puede tomar) y la lista *Mis laboratorios*, donde cada matrícula tiene una franja con el color del curso y los botones **Ver pase QR** y **Anular**. En el panel (c) se ve la misma pantalla en tema oscuro, ya con dos laboratorios.

Componentes y layout. Un `ScrollView` con `RefreshControl` permite deslizar para actualizar. Los cuatro KPI están en una fila con `flexWrap: 'wrap'` y cada `Kpi` usa `flexBasis: '47%'` y `flexGrow: 1`: en un teléfono se forman dos columnas y en una pantalla ancha caben cuatro sin cambiar el código. La tarjeta de cada matrícula es una fila donde la franja de color es un `View` de 5 dp de ancho y el contenido ocupa el resto con `flex: 1`. Anular usa el `Alert` nativo con un botón de estilo destructivo.

DashboardScreen.tsx · cuadrícula de indicadores

```
<View style={estilos.kpis}>
  <Kpi valor={String(activas.length)} etiqueta="Laboratorios matriculados" icono="flask-outline" />
  <Kpi valor={String(creditos)} etiqueta="Créditos con laboratorio" icono="ribbon-outline" color={c.
    marca} />
  ...
</View>
// kpis: { flexDirection: 'row', flexWrap: 'wrap', gap: espacio.m }
// kpi:  { flexBasis: '47%', flexGrow: 1 }
```

I3 · Catálogo de grupos con semáforo



(a) Mockup (b) Android (c) Variante

Figura 10. Catálogo: (a) mockup, (b) todos los grupos, (c) filtro Sin cruce de horario

Elementos. Lista todos los grupos de laboratorio agrupados por curso. Cada tarjeta muestra la letra del grupo, día y hora, ambiente y docente, la insignia del semáforo (*Agotado*, *Últimos cupos*, *Disponible*), una barra de ocupación y el texto *N de M PC libres*. Arriba hay un buscador y tres filtros. Con el filtro *Sin cruce de horario* (panel c) desaparece el grupo C de Plataformas Emergentes, que coincide con la teoría de Gestión de Proyectos del jueves.

Componentes y layout. La lista es una `SectionList` con una sección por curso; `renderSectionHeader` dibuja el punto de color y el nombre del curso, y `stickySectionHeadersEnabled` lo mantiene fijo mientras se desplazan sus grupos. Los filtros son chips dentro de un `ScrollView` horizontal, de modo que pueden agregarse más sin romper el diseño. En cada tarjeta, la letra del grupo es un cuadro de 44 × 44 dp centrado con `alignItems` y `justifyContent`, el texto toma el espacio libre con `flex: 1` y la insignia queda al final de la fila.

CatalogoScreen.tsx · lista por secciones

```
<SectionList
  sections={secciones}                  // [{ curso, data: Turno[] }, ...]
  keyExtractor={(t) => t.id}
  renderItem={renderTurno}              // Tarjeta tocable con semáforo y barra
  stickySectionHeadersEnabled
  renderSectionHeader={({ section }) => (
    <View style={estilos.seccion}>{/* punto de color + nombre del curso */}</View>
  )}
  ListEmptyComponent={<Vacio icono="search-outline" titulo="Sin resultados" detalle="..." />}
/>
```

I4 · Detalle del grupo y confirmación

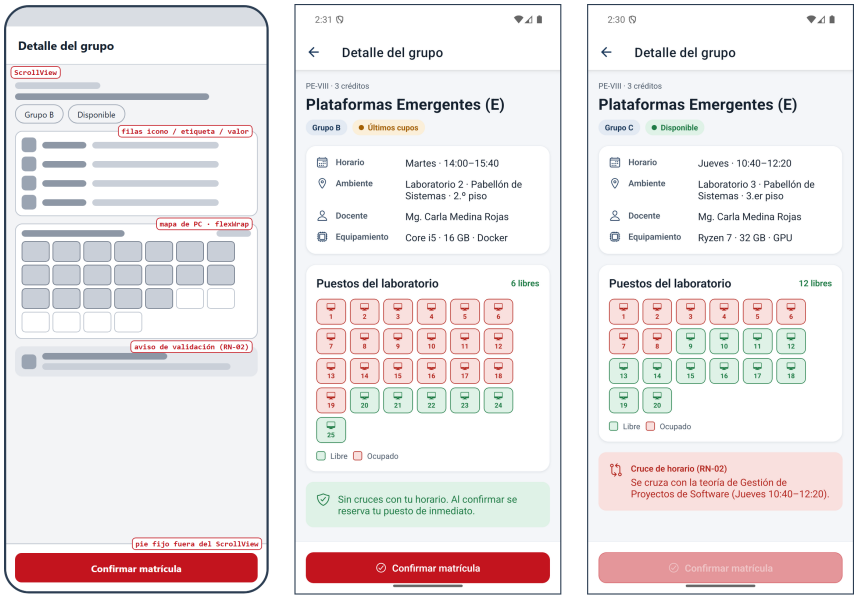


Figura 11. Detalle del grupo: (a) mockup, (b) grupo B disponible, (c) grupo C bloqueado por cruce con la teoría (RN-02)

Elementos. Se abre como hoja modal desde el catálogo. Presenta el curso, las insignias de grupo y semáforo, una tarjeta con horario, ambiente, docente y equipamiento, y el **mapa de puestos**: una celda por computadora, en rojo las ocupadas y en verde las libres. Debajo, un aviso verde confirma que no hay cruces; si los hay, el aviso rojo explica con qué curso y en qué horario, y el botón **Confirmar matrícula** queda deshabilitado (panel c).

Componentes y layout. El mapa de puestos es un `View` con `flexDirection: 'row'`, `flexWrap: 'wrap'` y `gap: 6`, cuyas celdas miden 46 × 40 dp. Con 20, 25 o 30 computadoras la cuadrícula se reacomoda sola en cualquier ancho. El botón de confirmación vive en un pie **fuera** del `ScrollView`, así siempre está visible. Al confirmar, un `ActivityIndicator` simula la latencia del servidor y luego `navigation.replace` sustituye el detalle por el pase, para que Atrás no regrese a un formulario ya enviado.

DetalleTurnoScreen.tsx · mapa de puestos

```
<View style={estilos.mapa}>   {/* flexDirection: 'row', flexWrap: 'wrap', gap: 6 */}
  {Array.from({ length: turno.cuposTotales }, (_, i) => (
    <View key={i} style={[estilos.pc, { backgroundColor: i < turno.cuposOcupados ? c.peligroSuave : c.
    exitoSuave }]}>
      <Ionicons name="desktop-outline" size={14} color={i < turno.cuposOcupados ? c.peligro : c.exito}
    />
      <Text style={estilos.numeroPc}>{i + 1}</Text>
    </View>
  ))}
</View>
// pc: { width: 46, height: 40, alignItems: 'center', justifyContent: 'center' }
```

I5 · Pase de laboratorio con QR



(a) Mockup (b) Android

Figura 12. Pase de laboratorio: (a) mockup, (b) pase emitido tras confirmar la matrícula

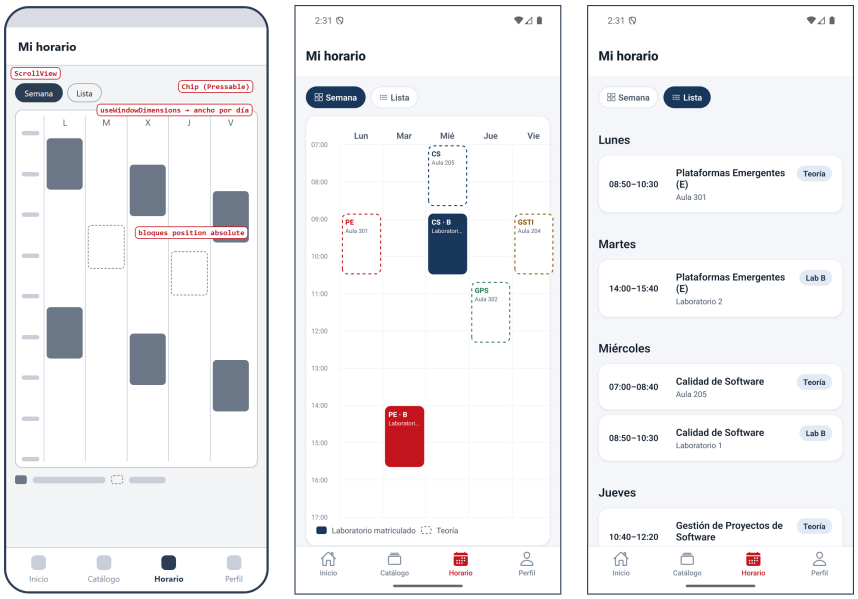
Elementos. Es la constancia digital que el estudiante muestra en la puerta del laboratorio. La parte superior, en color primario, lleva el curso, el grupo, el horario y el sello *Confirmada*; la inferior, el código QR, el código de verificación de ocho caracteres y los datos del estudiante, el ambiente y la matrícula. El botón **Compartir pase** abre la hoja nativa de compartir.

Componentes y layout. La tarjeta del pase es un `View` con `overflow: 'hidden'` dividido en dos zonas. Entre ellas, la perforación se dibuja con dos círculos del color del fondo en `position: 'absolute'` (`left: -12` y `right: -12`) y una línea discontinua. El sello usa `position: 'absolute'` con `top` y `right`. El lado del QR se calcula con `useWindowDimensions` (`Math.min(220, width - 140)`) y los datos se ordenan en dos columnas con `flexWrap` y `width: '50%'` en cada dato.

FichaQRScreen.tsx · perforación y código QR

```
<View style={estilos.perforacion}>
  <View style={[estilos.muesca, { left: -12, backgroundColor: c.fondo }]} />
  <View style={[estilos.guiones, { borderColor: c.borde }]} />
  <View style={[estilos.muesca, { right: -12, backgroundColor: c.fondo }]} />
</View>
<QRCode value={payloadQR(m)} size={ladoQR} color="#14202E" backgroundColor="#FFFFFF" />
// muesca: { position: 'absolute', width: 24, height: 24, borderRadius: 12 }
```

I6 · Horario semanal consolidado



(a) Mockup (b) Android (c) Variante

Figura 13. Horario: (a) mockup, (b) vista semanal, (c) vista de lista por día

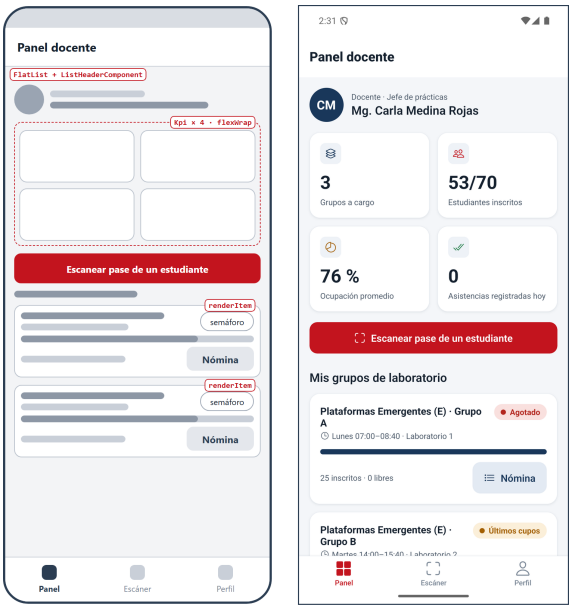
Elementos. Consolida en una semana los laboratorios matriculados (bloques rellenos con el color del curso) y las clases de teoría (bloques con borde discontinuo). El selector *Semana / Lista* cambia a una vista de tarjetas agrupadas por día, más cómoda para leer en un teléfono pequeño.

Componentes y layout. La cuadrícula se construye a mano porque React Native no tiene CSS Grid. Una columna de horas y cinco columnas de día se colocan en fila; el ancho de cada día sale de `useWindowDimensions` con un mínimo de 64 dp. Dentro de cada día, cada bloque usa `position: 'absolute'` con `top` igual a la hora de inicio y `height` igual a la duración, ambos convertidos con una constante de 58 dp por hora. Si el ancho mínimo no cabe, la cuadrícula se envuelve en un `ScrollView` horizontal.

HorarioScreen.tsx · cuadrícula con posición absoluta

```
const disponible = width - espacio.l * 2 - CANAL_HORAS;
const anchoDia = Math.max(64, disponible / DIAS_VISIBLES.length);
const desborda = anchoDia * DIAS_VISIBLES.length > disponible + 1;
...
style={{
  top: (e.inicio - INICIO) / 60 * ALTO_HORA + 1,
  height: (e.fin - e.inicio) / 60 * ALTO_HORA - 2,
  borderStyle: lab ? 'solid' : 'dashed',
}}
...
{desborda ? <ScrollView horizontal>{cuadricula}</ScrollView> : cuadricula}
```

I7 · Panel del docente



(a) Mockup (b) Android

Figura 14. Panel del docente: (a) mockup, (b) panel con los grupos a cargo

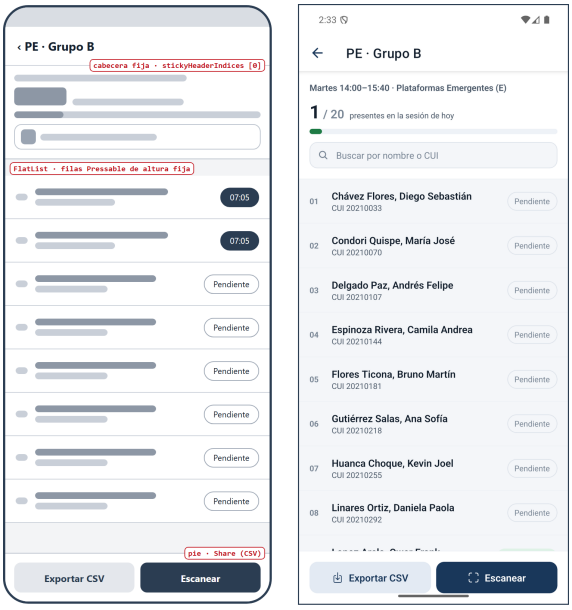
Elementos. Resume la carga del docente: grupos a cargo, estudiantes inscritos sobre la capacidad, ocupación promedio y asistencias registradas en el día. Un botón destacado lleva al escáner y debajo se listan los grupos con semáforo, barra de ocupación y acceso a la **Nómina**.

Componentes y layout. Toda la pantalla es una sola `FlatList`. El saludo, los KPI y el botón van en `ListHeaderComponent`, de modo que se desplazan junto con la lista sin anidar una lista dentro de un `ScrollView`, situación que React Native desaconseja porque anula la virtualización. Las cabeceras de cada tarjeta usan `justifyContent: 'space-between'`.

PanelDocenteScreen.tsx · cabecera dentro de la lista

```
<FlatList
  data={mios}
  keyExtractor={(t) => t.id}
  ItemSeparatorComponent={() => <View style={{ height: espacio.m }} />}
  ListHeaderComponent={<View>{/* saludo, KPI y botón Escanear */}</View>}
  renderItem={({ item }) => <Tarjeta>{/* curso, semáforo, barra y botón Nómina */}</Tarjeta>}
/>
```

I8 · Nómina y asistencia



(a) Mockup (b) Android

Figura 15. Nómina del grupo: (a) mockup, (b) dos asistencias registradas con su hora

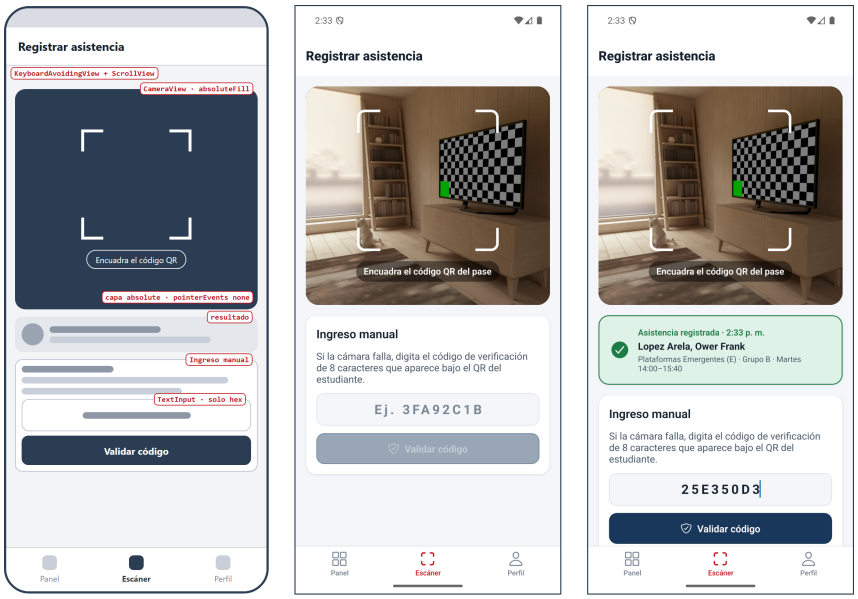
Elementos. Lista a los estudiantes del grupo ordenados alfabéticamente, con número, nombre y CUI. Tocar una fila registra la asistencia con la hora; la fila pasa de *Pendiente* a una píldora verde. El resumen superior muestra presentes sobre el total, una barra de avance y un buscador por nombre o CUI. El pie ofrece **Exportar CSV** y **Escanear**.

Componentes y layout. Es una `FlatList` cuyo primer elemento, el resumen, queda fijo con `stickyHeaderIndices={[0]}`. Las filas tienen alto constante (68 dp) para que el desplazamiento sea uniforme. Cada fila es un `Pressable` con `accessibilityRole="checkbox"` y `accessibilityState` marcado, de modo que el lector de pantalla anuncia si el estudiante ya está presente. El título de la cabecera se cambia con `navigation.setOptions`.

NominaScreen.tsx · resumen fijo y filas marcables

```
<FlatList
  data={visibles}
  keyExtractor={(f) => f.id}
  stickyHeaderIndices={[0]}
  ListHeaderComponent={<View style={estilos.resumen}>{/* presentes, barra y buscador */}</View>}
  renderItem={({ item, index }) => (
    <Pressable onPress={() => marcar(item.id)} disabled={Boolean(estado.asistencias[item.id])}
      accessibilityRole="checkbox" accessibilityState={{ checked: Boolean(estado.asistencias[item.id])
    }}>
      ...
    </Pressable>
  )}
/>
```

I9 · Escáner de asistencia



(a) Mockup (b) Android (c) Variante

Figura 16. Escáner: (a) mockup, (b) cámara activa en el emulador, (c) pase validado con el código de verificación

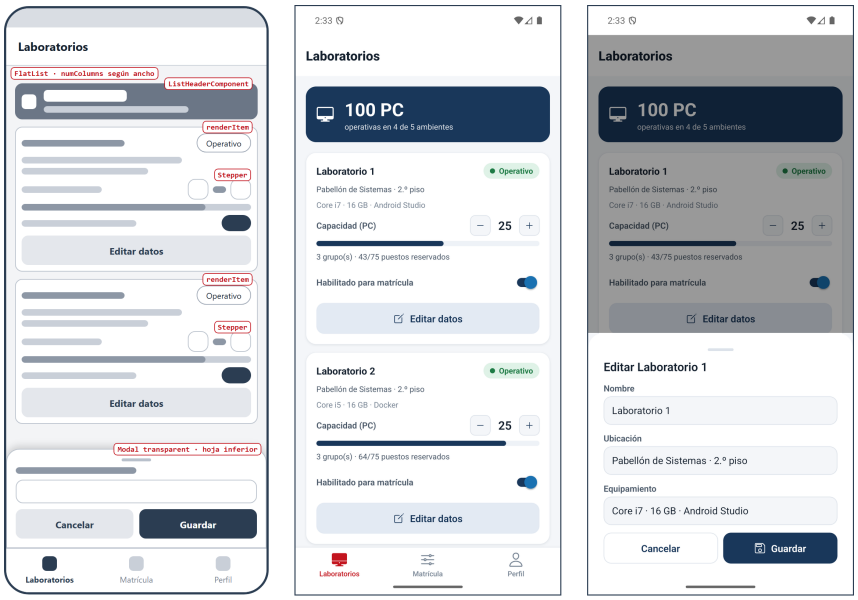
Elementos. Muestra la vista de la cámara con una guía de encuadre y lee el QR del pase. Si la cámara no está disponible, el docente puede digitar el código de verificación de ocho caracteres. El resultado aparece en una tarjeta verde con el nombre del estudiante, el curso y la hora, o en una roja si el pase es inválido o pertenece a otro docente.

Componentes y layout. El visor es un `View` de 340 dp de alto con `overflow: 'hidden'`. La `CameraView` lo cubre con `StyleSheet.absoluteFill` y encima se coloca otra capa con `absoluteFill` y `pointerEvents="none"`, que dibuja las cuatro esquinas del marco (cada una un `View` absoluto con dos bordes) y la leyenda. El campo manual normaliza lo escrito a mayúsculas hexadecimales y limita la longitud a ocho caracteres. Para evitar lecturas repetidas se bloquea el lector durante 2,5 s después de cada código.

EscanerScreen.tsx · cámara con capa superpuesta

```
<View style={estilos.visor}>
  <CameraView style={StyleSheet.absoluteFill} facing="back"
    barcodeScannerSettings={{ barcodeTypes: ['qr'] }} onBarcodeScanned={alLeer} />
  <View style={[StyleSheet.absoluteFill, estilos.capa]} pointerEvents="none">
    <View style={estilos.marco}>
      {(['si', 'sd', 'ii', 'id'] as const).map((p) => <View key={p} style={[estilos.esquina, estilos[p
    ]]} />)}
    </View>
    <Text style={estilos.ayuda}>Encuadra el código QR del pase</Text>
  </View>
</View>
```

I10 · Gestión de laboratorios



(a) Mockup (b) Android (c) Variante

Figura 17. Gestión de laboratorios: (a) mockup, (b) lista de ambientes, (c) edición en la hoja modal

Elementos. La coordinación ve el total de computadoras operativas y una tarjeta por ambiente con ubicación, equipamiento, capacidad ajustable con botones − y +, barra de puestos reservados, interruptor de habilitación y botón **Editar datos**, que abre una hoja inferior para cambiar nombre, ubicación y equipamiento.

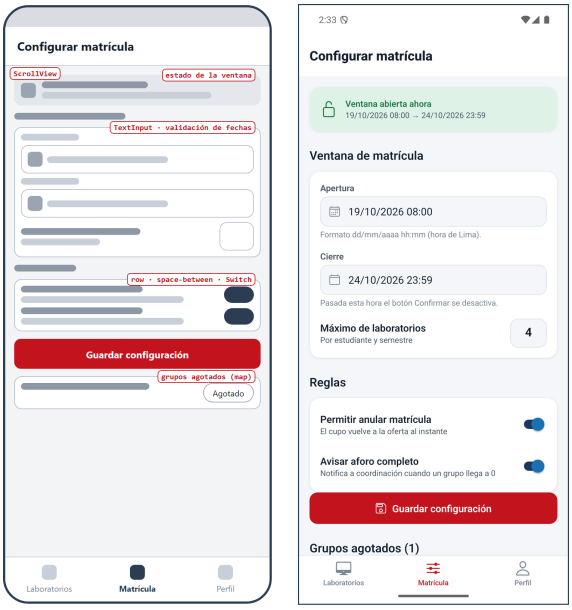
Componentes y layout. La lista es una `FlatList` con `numColumns` calculado a partir del ancho: una columna por debajo de 600 dp, dos hasta 960 dp y tres en adelante. `columnWrapperStyle` agrega el espacio entre columnas. La edición usa el `Modal` nativo con fondo transparente: un `Pressable` semitransparente ocupa la pantalla (tocarlo cierra) y la hoja con esquinas superiores redondeadas queda abajo.

LaboratoriosScreen.tsx · columnas según el ancho y edición modal

```
const { width } = useWindowDimensions();
const columnas = width < 600 ? 1 : width < 960 ? 2 : 3;

<FlatList
  key={columnas}
  data={estado.laboratorios}
  numColumns={columnas}
  columnWrapperStyle={columnas > 1 ? { gap: espacio.m } : undefined}
  ...
/>
<Modal visible={edicion !== null} animationType="slide" transparent onRequestClose={() => setEdicion(
    null)}>
```

I11 · Configuración de la matrícula



(a) Mockup (b) Android

Figura 18. Configuración de la matrícula: (a) mockup, (b) ventana abierta y reglas

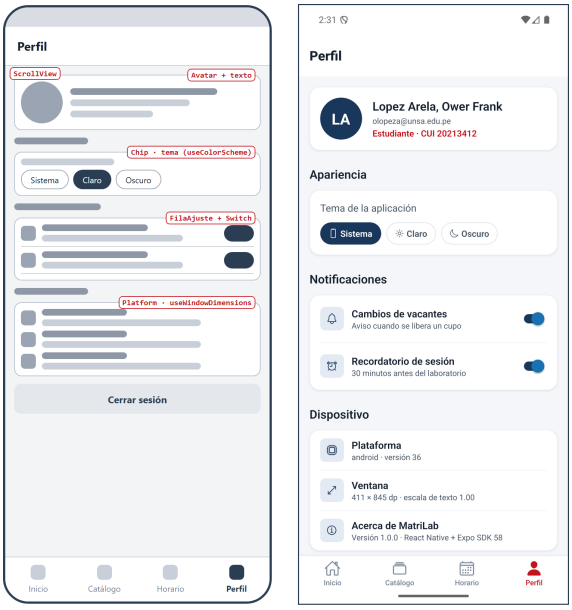
Elementos. Permite fijar la apertura y el cierre de la ventana de matrícula, el máximo de laboratorios por estudiante y dos reglas: permitir anular matrículas y avisar cuando un grupo se agota. Muestra el estado actual de la ventana y la lista de grupos agotados con la opción de solicitar un grupo adicional.

Componentes y layout. Es un formulario en un `ScrollView` con `keyboardShouldPersistTaps="handled"`, para que tocar un botón no solo cierre el teclado. Cada regla es una fila con `justifyContent: 'space-between'`: texto a la izquierda con `flex: 1` y `Switch` a la derecha. Antes de guardar se valida el formato `dd/mm/aaaa hh:mm`, que el cierre sea posterior a la apertura y que el máximo esté entre 1 y 6; el error aparece sobre el botón.

ConfiguracionScreen.tsx · validación de la ventana

```
const guardar = () => {
  const i = aISO(inicio);
  const f = aISO(fin);
  const max = Number(maximo);
  if (!i || !f) return setError('Usa el formato dd/mm/aaaa hh:mm en ambas fechas.');
  if (new Date(f) <= new Date(i)) return setError('El cierre debe ser posterior a la apertura.');
  if (!Number.isInteger(max) || max < 1 || max > 6) return setError('El máximo de laboratorios debe
    estar entre 1 y 6.');
  dispatch({ tipo: 'ACTUALIZAR_CONFIG', config: { inicio: i, fin: f, maxLaboratoriosPorEstudiante: max,
    ... } });
};
```

I12 · Perfil y ajustes



(a) Mockup (b) Android

Figura 19. Perfil y ajustes: (a) mockup, (b) perfil del estudiante

Elementos. Es común a los tres roles. Muestra los datos del usuario, el selector de tema (*Sistema*, *Claro*, *Oscuro*), dos interruptores de notificaciones, información del dispositivo y el botón **Cerrar sesión**, que reinicia la pila de navegación hacia el inicio de sesión.

Componentes y layout. Las filas de ajustes reutilizan el componente `FilaAjuste` (icono, texto con `flex: 1` y un control a la derecha). La sección *Dispositivo* evidencia el entorno de ejecución: `Platform.OS` y `Platform.Version` indican el sistema, y `useWindowDimensions` el tamaño de la ventana en dp y la escala de texto. En el emulador se lee *android · versión 36* y *411 × 914 dp*. El tema elegido se guarda en el `ThemeContext` y repinta toda la aplicación al instante.

PerfilScreen.tsx · datos del dispositivo

```
<FilaAjuste icono="hardware-chip-outline" titulo="Plataforma" detalle={`${Platform.OS} · versión ${
    String(Platform.Version)}`} />
<FilaAjuste icono="resize-outline" titulo="Ventana"
  detalle={`${Math.round(width)} × ${Math.round(height)} dp · escala de texto ${fontScale.toFixed(2)}`}
    />
```

4.7 Adaptación a la orientación y a otras plataformas

Para comprobar que el layout responde al tamaño de la ventana se giró el emulador a horizontal. `useWindowDimensions` entrega entonces unos 915 dp de ancho y la lista de laboratorios pasa de una a dos columnas sin ningún código específico para la rotación (Figura 20). Para que esto fuera posible se cambió `orientation` a `"default"` en `app.json`, ya que con `"portrait"` el sistema bloqueaba el giro.

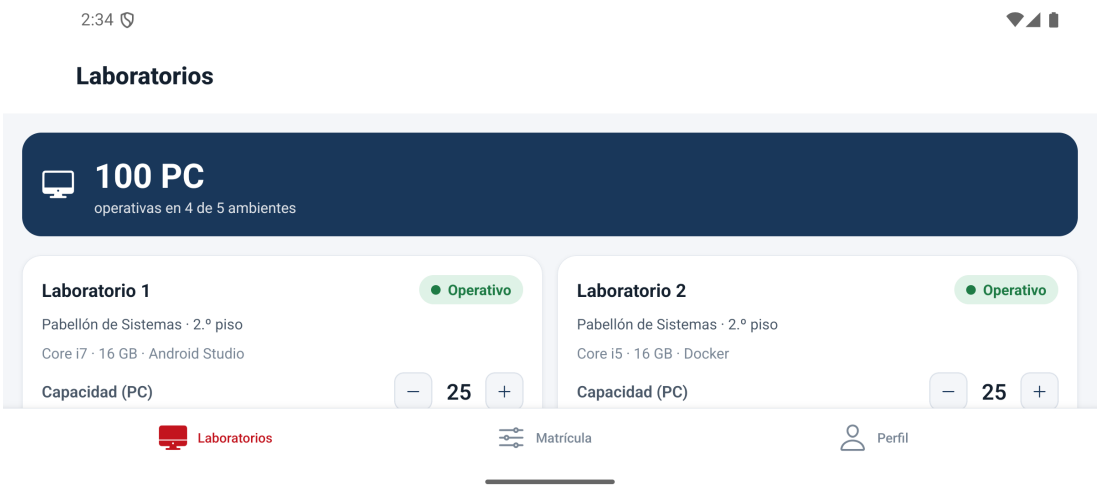
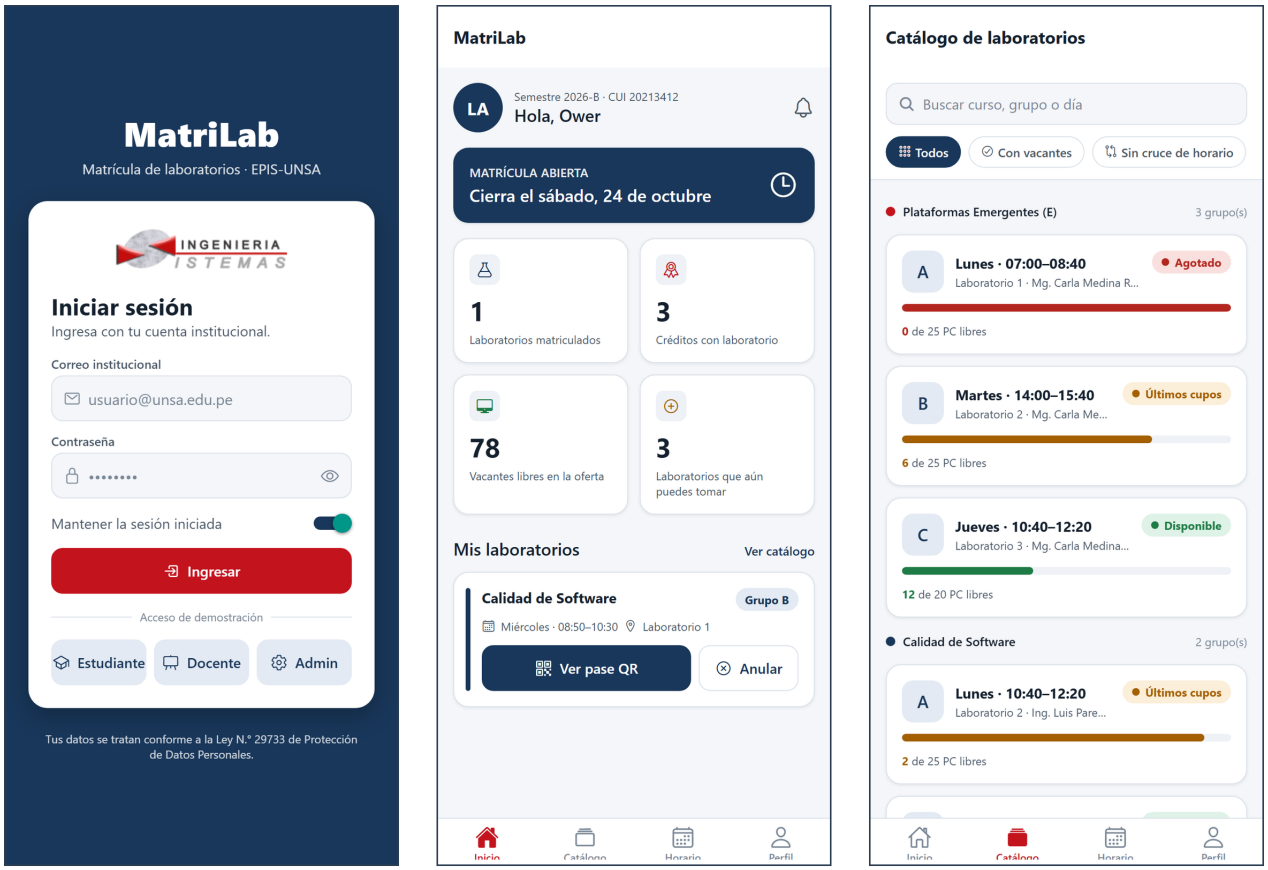


Figura 20. Gestión de laboratorios en orientación horizontal: la FlatList pasa a dos columnas

El mismo código se exportó a la web con `npx expo export --platform web`. `react-native-web` traduce los componentes al DOM y la aplicación funciona en el navegador (Figura 21); la versión publicada está en `https://owerlopez.github.io/matrilab-epis-react-native/`.



(a) Inicio de sesión **(b) Panel** **(c) Catálogo**

Figura 21. La misma aplicación ejecutándose en un navegador web: inicio de sesión, panel del estudiante y catálogo

4.8 Pruebas automatizadas y verificación

Las reglas de negocio y el reductor se verificaron con **31 pruebas unitarias** en Jest [22], [23]. Se ejecutan con `npm test` y terminan en alrededor de un segundo porque no necesitan emulador. La Tabla 13 resume qué verifica cada grupo.

Tabla 12. Pruebas unitarias agrupadas por regla verificada

Grupo	Pruebas	Qué se verifica
Conversión de horas	1	`'07:00'` equivale a 420 minutos y 640 minutos a `'10:40'`.
RN-02 cruce de horario	3	Solape el mismo día; bloques contiguos no se cruzan; días distintos no se cruzan.
Semáforo	1	Rojo sin cupo, ámbar con menos del 30 % libre y verde en otro caso.
validarMatricula	8	Acepta un grupo libre; rechaza cruce con teoría y con otro laboratorio, mismo grupo, mismo curso, grupo agotado, ventana cerrada y límite de laboratorios.
Reserva atómica (RF-05)	4	Suma un cupo sin mutar el arreglo; no reserva sin cupo; nunca supera el aforo con más solicitudes que vacantes; liberar no deja negativos.
Pase QR	3	El contenido generado se verifica; un QR alterado se rechaza; la huella es estable y de 8 caracteres.
Horario	1	Los bloques se ordenan por día y luego por hora.
Ventana y nómina	3	La ventana de demostración está abierta; la nómina tiene tantas filas como cupos; el CSV usa `;`.
Reductor	7	Matricular y anular actualizan cupos; sin sesión no se matricula; cerrar sesión; acciones de administración; código de verificación; la asistencia se marca una sola vez.

Resultado de npm run test:cobertura

```
PASS __tests__/reglas.test.ts
---------------|---------|----------|---------|---------|-------------------
File           | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
---------------|---------|----------|---------|---------|-------------------
All files      |    86.3 |    87.83 |   78.57 |   92.85 |
 domain        |   98.88 |    95.55 |     100 |     100 |
  datos.ts     |     100 |      100 |     100 |     100 |
  reglas.ts    |   98.52 |    95.34 |     100 |     100 | 132,140
 state         |   66.07 |    75.86 |      50 |   78.37 |
  AppStore.tsx |   66.07 |    75.86 |      50 |   78.37 | 76-84,95-99
---------------|---------|----------|---------|---------|-------------------
Tests:       31 passed, 31 total
Time:        1.088 s
```

El dominio alcanza 100 % de líneas cubiertas. En `AppStore.tsx` quedan sin cubrir el proveedor de contexto y algunos selectores que solo se ejercitan al renderizar pantallas; esas rutas se verificaron en el recorrido manual y automatizado sobre Android descrito a continuación. Además, `npm run typecheck` (`tsc --noEmit`) termina sin errores en modo estricto.

4.9 Ejecución en Android y recorrido automatizado de capturas

La aplicación se ejecutó en el emulador con `npx expo start --android`, que inicia el servidor Metro y abre el proyecto en Expo Go. Para que las capturas del informe fueran reproducibles se escribió el script `herramientas/capturas-android.mjs`. El script recorre la aplicación por adb: con `uiautomator dump` obtiene el árbol de la pantalla, localiza los controles por su `accessibilityLabel` y los toca, escribe en los campos y guarda una captura de cada estado. Como depende de las etiquetas de accesibilidad y no de coordenadas, el mismo recorrido sirve en otro tamaño de pantalla y funciona además como prueba de humo de la navegación completa: iniciar sesión con cada rol, matricularse, ver el pase, marcar asistencia, validar el código en el escáner y girar la pantalla.

herramientas/capturas-android.mjs · búsqueda de controles por accesibilidad

```
async function buscar(etiqueta, { exacto = true, intentos = 20, desplazar = false } = {}) {
  for (let i = 0; i < intentos; i++) {
    const lista = nodos();
    const coincide = (v) => (exacto ? v === etiqueta : v.includes(etiqueta));
    // Primero la etiqueta de accesibilidad (el control) y luego el texto visible (p. ej. un rótulo).
    const n = lista.find((x) => coincide(x.desc)) ?? lista.find((x) => coincide(x.texto));
    if (n && n.y > 0 && n.y < 2380) return n;
    if (desplazar) adb('shell', 'input', 'swipe', '540', '1700', '540', '800', '350');
    await dormir(700);
  }
  throw new Error(`No se encontró «${etiqueta}» en pantalla`);
}
```

4.10 Registro de cambios

La guía pide dejar constancia de las modificaciones realizadas durante el trabajo. La Tabla 13 registra los cambios que surgieron al probar la aplicación en el emulador; en el documento DOCX cada cambio lleva además un comentario de revisión.

Tabla 13. Cambios realizados durante el desarrollo y su motivo

N.°	Archivo	Cambio	Motivo
1	LoginScreen.tsx	Se agregó `<StatusBar style="light/>`.	La hora y los iconos del sistema quedaban oscuros sobre el fondo azul.
2	LaboratoriosScreen.tsx	Una columna por debajo de 600 dp (antes dos desde 400 dp).	En el Pixel 7a (411 dp) las tarjetas quedaban comprimidas y el stepper se cortaba.
3	app.json	`orientation` pasó de `"portrait"` a `"default"`.	El sistema bloqueaba el giro y no se podía comprobar el layout horizontal.
4	EscanerScreen.tsx	El campo manual solo acepta hexadecimal en mayúsculas y 8 caracteres.	Evitar rechazos por minúsculas o espacios al dictar el código.
5	NominaScreen.tsx	Se retiró `getItemLayout`.	La cabecera fija tiene alto variable y desplazaba los índices calculados.
6	tsconfig.json	Se declararon los tipos `jest` y `node`.	TypeScript 6 no incluye tipos globales por defecto y `tsc` fallaba en las pruebas.
7	reglas.test.ts	Se agregaron 8 pruebas (de 23 a 31).	Cubrir los motivos YA_MATRICULADO y cruce con laboratorio, y las acciones del reductor.
8	app.json	Se agregó `experiments.baseUrl` para la web.	Publicar la exportación web en GitHub Pages bajo una subruta.

Cambio 2: la regla de columnas se ajustó después de ver las tarjetas cortadas en el emulador; el umbral de 600 dp coincide con el límite habitual entre teléfono y tableta pequeña.

Cambio 3: con la orientación libre, la Figura 20 demuestra la adaptación automática de la rejilla.

Cambios 6 y 7: el comando `npm run typecheck` y las 31 pruebas se ejecutan sin errores después de estos ajustes.

4.11 Repositorio, ejecución y versión web

Todo el código, las capturas y este informe están en el repositorio `https://github.com/OwerLopez/matrilab-epis-react-native`. La versión web se publica automáticamente con GitHub Actions en `https://owerlopez.github.io/matrilab-epis-react-native/`. La carpeta `informe/` contiene las fuentes LaTeX de este documento, la versión DOCX, los diagramas en HTML y los scripts que los generan. Para ejecutar el proyecto:

Comandos de instalación y ejecución

```
git clone https://github.com/OwerLopez/matrilab-epis-react-native.git
cd matrilab-epis-react-native
npm install
npx expo start            # escanear el QR con Expo Go o pulsar «a» para el emulador Android
npm test                  # 31 pruebas unitarias
npm run typecheck         # verificación de tipos
npm run export:web        # versión web en dist/
```

Credenciales de demostración: `olopeza@unsa.edu.pe`, `cmedina@unsa.edu.pe` o `laboratorios.epis@unsa.edu.pe` con la contraseña `unsa2026`, o los botones *Estudiante*, *Docente* y *Admin* del inicio de sesión.

5. APRECIACIÓN DE LA HERRAMIENTA

La valoración se basa en lo que el equipo observó al construir y probar MatriLab con React Native y Expo, no en afirmaciones generales sobre el framework.

5.1 Características favorables

A. Interfaces nativas reales a partir de un solo código

Cada componente del núcleo se convierte en una vista nativa del sistema [2]. En la práctica, el `Switch` de la configuración es el interruptor de Android, el teclado y la hoja de compartir son los del teléfono, el `Alert` es el diálogo del sistema y `native-stack` usa las transiciones y el gesto de retroceso nativos. Con ese mismo código, sin una sola línea específica, la aplicación se exportó a la web (Figura 21). Las diferencias por plataforma que sí hubo que escribir fueron mínimas y explícitas: el `behavior` de `KeyboardAvoidingView` y el `android_ripple` de `Pressable`. Para un proyecto universitario que debe llegar a estudiantes con Android y con iPhone, esto reduce a la mitad el trabajo frente al desarrollo nativo separado.

B. Ciclo de desarrollo rápido y ecosistema JavaScript

Con Expo Go no fue necesario compilar proyectos de Android Studio: el emulador cargaba el paquete desde Metro y *Fast Refresh* aplicaba cada cambio de estilo en uno o dos segundos conservando el estado de la pantalla, lo que permitió ajustar márgenes, columnas y colores viendo el resultado al instante. El ecosistema de JavaScript aportó piezas maduras: TypeScript detectó errores de navegación entre pantallas antes de ejecutarlas, `react-native-qrcode-svg` resolvió el QR en una línea y Jest probó las reglas de negocio en un segundo. El mismo lenguaje sirvió para la aplicación, las pruebas y el script de capturas.

5.2 Características desfavorables

A. Dependencia de la compatibilidad entre versiones

Una aplicación React Native depende de una cadena de versiones que deben coincidir: Expo SDK, React Native, React, cada librería con código nativo (`react-native-screens`, `react-native-svg`, `expo-camera`) y la versión de Expo Go instalada en el teléfono, que solo abre proyectos de su mismo SDK. Fue obligatorio instalar las dependencias con `npx expo install` para no romper esa cadena. El ritmo de cambios también se notó: el `SafeAreaView` del núcleo figura como obsoleto [16], la Nueva Arquitectura cambió el comportamiento interno de varias librerías y TypeScript 6 dejó de incluir tipos globales por defecto, lo que obligó a corregir `tsconfig.json` (cambio 6). En un proyecto que dure varios semestres, actualizar dependencias requiere tiempo dedicado.

B. Layout limitado a Flexbox y diferencias entre plataformas

React Native solo ofrece Flexbox y posición absoluta: no hay CSS Grid ni *media queries*. La cuadrícula del horario tuvo que calcularse a mano con `useWindowDimensions` y `position: 'absolute'`, y la rejilla adaptable de laboratorios obligó a volver a montar la `FlatList` con `key` porque `numColumns` no puede cambiar en caliente. Algunos componentes se comportan distinto en cada sistema (`KeyboardAvoidingView`, sombras, ripple) y hay que probar en ambos. Además, la depuración de lo que ocurre del lado nativo exige conocer las herramientas de Android: los permisos de cámara de Expo Go, el teclado del emulador y la rotación tuvieron que resolverse con adb.

5.3 Valoración general

Tabla 14. Resumen de la apreciación de React Native con Expo

Aspecto	Valoración	Evidencia en el laboratorio
Componentes propios de la plataforma	Favorable	Switch, Alert, Share, teclado y pila de navegación nativos en Android.
Productividad	Favorable	Fast Refresh, Expo Go, TypeScript y pruebas del dominio en 1 s.
Mantenimiento de versiones	Desfavorable	Cadena SDK–RN–librerías–Expo Go; APIs obsoletas; ajuste de tsconfig.
Sistema de layout	Desfavorable	Sin Grid ni media queries; horario y rejilla calculados a mano.

6. CONCLUSIONES

- Se desarrolló MatriLab EPIS con React Native 0.88 y Expo SDK 58: doce interfaces para tres roles que se ejecutan como aplicación nativa en Android y como aplicación web con el mismo código, usando solo componentes del núcleo del framework para la organización de la interfaz.
- El árbol de problemas y el árbol de objetivos permitieron justificar cada pantalla: los cinco medios fundamentales se tradujeron en trece funcionalidades y estas en las doce interfaces, con trazabilidad explícita (Tabla 4).
- La investigación de componentes mostró que el layout en React Native descansa en Flexbox con valores por defecto distintos a los de la web, en listas virtualizadas para colecciones grandes y en `position: 'absolute'` y `useWindowDimensions` para superposiciones y adaptación. La rejilla de laboratorios pasó de una a dos columnas al girar la pantalla sin código adicional.
- Separar el dominio en funciones puras permitió verificar las reglas de negocio con 31 pruebas que cubren el 100 % de las líneas del dominio, y el recorrido automatizado por accesibilidad sirvió como prueba de humo de la navegación completa en Android.
- Frente a FlutterFlow (LAB 02), React Native exige escribir más código, pero da control total sobre el layout y el estado y produce interfaces que usan los controles nativos de cada sistema; el costo principal es mantener la compatibilidad de versiones del ecosistema.

7. ANEXOS: CÓDIGO DE LA APLICACIÓN

Se incluye el código fuente completo de la aplicación tal como está en el repositorio. Cada archivo de pantalla comienza con un comentario que explica su layout.

A.1 Punto de entrada y proveedores

App.tsx

```tsx
import { StatusBar } from 'expo-status-bar';
import { SafeAreaProvider } from 'react-native-safe-area-context';
import RootNavigator from './src/navigation/RootNavigator';
import { ProveedorApp } from './src/state/AppStore';
import { ProveedorTema, useTema } from './src/theme/ThemeContext';

function BarraEstado() {
  const { oscuro } = useTema();
  return <StatusBar style={oscuro ? 'light' : 'dark'} />;
}

export default function App() {
  return (
    <SafeAreaProvider>
      <ProveedorTema>
        <ProveedorApp>
          <BarraEstado />
          <RootNavigator />
        </ProveedorApp>
      </ProveedorTema>
    </SafeAreaProvider>
  );
}
```

A.2 Configuración de Expo

app.json

```json
{
  "expo": {
    "name": "MatriLab EPIS",
    "slug": "matrilab-epis",
    "version": "1.0.0",
    "orientation": "default",
    "icon": "./assets/icon.png",
    "userInterfaceStyle": "automatic",
    "ios": {
      "supportsTablet": true,
      "bundleIdentifier": "pe.edu.unsa.epis.matrilab"
    },
    "android": {
      "adaptiveIcon": {
        "backgroundColor": "#19375A",
        "foregroundImage": "./assets/android-icon-foreground.png",
        "backgroundImage": "./assets/android-icon-background.png",
        "monochromeImage": "./assets/android-icon-monochrome.png"
      },
      "predictiveBackGestureEnabled": false,
      "package": "pe.edu.unsa.epis.matrilab"
    },
    "web": {
      "favicon": "./assets/favicon.png",
      "bundler": "metro",
```

```
      "output": "single"
    },
    "scheme": "matrilab",
    "description": "Matrícula de laboratorios de la EPIS-UNSA: catálogo con vacantes en vivo, validación de cruces,
      pase QR y control de asistencia.",
    "plugins": [
      [
        "expo-camera",
        {
          "cameraPermission": "MatriLab usa la cámara para leer los pases QR de asistencia."
        }
      ]
    ],
    "experiments": {
      "baseUrl": "/matrilab-epis-react-native"
    }
  }
}
```

A.3 Dependencias y scripts

package.json

```
{
  "name": "matrilab-epis",
  "version": "1.0.0",
  "main": "index.ts",
  "dependencies": {
    "@expo/metro-runtime": "~58.0.14",
    "@expo/vector-icons": "^15.0.2",
    "@react-navigation/bottom-tabs": "^7.21.0",
    "@react-navigation/native": "^7.5.1",
    "@react-navigation/native-stack": "^7.21.0",
    "expo": "~58.0.16",
    "expo-camera": "~58.0.11",
    "expo-linking": "~58.0.13",
    "expo-status-bar": "~58.0.3",
    "react": "19.3.0",
    "react-dom": "19.3.0",
    "react-native": "0.88.0",
    "react-native-qrcode-svg": "^6.3.26",
    "react-native-safe-area-context": "~5.10.1",
    "react-native-screens": "~4.30.0",
    "react-native-svg": "15.16.0",
    "react-native-web": "^0.21.4"
  },
  "devDependencies": {
    "@types/jest": "^30.0.0",
    "@types/react": "~19.3.0",
    "jest": "^30.5.2",
    "jest-expo": "^58.0.11",
    "typescript": "~6.0.3"
  },
  "scripts": {
    "start": "expo start",
    "android": "expo start --android",
    "ios": "expo start --ios",
    "web": "expo start --web",
    "test": "jest",
    "test:cobertura": "jest --coverage",
    "typecheck": "tsc --noEmit",
    "export:web": "expo export --platform web"
  },
  "private": true,
  "description": "Matrícula de laboratorios de la EPIS-UNSA: catálogo con vacantes en vivo, validación de cruces,
```

```
    pase QR y control de asistencia.",
  "jest": {
    "preset": "jest-expo",
    "testMatch": [
      "**/__tests__/**/*.test.ts?(x)"
    ]
  }
}
```

A.4 Modelo de dominio

src/domain/types.ts

```ts
/** Modelo de dominio de MatriLab EPIS-UNSA (heredado de los laboratorios 01 y 02). */

export type Rol = 'estudiante' | 'docente' | 'admin';

export type Dia = 'LUN' | 'MAR' | 'MIE' | 'JUE' | 'VIE' | 'SAB';

export interface Usuario {
  id: string;
  nombre: string;
  correo: string;
  rol: Rol;
  cui?: string;
  semestre?: number;
}

export interface Laboratorio {
  id: string;
  nombre: string;
  pabellon: string;
  capacidadPcs: number;
  operativo: boolean;
  equipamiento: string;
}

export interface Curso {
  id: string;
  codigo: string;
  nombre: string;
  creditos: number;
  color: string;
}

/** Bloque horario en minutos desde las 00:00 para comparar sin ambigüedad. */
export interface Bloque {
  dia: Dia;
  inicio: number;
  fin: number;
}

export interface Turno extends Bloque {
  id: string;
  cursoId: string;
  grupo: string;
  docenteId: string;
  laboratorioId: string;
  cuposTotales: number;
  cuposOcupados: number;
}

/** Clase de teoría ya registrada en el horario del estudiante (no se matricula aquí). */
export interface ClaseTeoria extends Bloque {
  id: string;
```

```
  cursoId: string;
  aula: string;
}

export interface Matricula {
  id: string;
  estudianteId: string;
  turnoId: string;
  fecha: string;
  codigo: string;
  estado: 'confirmada' | 'anulada';
}

export interface Asistencia {
  id: string;
  matriculaId: string;
  fecha: string;
  hora: string;
  validadoPor: string;
}

export interface ConfigMatricula {
  inicio: string;
  fin: string;
  maxLaboratoriosPorEstudiante: number;
  permitirAnulacion: boolean;
  notificarAforoCompleto: boolean;
}

export type Semaforo = 'verde' | 'ambar' | 'rojo';
```

A.5 Reglas de negocio

src/domain/reglas.ts

```
/**
 * Reglas de negocio puras de MatriLab. No dependen de React ni de la plataforma,
 * por eso se prueban con Jest sin emulador (ver __tests__/reglas.test.ts).
 */
import type { Bloque, ClaseTeoria, ConfigMatricula, Dia, Matricula, Semaforo, Turno } from './types';

export const DIAS: Dia[] = ['LUN', 'MAR', 'MIE', 'JUE', 'VIE', 'SAB'];
export const NOMBRE_DIA: Record<Dia, string> = {
  LUN: 'Lunes', MAR: 'Martes', MIE: 'Miércoles', JUE: 'Jueves', VIE: 'Viernes', SAB: 'Sábado',
};

/** "07:00" -> 420 */
export function aMinutos(hora: string): number {
  const [h, m] = hora.split(':').map(Number);
  return h * 60 + m;
}

/** 420 -> "07:00" */
export function aHora(minutos: number): string {
  const h = Math.floor(minutos / 60).toString().padStart(2, '0');
  const m = (minutos % 60).toString().padStart(2, '0');
  return `${h}:${m}`;
}

export function rango(b: Bloque): string {
  return `${aHora(b.inicio)}–${aHora(b.fin)}`;
}

/** RN-02: dos bloques se cruzan si comparten día y sus intervalos se solapan (los extremos no cuentan). */
export function haySolape(a: Bloque, b: Bloque): boolean {
```

```
  return a.dia === b.dia && a.inicio < b.fin && b.inicio < a.fin;
}

export function cuposLibres(t: Turno): number {
  return Math.max(0, t.cuposTotales - t.cuposOcupados);
}

/** Semáforo de vacantes: rojo sin cupo, ámbar con menos del 30 % libre, verde en otro caso. */
export function semaforo(t: Turno): Semaforo {
  const libres = cuposLibres(t);
  if (libres === 0) return 'rojo';
  return libres / t.cuposTotales < 0.3 ? 'ambar' : 'verde';
}

export function ventanaAbierta(config: ConfigMatricula, ahora: Date): boolean {
  const t = ahora.getTime();
  return t >= new Date(config.inicio).getTime() && t <= new Date(config.fin).getTime();
}

export type ResultadoValidacion =
  | { ok: true }
  | { ok: false; motivo: 'VENTANA_CERRADA' | 'SIN_CUPO' | 'YA_MATRICULADO' | 'MISMO_CURSO' | 'LIMITE' | 'CRUCE';
     detalle: string };

export interface ContextoValidacion {
  turno: Turno;
  turnos: Turno[];
  teoria: ClaseTeoria[];
  matriculasActivas: Matricula[];
  config: ConfigMatricula;
  ahora: Date;
  nombreCurso: (cursoId: string) => string;
}

/** Valida una solicitud de matrícula en el mismo orden del proceso BPMN definido en el LAB 01. */
export function validarMatricula(c: ContextoValidacion): ResultadoValidacion {
  if (!ventanaAbierta(c.config, c.ahora)) {
    return { ok: false, motivo: 'VENTANA_CERRADA', detalle: 'La ventana de matrícula no está abierta.' };
  }
  const propios = c.matriculasActivas
    .map((m) => c.turnos.find((t) => t.id === m.turnoId))
    .filter((t): t is Turno => Boolean(t));

  if (propios.some((t) => t.id === c.turno.id)) {
    return { ok: false, motivo: 'YA_MATRICULADO', detalle: 'Ya estás matriculado en este grupo.' };
  }
  if (propios.some((t) => t.cursoId === c.turno.cursoId)) {
    return { ok: false, motivo: 'MISMO_CURSO', detalle: 'Ya tienes un grupo de laboratorio de este curso.' };
  }
  if (propios.length >= c.config.maxLaboratoriosPorEstudiante) {
    return { ok: false, motivo: 'LIMITE', detalle: `El máximo es ${c.config.maxLaboratoriosPorEstudiante}
     laboratorios por estudiante.` };
  }
  const cruceLab = propios.find((t) => haySolape(t, c.turno));
  if (cruceLab) {
    return { ok: false, motivo: 'CRUCE', detalle: `Se cruza con el laboratorio de ${c.nombreCurso(cruceLab.cursoId
     )} (${NOMBRE_DIA[cruceLab.dia]} ${rango(cruceLab)}).` };
  }
  const cruceTeoria = c.teoria.find((t) => haySolape(t, c.turno));
  if (cruceTeoria) {
    return { ok: false, motivo: 'CRUCE', detalle: `Se cruza con la teoría de ${c.nombreCurso(cruceTeoria.cursoId)}
      (${NOMBRE_DIA[cruceTeoria.dia]} ${rango(cruceTeoria)}).` };
  }
  if (cuposLibres(c.turno) === 0) {
    return { ok: false, motivo: 'SIN_CUPO', detalle: 'El grupo no tiene vacantes disponibles.' };
  }
```

```
  return { ok: true };
}

/**
 * RF-05: reserva el cupo de forma atómica sobre el arreglo inmutable de turnos.
 * Si entre la consulta y la confirmación el cupo se agotó, no modifica nada.
 */
export function reservarCupo(turnos: Turno[], turnoId: string): { turnos: Turno[]; reservado: boolean } {
  const objetivo = turnos.find((t) => t.id === turnoId);
  if (!objetivo || cuposLibres(objetivo) === 0) return { turnos, reservado: false };
  return {
    reservado: true,
    turnos: turnos.map((t) => (t.id === turnoId ? { ...t, cuposOcupados: t.cuposOcupados + 1 } : t)),
  };
}

export function liberarCupo(turnos: Turno[], turnoId: string): Turno[] {
  return turnos.map((t) => (t.id === turnoId && t.cuposOcupados > 0 ? { ...t, cuposOcupados: t.cuposOcupados - 1 }
      : t));
}

/** Huella FNV-1a de 32 bits: identifica el pase sin depender de librerías nativas. No es un hash criptográfico.
    */
export function huella(texto: string): string {
  let h = 0x811c9dc5;
  for (let i = 0; i < texto.length; i++) {
    h ^= texto.charCodeAt(i);
    h = Math.imul(h, 0x01000193) >>> 0;
  }
  return h.toString(16).toUpperCase().padStart(8, '0');
}

/** Contenido del código QR: "MATRILAB|<matricula>|<turno>|<estudiante>|<huella>". */
export function payloadQR(m: Pick<Matricula, 'id' | 'turnoId' | 'estudianteId'>): string {
  const base = `MATRILAB|${m.id}|${m.turnoId}|${m.estudianteId}`;
  return `${base}|${huella(base)}`;
}

/** Comprueba que un QR leído tenga el formato esperado y una huella íntegra. */
export function verificarQR(texto: string): { valido: boolean; matriculaId?: string } {
  const partes = texto.split('|');
  if (partes.length !== 5 || partes[0] !== 'MATRILAB') return { valido: false };
  const base = partes.slice(0, 4).join('|');
  return huella(base) === partes[4] ? { valido: true, matriculaId: partes[1] } : { valido: false };
}

/** RF-10: nómina en CSV con separador ";" (Excel en español lo abre sin asistente). */
export function nominaCSV(filas: { cui: string; nombre: string; correo: string; asistio: boolean }[]): string {
  const cab = 'CUI;Apellidos y nombres;Correo;Asistencia';
  const cuerpo = filas.map((f) => [f.cui, f.nombre, f.correo, f.asistio ? 'Presente' : 'Pendiente'].join(';'));
  return [cab, ...cuerpo].join('\n');
}

/** Ordena bloques por día y hora para listas cronológicas. */
export function ordenarBloques<T extends Bloque>(bloques: T[]): T[] {
  return [...bloques].sort((a, b) => DIAS.indexOf(a.dia) - DIAS.indexOf(b.dia) || a.inicio - b.inicio);
}
```

A.6 Datos de demostración 2026-B

src/domain/datos.ts

```ts
/**
 * Datos de demostración del semestre 2026-B. Los nombres de estudiantes y docentes son ficticios,
 * salvo el de la cuenta de demostración del estudiante, que corresponde al equipo del laboratorio.
 */
import { aMinutos } from './reglas';
import type { ClaseTeoria, ConfigMatricula, Curso, Dia, Laboratorio, Matricula, Turno, Usuario } from './types';

export const CURSOS: Curso[] = [
  { id: 'PE', codigo: 'PE-VIII', nombre: 'Plataformas Emergentes (E)', creditos: 3, color: '#C3141E' },
  { id: 'CS', codigo: 'CS-VIII', nombre: 'Calidad de Software', creditos: 3, color: '#19375A' },
  { id: 'GPS', codigo: 'GPS-VIII', nombre: 'Gestión de Proyectos de Software', creditos: 3, color: '#2D7D5A' },
  { id: 'GSTI', codigo: 'GSTI-VIII', nombre: 'Gestión de Servicios de TI', creditos: 3, color: '#8A5A00' },
];

export const LABORATORIOS: Laboratorio[] = [
  { id: 'L1', nombre: 'Laboratorio 1', pabellon: 'Pabellón de Sistemas · 2.º piso', capacidadPcs: 25, operativo: true, equipamiento: 'Core i7 · 16 GB · Android Studio' },
  { id: 'L2', nombre: 'Laboratorio 2', pabellon: 'Pabellón de Sistemas · 2.º piso', capacidadPcs: 25, operativo: true, equipamiento: 'Core i5 · 16 GB · Docker' },
  { id: 'L3', nombre: 'Laboratorio 3', pabellon: 'Pabellón de Sistemas · 3.er piso', capacidadPcs: 20, operativo: true, equipamiento: 'Ryzen 7 · 32 GB · GPU' },
  { id: 'L4', nombre: 'Laboratorio 4', pabellon: 'Pabellón de Sistemas · 3.er piso', capacidadPcs: 25, operativo: false, equipamiento: 'Core i5 · 8 GB · en mantenimiento' },
  { id: 'CC', nombre: 'Centro de Cómputo', pabellon: 'Biblioteca de Ingenierías', capacidadPcs: 30, operativo: true, equipamiento: 'Core i7 · 16 GB · Office' },
];

export const USUARIOS: Usuario[] = [
  { id: 'E01', nombre: 'Lopez Arela, Ower Frank', correo: 'olopeza@unsa.edu.pe', rol: 'estudiante', cui: '20213412', semestre: 8 },
  { id: 'D01', nombre: 'Mg. Carla Medina Rojas', correo: 'cmedina@unsa.edu.pe', rol: 'docente' },
  { id: 'D02', nombre: 'Ing. Luis Paredes Vega', correo: 'lparedes@unsa.edu.pe', rol: 'docente' },
  { id: 'D03', nombre: 'Dr. Jorge Quispe Huanca', correo: 'jquispe@unsa.edu.pe', rol: 'docente' },
  { id: 'A01', nombre: 'Coordinación de Laboratorios', correo: 'laboratorios.epis@unsa.edu.pe', rol: 'admin' },
];

const t = (id: string, cursoId: string, grupo: string, dia: Dia, ini: string, fin: string, docenteId: string, laboratorioId: string, ocupados: number, total = 25): Turno => ({
  id, cursoId, grupo, dia, inicio: aMinutos(ini), fin: aMinutos(fin), docenteId, laboratorioId, cuposTotales: total, cuposOcupados: ocupados,
});

export const TURNOS: Turno[] = [
  t('PE-A', 'PE', 'A', 'LUN', '07:00', '08:40', 'D01', 'L1', 25),
  t('PE-B', 'PE', 'B', 'MAR', '14:00', '15:40', 'D01', 'L2', 19),
  t('PE-C', 'PE', 'C', 'JUE', '10:40', '12:20', 'D01', 'L3', 8, 20),
  t('CS-A', 'CS', 'A', 'LUN', '10:40', '12:20', 'D02', 'L2', 23),
  t('CS-B', 'CS', 'B', 'MIE', '08:50', '10:30', 'D02', 'L1', 12),
  t('GPS-A', 'GPS', 'A', 'MAR', '07:00', '08:40', 'D03', 'L3', 17, 20),
  t('GPS-B', 'GPS', 'B', 'VIE', '14:00', '15:40', 'D03', 'L1', 6),
  t('GSTI-A', 'GSTI', 'A', 'MIE', '14:00', '15:40', 'D02', 'L2', 21),
  t('GSTI-B', 'GSTI', 'B', 'JUE', '07:00', '08:40', 'D03', 'CC', 11, 30),
];

/** Teoría ya inscrita del estudiante de demostración: produce cruces reales con algunos grupos. */
export const TEORIA: ClaseTeoria[] = [
  { id: 'T-PE', cursoId: 'PE', dia: 'LUN', inicio: aMinutos('08:50'), fin: aMinutos('10:30'), aula: 'Aula 301' },
  { id: 'T-CS', cursoId: 'CS', dia: 'MIE', inicio: aMinutos('07:00'), fin: aMinutos('08:40'), aula: 'Aula 205' },
  { id: 'T-GPS', cursoId: 'GPS', dia: 'JUE', inicio: aMinutos('10:40'), fin: aMinutos('12:20'), aula: 'Aula 302' },
  { id: 'T-GSTI', cursoId: 'GSTI', dia: 'VIE', inicio: aMinutos('08:50'), fin: aMinutos('10:30'), aula: 'Aula 204' },
```

```
];

/** Estudiante de demostración ya matriculado en Calidad de Software (grupo B). */
export const MATRICULAS: Matricula[] = [
  { id: 'M-0001', estudianteId: 'E01', turnoId: 'CS-B', fecha: '2026-10-19T08:03:12-05:00', codigo: 'M-0001',
    estado: 'confirmada' },
];

/**
 * La ventana de matrícula de demostración se calcula respecto de la fecha actual
 * (abre ayer a las 08:00 y cierra en cuatro días) para que la app siempre pueda probarse.
 */
export function configInicial(ahora: Date = new Date()): ConfigMatricula {
  const inicio = new Date(ahora);
  inicio.setDate(inicio.getDate() - 1);
  inicio.setHours(8, 0, 0, 0);
  const fin = new Date(ahora);
  fin.setDate(fin.getDate() + 4);
  fin.setHours(23, 59, 0, 0);
  return {
    inicio: inicio.toISOString(),
    fin: fin.toISOString(),
    maxLaboratoriosPorEstudiante: 4,
    permitirAnulacion: true,
    notificarAforoCompleto: true,
  };
}

const NOMBRES = [
  'Apaza Mamani, Lucía Fernanda', 'Benavente Ccama, Renzo Alonso', 'Cáceres Huamán, Valeria Isabel', 'Chávez
    Flores, Diego Sebastián',
  'Condori Quispe, María José', 'Delgado Paz, Andrés Felipe', 'Espinoza Rivera, Camila Andrea', 'Flores Ticona,
    Bruno Martín',
  'Gutiérrez Salas, Ana Sofía', 'Huanca Choque, Kevin Joel', 'Linares Ortiz, Daniela Paola', 'Mendoza Torres,
    Joaquín Eduardo',
  'Núñez Vargas, Fiorella Antonia', 'Pacheco Rojas, Rodrigo Iván', 'Quispe Mamani, Brenda Lizeth', 'Ramos Cutipa,
    Sergio Alejandro',
  'Salinas Pérez, Nicole Abigail', 'Tapia Zúñiga, Gonzalo André', 'Urquizo Medina, Ariana Belén', 'Vilca Arce,
    Marco Antonio',
  'Yucra Lazo, Ximena Rocío', 'Zapata Coaguila, Luis Fernando', 'Alvarado Neyra, Paula Cristina', 'Bustamante Ríos,
     Hugo Daniel',
  'Coaquira Pari, Karla Noemí', 'Díaz Llerena, Mauricio José', 'Escobar Tejada, Silvia Mercedes', 'Gallegos Huerta,
     Adrián Elías',
  'Herrera Mendoza, Gabriela Inés', 'Ibáñez Calla, Franco Rafael',
];

/** Nómina sintética por turno, coherente con los cupos ocupados de cada grupo. */
export function nominaDemo(turno: Turno): { id: string; cui: string; nombre: string; correo: string }[] {
  const desplazamiento = TURNOS.findIndex((x) => x.id === turno.id) * 3;
  return Array.from({ length: turno.cuposOcupados }, (_, i) => {
    const nombre = NOMBRES[(i + desplazamiento) % NOMBRES.length];
    const usuario = nombre.split(',')[0].split(' ')[0].toLowerCase().normalize('NFD').replace(/[-]/g, '');
    return {
      id: `${turno.id}-${i + 1}`,
      cui: String(20210000 + ((i * 37 + desplazamiento * 11) % 9000)).padStart(8, '0'),
      nombre,
      correo: `${nombre.split(', ')[1][0].toLowerCase()}${usuario}@unsa.edu.pe`,
    };
  });
}
```

A.7 Estado global

src/state/AppStore.tsx

```tsx
/**
 * Estado global con Context + useReducer. El reductor es la única puerta para modificar cupos,
 * de modo que la reserva se aplica de forma atómica (RF-05) aunque dos acciones lleguen seguidas.
 */
import { createContext, useContext, useMemo, useReducer, type Dispatch, type ReactNode } from 'react';
import { CURSOS, LABORATORIOS, MATRICULAS, TEORIA, TURNOS, USUARIOS, configInicial, nominaDemo } from '../domain/datos';
import { huella, liberarCupo, payloadQR, reservarCupo } from '../domain/reglas';
import type { ConfigMatricula, Laboratorio, Matricula, Rol, Turno, Usuario } from '../domain/types';

export interface Estado {
  usuario: Usuario | null;
  turnos: Turno[];
  matriculas: Matricula[];
  laboratorios: Laboratorio[];
  config: ConfigMatricula;
  /** Asistencias marcadas en la sesión actual: id de fila de nómina -> hora "HH:MM". */
  asistencias: Record<string, string>;
}

type Accion =
  | { tipo: 'INICIAR_SESION'; usuario: Usuario }
  | { tipo: 'CERRAR_SESION' }
  | { tipo: 'MATRICULAR'; turnoId: string; ahora: string }
  | { tipo: 'ANULAR'; matriculaId: string }
  | { tipo: 'MARCAR_ASISTENCIA'; filaId: string; hora: string }
  | { tipo: 'ACTUALIZAR_LABORATORIO'; laboratorio: Laboratorio }
  | { tipo: 'ACTUALIZAR_CONFIG'; config: ConfigMatricula };

export const estadoInicial: Estado = {
  usuario: null,
  turnos: TURNOS,
  matriculas: MATRICULAS,
  laboratorios: LABORATORIOS,
  config: configInicial(),
  asistencias: {},
};

export function reductor(e: Estado, a: Accion): Estado {
  switch (a.tipo) {
    case 'INICIAR_SESION':
      return { ...e, usuario: a.usuario };
    case 'CERRAR_SESION':
      return { ...e, usuario: null };
    case 'MATRICULAR': {
      if (!e.usuario) return e;
      const { turnos, reservado } = reservarCupo(e.turnos, a.turnoId);
      if (!reservado) return e;
      const id = `M-${String(e.matriculas.length + 1).padStart(4, '0')}`;
      return {
        ...e,
        turnos,
        matriculas: [...e.matriculas, { id, estudianteId: e.usuario.id, turnoId: a.turnoId, fecha: a.ahora, codigo: id, estado: 'confirmada' }],
      };
    }
    case 'ANULAR': {
      const m = e.matriculas.find((x) => x.id === a.matriculaId);
      if (!m || m.estado === 'anulada') return e;
      return {
        ...e,
        turnos: liberarCupo(e.turnos, m.turnoId),
```

```
        matriculas: e.matriculas.map((x) => (x.id === m.id ? { ...x, estado: 'anulada' } : x)),
      };
    }
    case 'MARCAR_ASISTENCIA':
      return e.asistencias[a.filaId] ? e : { ...e, asistencias: { ...e.asistencias, [a.filaId]: a.hora } };
    case 'ACTUALIZAR_LABORATORIO':
      return { ...e, laboratorios: e.laboratorios.map((l) => (l.id === a.laboratorio.id ? a.laboratorio : l)) };
    case 'ACTUALIZAR_CONFIG':
      return { ...e, config: a.config };
  }
}

const Contexto = createContext<{ estado: Estado; dispatch: Dispatch<Accion> } | null>(null);

export function ProveedorApp({ children }: { children: ReactNode }) {
  const [estado, dispatch] = useReducer(reductor, estadoInicial);
  const valor = useMemo(() => ({ estado, dispatch }), [estado]);
  return <Contexto.Provider value={valor}>{children}</Contexto.Provider>;
}

export function useApp() {
  const v = useContext(Contexto);
  if (!v) throw new Error('useApp debe usarse dentro de ProveedorApp');
  return v;
}

// ---------- Selectores ----------

export const curso = (id: string) => CURSOS.find((c) => c.id === id)!;
export const usuario = (id: string) => USUARIOS.find((u) => u.id === id);
export const teoriaDelEstudiante = () => TEORIA;
export const usuarioDemo = (rol: Rol) => USUARIOS.find((u) => u.rol === rol)!;

export function matriculasActivas(e: Estado): Matricula[] {
  return e.matriculas.filter((m) => m.estado === 'confirmada' && m.estudianteId === e.usuario?.id);
}

export function laboratorio(e: Estado, id: string): Laboratorio {
  return e.laboratorios.find((l) => l.id === id)!;
}

/** Huella corta que el docente puede digitar si la cámara no está disponible. */
export function codigoVerificacion(m: Matricula): string {
  return huella(payloadQR(m).split('|').slice(0, 4).join('|'));
}

export interface FilaNomina {
  id: string;
  cui: string;
  nombre: string;
  correo: string;
  matriculaId?: string;
}

/** Nómina del turno: estudiantes de demostración más las matrículas reales hechas en la app. */
export function nomina(e: Estado, turnoId: string): FilaNomina[] {
  const turno = e.turnos.find((t) => t.id === turnoId)!;
  const reales = e.matriculas.filter((m) => m.turnoId === turnoId && m.estado === 'confirmada');
  const sinteticas = nominaDemo({ ...turno, cuposOcupados: Math.max(0, turno.cuposOcupados - reales.length) });
  const propias: FilaNomina[] = reales.map((m) => {
    const u = usuario(m.estudianteId)!;
    return { id: m.id, cui: u.cui ?? '', nombre: u.nombre, correo: u.correo, matriculaId: m.id };
  });
  return [...propias, ...sinteticas].sort((a, b) => a.nombre.localeCompare(b.nombre, 'es'));
}
```

A.8 Tokens de diseño

src/theme/tokens.ts

```ts
/**
 * Tokens de diseño. Los colores institucionales (#C3141E, #19375A) se mantienen desde el LAB 01;
 * cada token tiene su variante clara y oscura para respetar el tema del sistema operativo.
 */
export interface Paleta {
  fondo: string;
  superficie: string;
  superficieAlta: string;
  borde: string;
  texto: string;
  textoSecundario: string;
  textoTenue: string;
  primario: string;
  primarioSuave: string;
  marca: string;
  marcaSuave: string;
  exito: string;
  exitoSuave: string;
  alerta: string;
  alertaSuave: string;
  peligro: string;
  peligroSuave: string;
  sobrePrimario: string;
}

export const CLARO: Paleta = {
  fondo: '#F4F6F9',
  superficie: '#FFFFFF',
  superficieAlta: '#EEF2F7',
  borde: '#DCE2EA',
  texto: '#14202E',
  textoSecundario: '#4A5868',
  textoTenue: '#7A8796',
  primario: '#19375A',
  primarioSuave: '#E3EAF3',
  marca: '#C3141E',
  marcaSuave: '#FBE5E6',
  exito: '#1E7B45',
  exitoSuave: '#DFF2E7',
  alerta: '#A35F00',
  alertaSuave: '#FBEFD9',
  peligro: '#B3261E',
  peligroSuave: '#F9E0DE',
  sobrePrimario: '#FFFFFF',
};

export const OSCURO: Paleta = {
  fondo: '#0E151D',
  superficie: '#16202B',
  superficieAlta: '#1E2A37',
  borde: '#2B3948',
  texto: '#E7EDF4',
  textoSecundario: '#B3C0CD',
  textoTenue: '#8796A6',
  primario: '#7FA8D8',
  primarioSuave: '#1D3049',
  marca: '#F0626A',
  marcaSuave: '#3A1C20',
  exito: '#5CC98A',
  exitoSuave: '#173323',
  alerta: '#F0B65A',
  alertaSuave: '#35290F',
```

```
  peligro: '#FF8A80',
  peligroSuave: '#3A1D1B',
  sobrePrimario: '#0E151D',
};

/** Escala de espaciado de 4 pt: todas las separaciones del layout salen de aquí. */
export const espacio = { xs: 4, s: 8, m: 12, l: 16, xl: 24, xxl: 32 } as const;
export const radio = { s: 8, m: 12, l: 16, pill: 999 } as const;
export const tipo = {
  titulo: { fontSize: 24, fontWeight: '700' as const, letterSpacing: 0.2 },
  subtitulo: { fontSize: 18, fontWeight: '600' as const },
  cuerpo: { fontSize: 15, fontWeight: '400' as const },
  etiqueta: { fontSize: 13, fontWeight: '600' as const },
  pie: { fontSize: 12, fontWeight: '400' as const },
  cifra: { fontSize: 28, fontWeight: '700' as const },
};
```

A.9 Contexto de tema

src/theme/ThemeContext.tsx

```
import { createContext, useContext, useMemo, useState, type ReactNode } from 'react';
import { useColorScheme } from 'react-native';
import { CLARO, OSCURO, type Paleta } from './tokens';

type Preferencia = 'sistema' | 'claro' | 'oscuro';

interface ValorTema {
  c: Paleta;
  oscuro: boolean;
  preferencia: Preferencia;
  setPreferencia: (p: Preferencia) => void;
}

const Contexto = createContext<ValorTema | null>(null);

/** El tema sigue al sistema (useColorScheme) salvo que el usuario fije uno desde Perfil. */
export function ProveedorTema({ children }: { children: ReactNode }) {
  const sistema = useColorScheme();
  const [preferencia, setPreferencia] = useState<Preferencia>('sistema');
  const oscuro = preferencia === 'oscuro' || (preferencia === 'sistema' && sistema === 'dark');
  const valor = useMemo(() => ({ c: oscuro ? OSCURO : CLARO, oscuro, preferencia, setPreferencia }), [oscuro,
     preferencia]);
  return <Contexto.Provider value={valor}>{children}</Contexto.Provider>;
}

export function useTema(): ValorTema {
  const v = useContext(Contexto);
  if (!v) throw new Error('useTema debe usarse dentro de ProveedorTema');
  return v;
}
```

A.10 Componentes reutilizables

src/components/ui.tsx

```tsx
/**
 * Componentes de interfaz reutilizables construidos solo con primitivas de React Native
 * (View, Text, Pressable, Image). Ninguno usa librerías de UI externas.
 */
import { Ionicons } from '@expo/vector-icons';
import type { ComponentProps, ReactNode } from 'react';
import { Pressable, StyleSheet, Text, View, type StyleProp, type ViewStyle } from 'react-native';
import type { Semaforo } from '../domain/types';
import { useTema } from '../theme/ThemeContext';
import { espacio, radio, tipo } from '../theme/tokens';

type Icono = ComponentProps<typeof Ionicons>['name'];

export function Tarjeta({ children, style, onPress, accessibilityLabel }: { children: ReactNode; style?: StyleProp<ViewStyle>; onPress?: () => void; accessibilityLabel?: string }) {
  const { c } = useTema();
  const base = [estilos.tarjeta, { backgroundColor: c.superficie, borderColor: c.borde }, style];
  if (!onPress) return <View style={base}>{children}</View>;
  return (
    <Pressable
      onPress={onPress}
      accessibilityRole="button"
      accessibilityLabel={accessibilityLabel}
      android_ripple={{ color: c.primarioSuave }}
      style={({ pressed }) => [base, pressed && { opacity: 0.92, transform: [{ scale: 0.99 }] }]}
    >
      {children}
    </Pressable>
  );
}

type VarianteBoton = 'primario' | 'marca' | 'secundario' | 'fantasma';

export function Boton({ titulo, onPress, variante = 'primario', icono, deshabilitado, cargando, accessibilityLabel, style }: {
  titulo: string; onPress: () => void; variante?: VarianteBoton; icono?: Icono; deshabilitado?: boolean; cargando?: boolean; accessibilityLabel?: string; style?: StyleProp<ViewStyle>;
}) {
  const { c } = useTema();
  const fondo = { primario: c.primario, marca: c.marca, secundario: c.primarioSuave, fantasma: 'transparent' }[variante];
  const color = variante === 'primario' || variante === 'marca' ? c.sobrePrimario : c.primario;
  return (
    <Pressable
      onPress={onPress}
      disabled={deshabilitado || cargando}
      accessibilityRole="button"
      accessibilityLabel={accessibilityLabel ?? titulo}
      accessibilityState={{ disabled: Boolean(deshabilitado), busy: Boolean(cargando) }}
      android_ripple={{ color: 'rgba(255,255,255,0.25)' }}
      style={({ pressed }) => [
        estilos.boton,
        { backgroundColor: fondo, borderColor: variante === 'fantasma' ? c.borde : fondo, opacity: deshabilitado ? 0.45 : pressed ? 0.85 : 1 },
        style,
      ]}
    >
      {icono && <Ionicons name={icono} size={18} color={color} />}
      <Text style={[tipo.etiqueta, { color, fontSize: 15 }]}>{cargando ? '…Procesando' : titulo}</Text>
    </Pressable>
  );
}
```

```
58
59 export function Chip({ texto, activo, onPress, icono }: { texto: string; activo?: boolean; onPress?: () => void;
       icono?: Icono }) {
60   const { c } = useTema();
61   return (
62     <Pressable
63       onPress={onPress}
64       accessibilityRole="button"
65       accessibilityState={{ selected: Boolean(activo) }}
66       accessibilityLabel={texto}
67       style={[estilos.chip, { backgroundColor: activo ? c.primario : c.superficie, borderColor: activo ? c.
       primario : c.borde }]}
68     >
69       {icono && <Ionicons name={icono} size={14} color={activo ? c.sobrePrimario : c.textoSecundario} />}
70       <Text style={[tipo.etiqueta, { color: activo ? c.sobrePrimario : c.textoSecundario }]}>{texto}</Text>
71     </Pressable>
72   );
73 }
74
75 const ETIQUETA_SEMAFORO: Record<Semaforo, string> = { verde: 'Disponible', ambar: 'Últimos cupos', rojo: 'Agotado'
       };
76
77 export function Insignia({ estado, texto }: { estado: Semaforo | 'info'; texto?: string }) {
78   const { c } = useTema();
79   const [fondo, color] = {
80     verde: [c.exitoSuave, c.exito], ambar: [c.alertaSuave, c.alerta], rojo: [c.peligroSuave, c.peligro], info: [c.
       primarioSuave, c.primario],
81   }[estado];
82   return (
83     <View style={[estilos.insignia, { backgroundColor: fondo }]} accessibilityLabel={texto ?? ETIQUETA_SEMAFORO[
       estado as Semaforo]}>
84       {estado !== 'info' && <View style={[estilos.punto, { backgroundColor: color }]} />}
85       <Text style={[tipo.pie, { color, fontWeight: '700' }]}>{texto ?? ETIQUETA_SEMAFORO[estado as Semaforo]}</
       Text>
86     </View>
87   );
88 }
89
90 /** Barra de ocupación: el ancho del relleno es un porcentaje del contenedor (layout relativo). */
91 export function BarraProgreso({ valor, total, color }: { valor: number; total: number; color?: string }) {
92   const { c } = useTema();
93   const pct = total === 0 ? 0 : Math.min(100, Math.round((valor / total) * 100));
94   return (
95     <View style={[estilos.barra, { backgroundColor: c.superficieAlta }]} accessibilityRole="progressbar"
       accessibilityValue={{ min: 0, max: total, now: valor }}>
96       <View style={{ width: `${pct}%`, height: '100%', borderRadius: radio.pill, backgroundColor: color ?? c.
       primario }} />
97     </View>
98   );
99 }
100
101 export function Kpi({ valor, etiqueta, icono, color }: { valor: string; etiqueta: string; icono: Icono; color?:
       string }) {
102   const { c } = useTema();
103   const tono = color ?? c.primario;
104   return (
105     <Tarjeta style={estilos.kpi}>
106       <View style={[estilos.kpiIcono, { backgroundColor: c.superficieAlta }]}>
107         <Ionicons name={icono} size={18} color={tono} />
108       </View>
109       <Text style={[tipo.cifra, { color: c.texto }]}>{valor}</Text>
110       <Text style={[tipo.pie, { color: c.textoSecundario }]} numberOfLines={2}>{etiqueta}</Text>
111     </Tarjeta>
112   );
113 }
114
```

```
115 export function TituloSeccion({ titulo, accion, onAccion }: { titulo: string; accion?: string; onAccion?: () =>
        void }) {
116   const { c } = useTema();
117   return (
118     <View style={estilos.tituloSeccion}>
119       <Text style={[tipo.subtitulo, { color: c.texto }]} accessibilityRole="header">{titulo}</Text>
120       {accion && (
121         <Pressable onPress={onAccion} hitSlop={8} accessibilityRole="button">
122           <Text style={[tipo.etiqueta, { color: c.primario }]}>{accion}</Text>
123         </Pressable>
124       )}
125     </View>
126   );
127 }
128
129 export function FilaAjuste({ icono, titulo, detalle, derecha, onPress }: { icono: Icono; titulo: string; detalle?:
        string; derecha?: ReactNode; onPress?: () => void }) {
130   const { c } = useTema();
131   return (
132     <Pressable onPress={onPress} disabled={!onPress} accessibilityRole={onPress ? 'button' : undefined} style={[
       estilos.fila, { borderBottomColor: c.borde }]}>
133       <View style={[estilos.filaIcono, { backgroundColor: c.primarioSuave }]}>
134         <Ionicons name={icono} size={18} color={c.primario} />
135       </View>
136       <View style={{ flex: 1 }}>
137         <Text style={[tipo.cuerpo, { color: c.texto, fontWeight: '600' }]}>{titulo}</Text>
138         {detalle && <Text style={[tipo.pie, { color: c.textoSecundario }]}>{detalle}</Text>}
139       </View>
140       {derecha ?? (onPress && <Ionicons name="chevron-forward" size={18} color={c.textoTenue} />)}
141     </Pressable>
142   );
143 }
144
145 export function Avatar({ nombre, tamano = 44 }: { nombre: string; tamano?: number }) {
146   const { c } = useTema();
147   const iniciales = nombre.replace(/^(Mg\.|Ing\.|Dr\.|Dra\.)\s*/, '').split(/[ ,]+/).filter(Boolean).slice(0, 2).
        map((p) => p[0]).join('').toUpperCase();
148   return (
149     <View style={{ width: tamano, height: tamano, borderRadius: tamano / 2, backgroundColor: c.primario,
       alignItems: 'center', justifyContent: 'center' }}>
150       <Text style={{ color: c.sobrePrimario, fontWeight: '700', fontSize: tamano * 0.36 }}>{iniciales}</Text>
151     </View>
152   );
153 }
154
155 export function Vacio({ icono, titulo, detalle }: { icono: Icono; titulo: string; detalle: string }) {
156   const { c } = useTema();
157   return (
158     <View style={estilos.vacio}>
159       <Ionicons name={icono} size={40} color={c.textoTenue} />
160       <Text style={[tipo.subtitulo, { color: c.texto, textAlign: 'center' }]}>{titulo}</Text>
161       <Text style={[tipo.cuerpo, { color: c.textoSecundario, textAlign: 'center' }]}>{detalle}</Text>
162     </View>
163   );
164 }
165
166 const estilos = StyleSheet.create({
167   tarjeta: { borderRadius: radio.l, borderWidth: StyleSheet.hairlineWidth, padding: espacio.l, boxShadow: '0 1px 3
        px rgba(16, 32, 48, 0.08)' },
168   boton: { minHeight: 48, flexDirection: 'row', alignItems: 'center', justifyContent: 'center', gap: espacio.s,
        paddingHorizontal: espacio.l, borderRadius: radio.m, borderWidth: 1, overflow: 'hidden' },
169   chip: { flexDirection: 'row', alignItems: 'center', gap: espacio.xs, paddingHorizontal: espacio.m,
        paddingVertical: 7, borderRadius: radio.pill, borderWidth: 1 },
170   insignia: { flexDirection: 'row', alignItems: 'center', gap: 6, alignSelf: 'flex-start', paddingHorizontal: 10,
        paddingVertical: 4, borderRadius: radio.pill },
171   punto: { width: 8, height: 8, borderRadius: 4 },
```

```
  barra: { height: 8, borderRadius: radio.pill, overflow: 'hidden' },
  kpi: { flexBasis: '47%', flexGrow: 1, gap: espacio.xs },
  kpiIcono: { width: 32, height: 32, borderRadius: radio.s, alignItems: 'center', justifyContent: 'center',
     marginBottom: espacio.xs },
  tituloSeccion: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'baseline', marginTop:
     espacio.xl, marginBottom: espacio.m },
  fila: { flexDirection: 'row', alignItems: 'center', gap: espacio.m, paddingVertical: espacio.m,
     borderBottomWidth: StyleSheet.hairlineWidth },
  filaIcono: { width: 36, height: 36, borderRadius: radio.s, alignItems: 'center', justifyContent: 'center' },
  vacio: { alignItems: 'center', gap: espacio.s, paddingVertical: espacio.xxl, paddingHorizontal: espacio.xl },
});
```

A.11 Tipos de navegación

src/navigation/tipos.ts

```
import type { BottomTabScreenProps } from '@react-navigation/bottom-tabs';
import type { CompositeScreenProps, NavigatorScreenParams } from '@react-navigation/native';
import type { NativeStackScreenProps } from '@react-navigation/native-stack';

export type TabsEstudiante = { Inicio: undefined; Catalogo: undefined; Horario: undefined; PerfilEstudiante:
     undefined };
export type TabsDocente = { Panel: undefined; Escaner: undefined; PerfilDocente: undefined };
export type TabsAdmin = { Laboratorios: undefined; Matricula: undefined; PerfilAdmin: undefined };

export type PilaRaiz = {
  Login: undefined;
  Estudiante: NavigatorScreenParams<TabsEstudiante>;
  Docente: NavigatorScreenParams<TabsDocente>;
  Admin: NavigatorScreenParams<TabsAdmin>;
  DetalleTurno: { turnoId: string };
  FichaQR: { matriculaId: string };
  Nomina: { turnoId: string };
};

export type PantallaRaiz<K extends keyof PilaRaiz> = NativeStackScreenProps<PilaRaiz, K>;
export type PantallaEstudiante<K extends keyof TabsEstudiante> = CompositeScreenProps<BottomTabScreenProps<
     TabsEstudiante, K>, NativeStackScreenProps<PilaRaiz>>;
export type PantallaDocente<K extends keyof TabsDocente> = CompositeScreenProps<BottomTabScreenProps<TabsDocente,
     K>, NativeStackScreenProps<PilaRaiz>>;
export type PantallaAdmin<K extends keyof TabsAdmin> = CompositeScreenProps<BottomTabScreenProps<TabsAdmin, K>,
     NativeStackScreenProps<PilaRaiz>>;
```

A.12 Navegación

src/navigation/RootNavigator.tsx

```
/**
 * Navegación: una pila nativa (native-stack usa UINavigationController en iOS y Fragment en Android)
 * con tres juegos de pestañas inferiores, uno por rol. El detalle del grupo se presenta como modal nativo.
 */
import { Ionicons } from '@expo/vector-icons';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
import { DarkTheme, DefaultTheme, NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import type { ComponentProps } from 'react';
import ConfiguracionScreen from '../screens/admin/ConfiguracionScreen';
import LaboratoriosScreen from '../screens/admin/LaboratoriosScreen';
import EscanerScreen from '../screens/docente/EscanerScreen';
import NominaScreen from '../screens/docente/NominaScreen';
import PanelDocenteScreen from '../screens/docente/PanelDocenteScreen';
import CatalogoScreen from '../screens/estudiante/CatalogoScreen';
import DashboardScreen from '../screens/estudiante/DashboardScreen';
```

```
import DetalleTurnoScreen from '../screens/estudiante/DetalleTurnoScreen';
import FichaQRScreen from '../screens/estudiante/FichaQRScreen';
import HorarioScreen from '../screens/estudiante/HorarioScreen';
import LoginScreen from '../screens/LoginScreen';
import PerfilScreen from '../screens/PerfilScreen';
import { useTema } from '../theme/ThemeContext';
import type { PilaRaiz, TabsAdmin, TabsDocente, TabsEstudiante } from './tipos';

type Icono = ComponentProps<typeof Ionicons>['name'];

const Pila = createNativeStackNavigator<PilaRaiz>();
const TabE = createBottomTabNavigator<TabsEstudiante>();
const TabD = createBottomTabNavigator<TabsDocente>();
const TabA = createBottomTabNavigator<TabsAdmin>();

function useOpcionesTabs(iconos: Record<string, [Icono, Icono]>) {
  const { c } = useTema();
  return ({ route }: { route: { name: string } }) => ({
    tabBarActiveTintColor: c.marca,
    tabBarInactiveTintColor: c.textoTenue,
    tabBarStyle: { backgroundColor: c.superficie, borderTopColor: c.borde },
    tabBarLabelStyle: { fontSize: 11, fontWeight: '600' as const },
    headerStyle: { backgroundColor: c.superficie },
    headerTitleStyle: { color: c.texto, fontWeight: '700' as const },
    headerShadowVisible: false,
    tabBarIcon: ({ focused, color, size }: { focused: boolean; color: string; size: number }) => (
      <Ionicons name={iconos[route.name][focused ? 0 : 1]} size={size} color={color} />
    ),
  });
}

function PestanasEstudiante() {
  const opciones = useOpcionesTabs({
    Inicio: ['home', 'home-outline'], Catalogo: ['albums', 'albums-outline'], Horario: ['calendar', 'calendar-
     outline'], PerfilEstudiante: ['person', 'person-outline'],
  });
  return (
    <TabE.Navigator screenOptions={opciones}>
      <TabE.Screen name="Inicio" component={DashboardScreen} options={{ title: 'MatriLab', tabBarLabel: 'Inicio' }}
      />
      <TabE.Screen name="Catalogo" component={CatalogoScreen} options={{ title: 'Catálogo de laboratorios',
     tabBarLabel: 'Catálogo' }} />
      <TabE.Screen name="Horario" component={HorarioScreen} options={{ title: 'Mi horario', tabBarLabel: 'Horario'
      }} />
      <TabE.Screen name="PerfilEstudiante" component={PerfilScreen} options={{ title: 'Perfil', tabBarLabel: '
     Perfil' }} />
    </TabE.Navigator>
  );
}

function PestanasDocente() {
  const opciones = useOpcionesTabs({ Panel: ['grid', 'grid-outline'], Escaner: ['scan', 'scan-outline'],
     PerfilDocente: ['person', 'person-outline'] });
  return (
    <TabD.Navigator screenOptions={opciones}>
      <TabD.Screen name="Panel" component={PanelDocenteScreen} options={{ title: 'Panel docente', tabBarLabel: '
     Panel' }} />
      <TabD.Screen name="Escaner" component={EscanerScreen} options={{ title: 'Registrar asistencia', tabBarLabel:
      'Escáner' }} />
      <TabD.Screen name="PerfilDocente" component={PerfilScreen} options={{ title: 'Perfil', tabBarLabel: 'Perfil'
      }} />
    </TabD.Navigator>
  );
}

function PestanasAdmin() {
```

```
  const opciones = useOpcionesTabs({ Laboratorios: ['desktop', 'desktop-outline'], Matricula: ['options', 'options-outline'], PerfilAdmin: ['person', 'person-outline'] });
  return (
    <TabA.Navigator screenOptions={opciones}>
      <TabA.Screen name="Laboratorios" component={LaboratoriosScreen} options={{ title: 'Laboratorios', tabBarLabel: 'Laboratorios' }} />
      <TabA.Screen name="Matricula" component={ConfiguracionScreen} options={{ title: 'Configurar matrícula', tabBarLabel: 'Matrícula' }} />
      <TabA.Screen name="PerfilAdmin" component={PerfilScreen} options={{ title: 'Perfil', tabBarLabel: 'Perfil' }} />
    </TabA.Navigator>
  );
}

export default function RootNavigator() {
  const { c, oscuro } = useTema();
  const base = oscuro ? DarkTheme : DefaultTheme;
  const tema = { ...base, colors: { ...base.colors, primary: c.marca, background: c.fondo, card: c.superficie, text: c.texto, border: c.borde } };
  return (
    <NavigationContainer theme={tema}>
      <Pila.Navigator
        initialRouteName="Login"
        screenOptions={{ headerStyle: { backgroundColor: c.superficie }, headerTintColor: c.primario, headerTitleStyle: { color: c.texto }, headerBackButtonDisplayMode: 'minimal' }}
      >
        <Pila.Screen name="Login" component={LoginScreen} options={{ headerShown: false }} />
        <Pila.Screen name="Estudiante" component={PestanasEstudiante} options={{ headerShown: false }} />
        <Pila.Screen name="Docente" component={PestanasDocente} options={{ headerShown: false }} />
        <Pila.Screen name="Admin" component={PestanasAdmin} options={{ headerShown: false }} />
        <Pila.Screen name="DetalleTurno" component={DetalleTurnoScreen} options={{ title: 'Detalle del grupo', presentation: 'modal' }} />
        <Pila.Screen name="FichaQR" component={FichaQRScreen} options={{ title: 'Mi pase de laboratorio' }} />
        <Pila.Screen name="Nomina" component={NominaScreen} options={{ title: 'Nómina' }} />
      </Pila.Navigator>
    </NavigationContainer>
  );
}
```

A.13 I1 · Inicio de sesión

src/screens/LoginScreen.tsx

```
/**
 * Interfaz 1 · Inicio de sesión institucional (RF-01).
 * Layout: KeyboardAvoidingView + ScrollView para que el formulario no quede bajo el teclado,
 * y una columna flex centrada con ancho máximo para tabletas y web.
 */
import { Ionicons } from '@expo/vector-icons';
import { StatusBar } from 'expo-status-bar';
import { useState } from 'react';
import { Image, KeyboardAvoidingView, Platform, Pressable, ScrollView, StyleSheet, Switch, Text, TextInput, View } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { Boton } from '../components/ui';
import { USUARIOS } from '../domain/datos';
import type { Rol, Usuario } from '../domain/types';
import type { PantallaRaiz } from '../navigation/tipos';
import { useApp, usuarioDemo } from '../state/AppStore';
import { useTema } from '../theme/ThemeContext';
import { espacio, radio, tipo } from '../theme/tokens';

const CLAVE_DEMO = 'unsa2026';
const DESTINO: Record<Rol, 'Estudiante' | 'Docente' | 'Admin'> = { estudiante: 'Estudiante', docente: 'Docente', admin: 'Admin' };
```

```
21
22  export default function LoginScreen({ navigation }: PantallaRaiz<'Login'>) {
23    const { c } = useTema();
24    const { dispatch } = useApp();
25    const [correo, setCorreo] = useState('');
26    const [clave, setClave] = useState('');
27    const [verClave, setVerClave] = useState(false);
28    const [recordar, setRecordar] = useState(true);
29    const [error, setError] = useState<string | null>(null);
30
31    const entrar = (u: Usuario) => {
32      dispatch({ tipo: 'INICIAR_SESION', usuario: u });
33      navigation.reset({ index: 0, routes: [{ name: DESTINO[u.rol] }] });
34    };
35
36    const validar = () => {
37      const limpio = correo.trim().toLowerCase();
38      if (!/^[a-z0-9._-]+@unsa\.edu\.pe$/.test(limpio)) return setError('Usa tu correo institucional @unsa.edu.pe.')
        ;
39      if (clave.length < 6) return setError('La contraseña debe tener al menos 6 caracteres.');
40      const u = USUARIOS.find((x) => x.correo === limpio);
41      if (!u || clave !== CLAVE_DEMO) return setError('Correo o contraseña incorrectos.');
42      setError(null);
43      entrar(u);
44    };
45
46    return (
47      <SafeAreaView style={{ flex: 1, backgroundColor: c.primario }} edges={['top', 'bottom']}>
48        <StatusBar style="light" />
49        <KeyboardAvoidingView style={{ flex: 1 }} behavior={Platform.OS === 'ios' ? 'padding' : 'height'}>
50          <ScrollView contentContainerStyle={estilos.scroll} keyboardShouldPersistTaps="handled">
51            <View style={estilos.cabecera}>
52              <Text style={[estilos.marca, { color: c.sobrePrimario }]}>MatriLab</Text>
53              <Text style={{ color: c.sobrePrimario, opacity: 0.85, fontSize: 15 }}>Matrícula de laboratorios · EPIS-
      UNSA</Text>
54            </View>
55
56            <View style={[estilos.tarjeta, { backgroundColor: c.superficie }]}>
57              <View style={estilos.logoCaja}>
58                <Image source={require('../../assets/img/logo_episunsa.png')} style={estilos.logo} resizeMode="
      contain" accessibilityLabel="Logo EPIS-UNSA" />
59              </View>
60              <Text style={[tipo.titulo, { color: c.texto }]}>Iniciar sesión</Text>
61              <Text style={[tipo.cuerpo, { color: c.textoSecundario, marginBottom: espacio.l }]}>Ingresa con tu
      cuenta institucional.</Text>
62
63              <Text style={[tipo.etiqueta, { color: c.textoSecundario }]}>Correo institucional</Text>
64              <View style={[estilos.campo, { borderColor: c.borde, backgroundColor: c.fondo }]}>
65                <Ionicons name="mail-outline" size={18} color={c.textoTenue} />
66                <TextInput
67                  value={correo}
68                  onChangeText={setCorreo}
69                  placeholder="usuario@unsa.edu.pe"
70                  placeholderTextColor={c.textoTenue}
71                  keyboardType="email-address"
72                  autoCapitalize="none"
73                  autoComplete="email"
74                  textContentType="emailAddress"
75                  returnKeyType="next"
76                  accessibilityLabel="Correo institucional"
77                  style={[estilos.input, { color: c.texto }]}
78                />
79              </View>
80
81              <Text style={[tipo.etiqueta, { color: c.textoSecundario, marginTop: espacio.m }]}>Contraseña</Text>
82              <View style={[estilos.campo, { borderColor: c.borde, backgroundColor: c.fondo }]}>
```

```
              <Ionicons name="lock-closed-outline" size={18} color={c.textoTenue} />
              <TextInput
                value={clave}
                onChangeText={setClave}
                placeholder="••••••••"
                placeholderTextColor={c.textoTenue}
                secureTextEntry={!verClave}
                textContentType="password"
                onSubmitEditing={validar}
                accessibilityLabel="Contraseña"
                style={[estilos.input, { color: c.texto }]}
              />
              <Pressable onPress={() => setVerClave((v) => !v)} hitSlop={10} accessibilityLabel={verClave ? '
    Ocultar contraseña' : 'Mostrar contraseña'}>
                <Ionicons name={verClave ? 'eye-off-outline' : 'eye-outline'} size={20} color={c.textoTenue} />
              </Pressable>
            </View>

            <View style={estilos.recordar}>
              <Text style={[tipo.cuerpo, { color: c.textoSecundario }]}>Mantener la sesión iniciada</Text>
              <Switch value={recordar} onValueChange={setRecordar} trackColor={{ true: c.primario, false: c.borde
    }} accessibilityLabel="Mantener la sesión iniciada" />
            </View>

            {error && (
              <View style={[estilos.error, { backgroundColor: c.peligroSuave }]} accessibilityLiveRegion="polite">
                <Ionicons name="alert-circle" size={18} color={c.peligro} />
                <Text style={[tipo.etiqueta, { color: c.peligro, flex: 1 }]}>{error}</Text>
              </View>
            )}

            <Boton titulo="Ingresar" icono="log-in-outline" onPress={validar} variante="marca" accessibilityLabel=
    "Ingresar" />

            <View style={estilos.separador}>
              <View style={[estilos.linea, { backgroundColor: c.borde }]} />
              <Text style={[tipo.pie, { color: c.textoTenue }]}>Acceso de demostración</Text>
              <View style={[estilos.linea, { backgroundColor: c.borde }]} />
            </View>
            <View style={estilos.demos}>
              {(['estudiante', 'docente', 'admin'] as Rol[]).map((rol) => (
                <Boton
                  key={rol}
                  variante="secundario"
                  titulo={{ estudiante: 'Estudiante', docente: 'Docente', admin: 'Admin' }[rol]}
                  icono={{ estudiante: 'school-outline', docente: 'easel-outline', admin: 'settings-outline' }[rol
    ] as never}
                  onPress={() => entrar(usuarioDemo(rol))}
                  accessibilityLabel={`Demo ${rol}`}
                  style={{ flex: 1, paddingHorizontal: espacio.s }}
                />
              ))}
            </View>
          </View>

          <Text style={[tipo.pie, estilos.legal, { color: c.sobrePrimario }]}>
            Tus datos se tratan conforme a la Ley N.° 29733 de Protección de Datos Personales.
          </Text>
        </ScrollView>
      </KeyboardAvoidingView>
    </SafeAreaView>
  );
}

const estilos = StyleSheet.create({
  scroll: { flexGrow: 1, justifyContent: 'center', padding: espacio.xl, gap: espacio.xl },
```

```
  cabecera: { alignItems: 'center', gap: espacio.xs },
  marca: { fontSize: 36, fontWeight: '800', letterSpacing: 0.5 },
  tarjeta: { borderRadius: radio.l + 4, padding: espacio.xl, width: '100%', maxWidth: 440, alignSelf: 'center',
     boxShadow: '0 8px 24px rgba(0, 0, 0, 0.25)' },
  logoCaja: { alignItems: 'center', marginBottom: espacio.l },
  logo: { width: 180, height: 54 },
  campo: { flexDirection: 'row', alignItems: 'center', gap: espacio.s, borderWidth: 1, borderRadius: radio.m,
     paddingHorizontal: espacio.m, minHeight: 48, marginTop: espacio.xs },
  input: { flex: 1, fontSize: 16, paddingVertical: espacio.s },
  recordar: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center', marginVertical: espacio.
     l },
  error: { flexDirection: 'row', gap: espacio.s, alignItems: 'center', padding: espacio.m, borderRadius: radio.m,
     marginBottom: espacio.m },
  separador: { flexDirection: 'row', alignItems: 'center', gap: espacio.s, marginVertical: espacio.l },
  linea: { flex: 1, height: StyleSheet.hairlineWidth },
  demos: { flexDirection: 'row', gap: espacio.s },
  legal: { textAlign: 'center', opacity: 0.8, paddingHorizontal: espacio.l },
});
```

A.14 I2 · Panel del estudiante

src/screens/estudiante/DashboardScreen.tsx

```
/**
 * Interfaz 2 · Panel del estudiante.
 * Layout: ScrollView con RefreshControl; fila de KPI con flexWrap (dos columnas que se reacomodan
 * según el ancho) y tarjetas con una franja de color posicionada con flexDirection: 'row'.
 */
import { Ionicons } from '@expo/vector-icons';
import { useCallback, useState } from 'react';
import { Alert, RefreshControl, ScrollView, StyleSheet, Text, View } from 'react-native';
import { Avatar, Boton, Insignia, Kpi, Tarjeta, TituloSeccion, Vacio } from '../../components/ui';
import { NOMBRE_DIA, cuposLibres, ordenarBloques, rango, ventanaAbierta } from '../../domain/reglas';
import type { PantallaEstudiante } from '../../navigation/tipos';
import { curso, laboratorio, matriculasActivas, useApp } from '../../state/AppStore';
import { useTema } from '../../theme/ThemeContext';
import { espacio, radio, tipo } from '../../theme/tokens';

export default function DashboardScreen({ navigation }: PantallaEstudiante<'Inicio'>) {
  const { c } = useTema();
  const { estado, dispatch } = useApp();
  const [refrescando, setRefrescando] = useState(false);

  const activas = matriculasActivas(estado);
  const turnosPropios = ordenarBloques(activas.map((m) => ({ ...estado.turnos.find((t) => t.id === m.turnoId)!,
     matriculaId: m.id })));
  const abierta = ventanaAbierta(estado.config, new Date());
  const creditos = turnosPropios.reduce((s, t) => s + curso(t.cursoId).creditos, 0);
  const libres = estado.turnos.reduce((s, t) => s + cuposLibres(t), 0);
  const cierre = new Date(estado.config.fin);

  const refrescar = useCallback(() => {
    setRefrescando(true);
    setTimeout(() => setRefrescando(false), 700);
  }, []);

  const anular = (matriculaId: string, nombre: string) =>
    Alert.alert('Anular matrícula', `¿Liberar tu cupo en ${nombre}? El cupo quedará disponible para otro
     estudiante.`, [
      { text: 'Cancelar', style: 'cancel' },
      { text: 'Anular', style: 'destructive', onPress: () => dispatch({ tipo: 'ANULAR', matriculaId }) },
    ]);

  return (
    <ScrollView
```

```
      style={{ backgroundColor: c.fondo }}
      contentContainerStyle={estilos.contenido}
      refreshControl={<RefreshControl refreshing={refrescando} onRefresh={refrescar} colors={[c.marca]} tintColor={c.marca} />}
    >
      <View style={estilos.saludo}>
        <Avatar nombre={estado.usuario?.nombre ?? 'Estudiante'} tamano={52} />
        <View style={{ flex: 1 }}>
          <Text style={[tipo.pie, { color: c.textoSecundario }]}>Semestre 2026-B · CUI {estado.usuario?.cui}</Text>
          <Text style={[tipo.subtitulo, { color: c.texto }]} numberOfLines={1}>Hola, {estado.usuario?.nombre.split(', ')[1]?.split(' ')[0]}</Text>
        </View>
        <Ionicons name="notifications-outline" size={24} color={c.textoSecundario} accessibilityLabel="Notificaciones" />
      </View>

      <Tarjeta style={[estilos.ventana, { backgroundColor: abierta ? c.primario : c.peligroSuave, borderColor: 'transparent' }]}>
        <View style={{ flex: 1, gap: espacio.xs }}>
          <Text style={[tipo.etiqueta, { color: abierta ? c.sobrePrimario : c.peligro, opacity: 0.9 }]}>{abierta ? 'MATRÍCULA ABIERTA' : 'MATRÍCULA CERRADA'}</Text>
          <Text style={[tipo.subtitulo, { color: abierta ? c.sobrePrimario : c.peligro }]}>
            {abierta ? `Cierra el ${cierre.toLocaleDateString('es-PE', { weekday: 'long', day: 'numeric', month: 'long' })}` : 'Consulta la nueva fecha con coordinación'}
          </Text>
        </View>
        <Ionicons name={abierta ? 'time-outline' : 'lock-closed'} size={32} color={abierta ? c.sobrePrimario : c.peligro} />
      </Tarjeta>

      <View style={estilos.kpis}>
        <Kpi valor={String(activas.length)} etiqueta="Laboratorios matriculados" icono="flask-outline" />
        <Kpi valor={String(creditos)} etiqueta="Créditos con laboratorio" icono="ribbon-outline" color={c.marca} />
        <Kpi valor={String(libres)} etiqueta="Vacantes libres en la oferta" icono="desktop-outline" color={c.exito} />
        <Kpi valor={String(estado.config.maxLaboratoriosPorEstudiante - activas.length)} etiqueta="Laboratorios que aún puedes tomar" icono="add-circle-outline" color={c.alerta} />
      </View>

      <TituloSeccion titulo="Mis laboratorios" accion="Ver catálogo" onAccion={() => navigation.navigate('Catalogo')} />
      {turnosPropios.length === 0 ? (
        <Vacio icono="flask-outline" titulo="Aún no tienes laboratorios" detalle="Elige un grupo en el catálogo; el sistema valida cruces de horario por ti." />
      ) : (
        <View style={{ gap: espacio.m }}>
          {turnosPropios.map((t) => {
            const cur = curso(t.cursoId);
            const lab = laboratorio(estado, t.laboratorioId);
            return (
              <Tarjeta key={t.id} style={estilos.matricula}>
                <View style={[estilos.franja, { backgroundColor: cur.color }]} />
                <View style={{ flex: 1, gap: espacio.s }}>
                  <View style={estilos.filaEntre}>
                    <Text style={[tipo.cuerpo, { color: c.texto, fontWeight: '700', flex: 1 }]} numberOfLines={2}>{cur.nombre}</Text>
                    <Insignia estado="info" texto={`Grupo ${t.grupo}`} />
                  </View>
                  <View style={estilos.meta}>
                    <Ionicons name="calendar-outline" size={14} color={c.textoSecundario} />
                    <Text style={[tipo.pie, { color: c.textoSecundario }]}>{NOMBRE_DIA[t.dia]} · {rango(t)}</Text>
                    <Ionicons name="location-outline" size={14} color={c.textoSecundario} />
                    <Text style={[tipo.pie, { color: c.textoSecundario }]}>{lab.nombre}</Text>
                  </View>
```

```
                  <View style={estilos.acciones}>
                    <Boton titulo="Ver pase QR" icono="qr-code-outline" onPress={() => navigation.navigate('
    FichaQR', { matriculaId: t.matriculaId })} style={{ flex: 1 }} accessibilityLabel={`Pase ${cur.nombre}`} />
                    {estado.config.permitirAnulacion && (
                      <Boton titulo="Anular" variante="fantasma" icono="close-circle-outline" onPress={() =>
    anular(t.matriculaId, cur.nombre)} accessibilityLabel={`Anular ${cur.nombre}`} />
                    )}
                  </View>
                </View>
              </Tarjeta>
            );
          })}
        </View>
      )}
    </ScrollView>
  );
}

const estilos = StyleSheet.create({
  contenido: { padding: espacio.l, paddingBottom: espacio.xxl },
  saludo: { flexDirection: 'row', alignItems: 'center', gap: espacio.m, marginBottom: espacio.l },
  ventana: { flexDirection: 'row', alignItems: 'center', gap: espacio.m },
  kpis: { flexDirection: 'row', flexWrap: 'wrap', gap: espacio.m, marginTop: espacio.l },
  matricula: { flexDirection: 'row', gap: espacio.m, paddingLeft: espacio.m },
  franja: { width: 5, borderRadius: radio.pill },
  filaEntre: { flexDirection: 'row', alignItems: 'flex-start', justifyContent: 'space-between', gap: espacio.s },
  meta: { flexDirection: 'row', alignItems: 'center', flexWrap: 'wrap', gap: 6 },
  acciones: { flexDirection: 'row', gap: espacio.s, marginTop: espacio.xs },
});
```

A.15 I3 · Catálogo

src/screens/estudiante/CatalogoScreen.tsx

```
/**
 * Interfaz 3 · Catálogo de grupos con semáforo de vacantes (RF-02, RF-03).
 * Layout: SectionList agrupada por curso con encabezados fijos (stickySectionHeadersEnabled),
 * buscador y una fila de filtros en un ScrollView horizontal.
 */
import { Ionicons } from '@expo/vector-icons';
import { useMemo, useState } from 'react';
import { ScrollView, SectionList, StyleSheet, Text, TextInput, View } from 'react-native';
import { BarraProgreso, Chip, Insignia, Tarjeta, Vacio } from '../../components/ui';
import { CURSOS } from '../../domain/datos';
import { NOMBRE_DIA, cuposLibres, haySolape, rango, semaforo } from '../../domain/reglas';
import type { Turno } from '../../domain/types';
import type { PantallaEstudiante } from '../../navigation/tipos';
import { laboratorio, matriculasActivas, teoriaDelEstudiante, useApp, usuario } from '../../state/AppStore';
import { useTema } from '../../theme/ThemeContext';
import { espacio, radio, tipo } from '../../theme/tokens';

type Filtro = 'todos' | 'cupo' | 'sinCruce';

export default function CatalogoScreen({ navigation }: PantallaEstudiante<'Catalogo'>) {
  const { c } = useTema();
  const { estado } = useApp();
  const [busqueda, setBusqueda] = useState('');
  const [filtro, setFiltro] = useState<Filtro>('todos');

  const ocupados = useMemo(() => {
    const propios = matriculasActivas(estado).map((m) => estado.turnos.find((t) => t.id === m.turnoId)!);
    return [...propios, ...teoriaDelEstudiante()];
  }, [estado]);

  const secciones = useMemo(() => {
```

```
    const q = busqueda.trim().toLowerCase();
    return CURSOS.map((cur) => ({
      curso: cur,
      data: estado.turnos.filter((t) => {
        if (t.cursoId !== cur.id) return false;
        if (q && !`${cur.nombre} ${t.grupo} ${NOMBRE_DIA[t.dia]}`.toLowerCase().includes(q)) return false;
        if (filtro === 'cupo' && cuposLibres(t) === 0) return false;
        if (filtro === 'sinCruce' && ocupados.some((o) => o !== undefined && 'id' in o && o.id !== t.id &&
     haySolape(o, t))) return false;
        return true;
      }),
    })).filter((s) => s.data.length > 0);
  }, [estado.turnos, busqueda, filtro, ocupados]);

  const renderTurno = ({ item }: { item: Turno }) => {
    const libres = cuposLibres(item);
    const estadoSem = semaforo(item);
    const lab = laboratorio(estado, item.laboratorioId);
    const color = { verde: c.exito, ambar: c.alerta, rojo: c.peligro }[estadoSem];
    return (
      <Tarjeta style={estilos.item} onPress={() => navigation.navigate('DetalleTurno', { turnoId: item.id })}
     accessibilityLabel={`Grupo ${item.id}`}>
        <View style={estilos.cabeceraItem}>
          <View style={[estilos.grupo, { backgroundColor: c.primarioSuave }]}>
            <Text style={[tipo.subtitulo, { color: c.primario }]}>{item.grupo}</Text>
          </View>
          <View style={{ flex: 1, gap: 2 }}>
            <Text style={[tipo.cuerpo, { color: c.texto, fontWeight: '700' }]}>{NOMBRE_DIA[item.dia]} · {rango(
     item)}</Text>
            <Text style={[tipo.pie, { color: c.textoSecundario }]} numberOfLines={1}>{lab.nombre} · {usuario(item.
     docenteId)?.nombre}</Text>
          </View>
          <Insignia estado={estadoSem} />
        </View>
        <BarraProgreso valor={item.cuposOcupados} total={item.cuposTotales} color={color} />
        <Text style={[tipo.pie, { color: c.textoSecundario }]}>
          <Text style={{ color, fontWeight: '700' }}>{libres}</Text> de {item.cuposTotales} PC libres
        </Text>
      </Tarjeta>
    );
  };

  return (
    <View style={{ flex: 1, backgroundColor: c.fondo }}>
      <View style={[estilos.barraBusqueda, { backgroundColor: c.superficie, borderBottomColor: c.borde }]}>
        <View style={[estilos.campo, { backgroundColor: c.fondo, borderColor: c.borde }]}>
          <Ionicons name="search" size={18} color={c.textoTenue} />
          <TextInput
            value={busqueda}
            onChangeText={setBusqueda}
            placeholder="Buscar curso, grupo o día"
            placeholderTextColor={c.textoTenue}
            style={[estilos.input, { color: c.texto }]}
            accessibilityLabel="Buscar en el catálogo"
            clearButtonMode="while-editing"
          />
        </View>
        <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={estilos.chips}>
          <Chip texto="Todos" activo={filtro === 'todos'} onPress={() => setFiltro('todos')} icono="apps-outline"
     />
          <Chip texto="Con vacantes" activo={filtro === 'cupo'} onPress={() => setFiltro('cupo')} icono="checkmark-
     circle-outline" />
          <Chip texto="Sin cruce de horario" activo={filtro === 'sinCruce'} onPress={() => setFiltro('sinCruce')}
     icono="git-compare-outline" />
        </ScrollView>
      </View>
```

```
      <SectionList
        sections={secciones}
        keyExtractor={(t) => t.id}
        renderItem={renderTurno}
        stickySectionHeadersEnabled
        contentContainerStyle={estilos.lista}
        ItemSeparatorComponent={() => <View style={{ height: espacio.m }} />}
        SectionSeparatorComponent={() => <View style={{ height: espacio.s }} />}
        renderSectionHeader={({ section }) => (
          <View style={[estilos.seccion, { backgroundColor: c.fondo }]}>
            <View style={[estilos.puntoCurso, { backgroundColor: section.curso.color }]} />
            <Text style={[tipo.etiqueta, { color: c.texto, flex: 1 }]}>{section.curso.nombre}</Text>
            <Text style={[tipo.pie, { color: c.textoTenue }]}>{section.data.length} grupo(s)</Text>
          </View>
        )}
        ListEmptyComponent={<Vacio icono="search-outline" titulo="Sin resultados" detalle="Prueba con otro término o quita el filtro." />}
      />
    </View>
  );
}

const estilos = StyleSheet.create({
  barraBusqueda: { padding: espacio.l, paddingBottom: espacio.m, gap: espacio.m, borderBottomWidth: StyleSheet.hairlineWidth },
  campo: { flexDirection: 'row', alignItems: 'center', gap: espacio.s, borderWidth: 1, borderRadius: radio.m, paddingHorizontal: espacio.m, minHeight: 44 },
  input: { flex: 1, fontSize: 15, paddingVertical: espacio.s },
  chips: { gap: espacio.s, paddingRight: espacio.l },
  lista: { padding: espacio.l, paddingBottom: espacio.xxl },
  seccion: { flexDirection: 'row', alignItems: 'center', gap: espacio.s, paddingVertical: espacio.s },
  puntoCurso: { width: 10, height: 10, borderRadius: 5 },
  item: { gap: espacio.m },
  cabeceraItem: { flexDirection: 'row', alignItems: 'center', gap: espacio.m },
  grupo: { width: 44, height: 44, borderRadius: radio.m, alignItems: 'center', justifyContent: 'center' },
});
```

A.16 I4 · Detalle del grupo

src/screens/estudiante/DetalleTurnoScreen.tsx

```
/**
 * Interfaz 4 · Detalle del grupo y confirmación de matrícula (RF-04, RF-05).
 * Se presenta como hoja modal nativa (presentation: 'modal' en native-stack).
 * Layout destacado: el mapa de puestos usa flexWrap con celdas de ancho fijo, de modo que
 * la cuadrícula de 20, 25 o 30 PC se reacomoda sola en cualquier ancho de pantalla.
 */
import { Ionicons } from '@expo/vector-icons';
import { useMemo, useState } from 'react';
import { ActivityIndicator, ScrollView, StyleSheet, Text, View } from 'react-native';
import { Boton, Insignia, Tarjeta } from '../../components/ui';
import { NOMBRE_DIA, cuposLibres, rango, semaforo, validarMatricula } from '../../domain/reglas';
import type { PantallaRaiz } from '../../navigation/tipos';
import { curso, laboratorio, matriculasActivas, teoriaDelEstudiante, useApp, usuario } from '../../state/AppStore';
import { useTema } from '../../theme/ThemeContext';
import { espacio, radio, tipo } from '../../theme/tokens';

export default function DetalleTurnoScreen({ route, navigation }: PantallaRaiz<'DetalleTurno'>) {
  const { c } = useTema();
  const { estado, dispatch } = useApp();
  const [procesando, setProcesando] = useState(false);
  const turno = estado.turnos.find((t) => t.id === route.params.turnoId)!;
  const cur = curso(turno.cursoId);
```

```
  const lab = laboratorio(estado, turno.laboratorioId);
  const libres = cuposLibres(turno);

  const validacion = useMemo(
    () => validarMatricula({
      turno,
      turnos: estado.turnos,
      teoria: teoriaDelEstudiante(),
      matriculasActivas: matriculasActivas(estado),
      config: estado.config,
      ahora: new Date(),
      nombreCurso: (id) => curso(id).nombre,
    }),
    [turno, estado],
  );

  const confirmar = () => {
    setProcesando(true);
    // Simula la latencia de la transacción en el servidor antes de asentar la reserva.
    setTimeout(() => {
      dispatch({ tipo: 'MATRICULAR', turnoId: turno.id, ahora: new Date().toISOString() });
      setProcesando(false);
      navigation.replace('FichaQR', { matriculaId: `M-${String(estado.matriculas.length + 1).padStart(4, '0')}` })
     ;
    }, 900);
  };

  const filas: [string, string, string][] = [
    ['calendar-outline', 'Horario', `${NOMBRE_DIA[turno.dia]} · ${rango(turno)}`],
    ['location-outline', 'Ambiente', `${lab.nombre} · ${lab.pabellon}`],
    ['person-outline', 'Docente', usuario(turno.docenteId)?.nombre ?? '—'],
    ['hardware-chip-outline', 'Equipamiento', lab.equipamiento],
  ];

  return (
    <View style={{ flex: 1, backgroundColor: c.fondo }}>
      <ScrollView contentContainerStyle={estilos.contenido}>
        <View style={{ gap: espacio.xs }}>
          <Text style={[tipo.pie, { color: c.textoSecundario }]}>{cur.codigo} · {cur.creditos} créditos</Text>
          <Text style={[tipo.titulo, { color: c.texto }]}>{cur.nombre}</Text>
          <View style={{ flexDirection: 'row', gap: espacio.s, marginTop: espacio.xs }}>
            <Insignia estado="info" texto={`Grupo ${turno.grupo}`} />
            <Insignia estado={semaforo(turno)} />
          </View>
        </View>

        <Tarjeta style={{ gap: espacio.m }}>
          {filas.map(([icono, etiqueta, valor]) => (
            <View key={etiqueta} style={estilos.fila}>
              <Ionicons name={icono as never} size={18} color={c.primario} />
              <Text style={[tipo.etiqueta, { color: c.textoSecundario, width: 96 }]}>{etiqueta}</Text>
              <Text style={[tipo.cuerpo, { color: c.texto, flex: 1 }]}>{valor}</Text>
            </View>
          ))}
        </Tarjeta>

        <Tarjeta style={{ gap: espacio.m }}>
          <View style={estilos.filaEntre}>
            <Text style={[tipo.subtitulo, { color: c.texto }]}>Puestos del laboratorio</Text>
            <Text style={[tipo.etiqueta, { color: libres ? c.exito : c.peligro }]}>{libres} libres</Text>
          </View>
          <View style={estilos.mapa} accessibilityLabel={`${turno.cuposOcupados} puestos ocupados de ${turno.
    cuposTotales}`}>
            {Array.from({ length: turno.cuposTotales }, (_, i) => {
              const ocupado = i < turno.cuposOcupados;
              return (
```

```
87                <View key={i} style={[estilos.pc, { backgroundColor: ocupado ? c.peligroSuave : c.exitoSuave,
    borderColor: ocupado ? c.peligro : c.exito }]}>
88                  <Ionicons name="desktop-outline" size={14} color={ocupado ? c.peligro : c.exito} />
89                  <Text style={[estilos.numeroPc, { color: ocupado ? c.peligro : c.exito }]}>{i + 1}</Text>
90                </View>
91              );
92            })}
93          </View>
94          <View style={estilos.leyenda}>
95            <View style={[estilos.cuadro, { backgroundColor: c.exitoSuave, borderColor: c.exito }]} />
96            <Text style={[tipo.pie, { color: c.textoSecundario }]}>Libre</Text>
97            <View style={[estilos.cuadro, { backgroundColor: c.peligroSuave, borderColor: c.peligro }]} />
98            <Text style={[tipo.pie, { color: c.textoSecundario }]}>Ocupado</Text>
99          </View>
100       </Tarjeta>
101
102       {!validacion.ok && (
103         <View style={[estilos.aviso, { backgroundColor: c.peligroSuave }]} accessibilityLiveRegion="polite">
104           <Ionicons name={validacion.motivo === 'CRUCE' ? 'git-compare-outline' : 'alert-circle-outline'} size
    ={22} color={c.peligro} />
105           <View style={{ flex: 1, gap: 2 }}>
106             <Text style={[tipo.etiqueta, { color: c.peligro }]}>{validacion.motivo === 'CRUCE' ? 'Cruce de
    horario (RN-02)' : 'No se puede matricular'}</Text>
107             <Text style={[tipo.cuerpo, { color: c.peligro }]}>{validacion.detalle}</Text>
108           </View>
109         </View>
110       )}
111       {validacion.ok && (
112         <View style={[estilos.aviso, { backgroundColor: c.exitoSuave }]}>
113           <Ionicons name="shield-checkmark-outline" size={22} color={c.exito} />
114           <Text style={[tipo.cuerpo, { color: c.exito, flex: 1 }]}>Sin cruces con tu horario. Al confirmar se
    reserva tu puesto de inmediato.</Text>
115         </View>
116       )}
117     </ScrollView>
118
119     <View style={[estilos.pie, { backgroundColor: c.superficie, borderTopColor: c.borde }]}>
120       {procesando ? (
121         <View style={estilos.procesando}>
122           <ActivityIndicator color={c.marca} />
123           <Text style={[tipo.cuerpo, { color: c.textoSecundario }]}>Reservando tu …puesto</Text>
124         </View>
125       ) : (
126         <Boton titulo="Confirmar matrícula" icono="checkmark-circle-outline" variante="marca" deshabilitado={!
    validacion.ok} onPress={confirmar} accessibilityLabel="Confirmar matrícula" />
127       )}
128     </View>
129   </View>
130 );
131 }
132
133 const estilos = StyleSheet.create({
134   contenido: { padding: espacio.l, gap: espacio.l, paddingBottom: espacio.xxl },
135   fila: { flexDirection: 'row', alignItems: 'flex-start', gap: espacio.m },
136   filaEntre: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center' },
137   mapa: { flexDirection: 'row', flexWrap: 'wrap', gap: 6 },
138   pc: { width: 46, height: 40, borderRadius: radio.s, borderWidth: 1, alignItems: 'center', justifyContent: '
      center' },
139   numeroPc: { fontSize: 10, fontWeight: '700' },
140   leyenda: { flexDirection: 'row', alignItems: 'center', gap: espacio.s },
141   cuadro: { width: 14, height: 14, borderRadius: 3, borderWidth: 1 },
142   aviso: { flexDirection: 'row', gap: espacio.m, padding: espacio.l, borderRadius: radio.m, alignItems: 'flex-
      start' },
143   pie: { padding: espacio.l, borderTopWidth: StyleSheet.hairlineWidth },
144   procesando: { flexDirection: 'row', alignItems: 'center', justifyContent: 'center', gap: espacio.m, minHeight:
      48 },
```

```
145 });
```

A.17 I5 · Pase QR

src/screens/estudiante/FichaQRScreen.tsx

```tsx
/**
 * Interfaz 5 · Ficha de matrícula con pase QR (RF-06).
 * Layout: la tarjeta del pase tiene dos zonas separadas por perforaciones dibujadas con
 * position: 'absolute', y el sello "Confirmada" se superpone en la esquina con absolute + top/right.
 */
import { Ionicons } from '@expo/vector-icons';
import { ScrollView, Share, StyleSheet, Text, View, useWindowDimensions } from 'react-native';
import QRCode from 'react-native-qrcode-svg';
import { Boton } from '../../components/ui';
import { NOMBRE_DIA, payloadQR, rango } from '../../domain/reglas';
import type { PantallaRaiz } from '../../navigation/tipos';
import { codigoVerificacion, curso, laboratorio, useApp, usuario } from '../../state/AppStore';
import { useTema } from '../../theme/ThemeContext';
import { espacio, radio, tipo } from '../../theme/tokens';

export default function FichaQRScreen({ route }: PantallaRaiz<'FichaQR'>) {
  const { c } = useTema();
  const { estado } = useApp();
  const { width } = useWindowDimensions();
  const m = estado.matriculas.find((x) => x.id === route.params.matriculaId);
  if (!m) return null;
  const turno = estado.turnos.find((t) => t.id === m.turnoId)!;
  const cur = curso(turno.cursoId);
  const lab = laboratorio(estado, turno.laboratorioId);
  const est = usuario(m.estudianteId)!;
  const ladoQR = Math.min(220, width - 140);
  const codigo = codigoVerificacion(m);

  const compartir = () =>
    Share.share({ message: `Pase MatriLab ${m.id} · ${cur.nombre} grupo ${turno.grupo} · ${NOMBRE_DIA[turno.dia]}
     ${rango(turno)} · ${lab.nombre} · Código ${codigo}` });

  return (
    <ScrollView style={{ backgroundColor: c.fondo }} contentContainerStyle={estilos.contenido}>
      <View style={[estilos.pase, { backgroundColor: c.superficie, borderColor: c.borde }]}>
        <View style={[estilos.superior, { backgroundColor: c.primario }]}>
          <Text style={[tipo.pie, { color: c.sobrePrimario, opacity: 0.85 }]}>PASE DE LABORATORIO · 2026-B</Text>
          <Text style={[tipo.subtitulo, { color: c.sobrePrimario }]}>{cur.nombre}</Text>
          <Text style={[tipo.cuerpo, { color: c.sobrePrimario, opacity: 0.9 }]}>Grupo {turno.grupo} · {NOMBRE_DIA[
    turno.dia]} {rango(turno)}</Text>
          <View style={[estilos.sello, { backgroundColor: c.exito }]}>
            <Ionicons name="checkmark-circle" size={14} color="#FFFFFF" />
            <Text style={estilos.textoSello}>{m.estado === 'confirmada' ? 'Confirmada' : 'Anulada'}</Text>
          </View>
        </View>

        <View style={estilos.perforacion}>
          <View style={[estilos.muesca, { left: -12, backgroundColor: c.fondo }]} />
          <View style={[estilos.guiones, { borderColor: c.borde }]} />
          <View style={[estilos.muesca, { right: -12, backgroundColor: c.fondo }]} />
        </View>

        <View style={estilos.inferior}>
          <View style={[estilos.marcoQR, { borderColor: c.borde }]} accessibilityLabel="Código QR del pase">
            <QRCode value={payloadQR(m)} size={ladoQR} color="#14202E" backgroundColor="#FFFFFF" />
          </View>
          <Text style={[tipo.pie, { color: c.textoSecundario }]}>Código de verificación</Text>
          <Text style={[estilos.codigo, { color: c.texto }]} selectable>{codigo}</Text>
          <View style={estilos.datos}>
```

```
            {[
              ['Estudiante', est.nombre],
              ['CUI', est.cui ?? ''],
              ['Ambiente', `${lab.nombre} · ${lab.pabellon}`],
              ['Matrícula', `${m.id} · ${new Date(m.fecha).toLocaleString('es-PE', { dateStyle: 'medium', timeStyle: 'short' })}`],
            ].map(([k, v]) => (
              <View key={k} style={estilos.dato}>
                <Text style={[tipo.pie, { color: c.textoTenue }]}>{k}</Text>
                <Text style={[tipo.etiqueta, { color: c.texto }]}>{v}</Text>
              </View>
            ))}
          </View>
        </View>
      </View>

      <Text style={[tipo.pie, { color: c.textoSecundario, textAlign: 'center' }]}>Muestra este código al docente en la puerta del laboratorio para registrar tu asistencia.</Text>
      <Boton titulo="Compartir pase" icono="share-social-outline" variante="secundario" onPress={compartir} />
    </ScrollView>
  );
}

const estilos = StyleSheet.create({
  contenido: { padding: espacio.l, gap: espacio.l, paddingBottom: espacio.xxl },
  pase: { borderRadius: radio.l + 4, borderWidth: StyleSheet.hairlineWidth, overflow: 'hidden', boxShadow: '0 6px 18px rgba(16, 32, 48, 0.12)' },
  superior: { padding: espacio.xl, gap: espacio.xs, paddingRight: 120 },
  sello: { position: 'absolute', top: espacio.l, right: espacio.l, flexDirection: 'row', alignItems: 'center', gap: 4, paddingHorizontal: 10, paddingVertical: 5, borderRadius: radio.pill },
  textoSello: { color: '#FFFFFF', fontWeight: '700', fontSize: 12 },
  perforacion: { height: 24, justifyContent: 'center' },
  muesca: { position: 'absolute', width: 24, height: 24, borderRadius: 12 },
  guiones: { marginHorizontal: espacio.xl, borderTopWidth: 2, borderStyle: 'dashed' },
  inferior: { alignItems: 'center', padding: espacio.xl, paddingTop: espacio.s, gap: espacio.s },
  marcoQR: { padding: espacio.m, backgroundColor: '#FFFFFF', borderRadius: radio.m, borderWidth: 1 },
  codigo: { fontSize: 22, fontWeight: '800', letterSpacing: 4, fontVariant: ['tabular-nums'] },
  datos: { alignSelf: 'stretch', flexDirection: 'row', flexWrap: 'wrap', rowGap: espacio.m, marginTop: espacio.m },

  dato: { width: '50%', paddingRight: espacio.s, gap: 2 },
});
```

A.18 I6 · Horario

src/screens/estudiante/HorarioScreen.tsx

```
/**
 * Interfaz 6 · Horario semanal consolidado (RF-07).
 * Layout: cuadrícula calculada con useWindowDimensions. Cada columna es un día y cada bloque se
 * coloca con position: 'absolute' (top = hora de inicio, height = duración). Si la pantalla es
 * angosta, la cuadrícula pasa a un ScrollView horizontal con columnas de ancho mínimo.
 */
import { useState } from 'react';
import { ScrollView, StyleSheet, Text, View, useWindowDimensions } from 'react-native';
import { Chip, Insignia, Tarjeta, TituloSeccion } from '../../components/ui';
import { DIAS, NOMBRE_DIA, aHora, ordenarBloques, rango } from '../../domain/reglas';
import type { Bloque } from '../../domain/types';
import { curso, laboratorio, matriculasActivas, teoriaDelEstudiante, useApp } from '../../state/AppStore';
import { useTema } from '../../theme/ThemeContext';
import { espacio, radio, tipo } from '../../theme/tokens';

const INICIO = 7 * 60;
const FIN = 17 * 60;
const ALTO_HORA = 58;
const CANAL_HORAS = 44;
```

```
20 const DIAS_VISIBLES = DIAS.slice(0, 5);
21
22 interface Evento extends Bloque { id: string; cursoId: string; tipo: 'lab' | 'teoria'; lugar: string; grupo?:
       string }
23
24 export default function HorarioScreen() {
25   const { c } = useTema();
26   const { estado } = useApp();
27   const { width } = useWindowDimensions();
28   const [vista, setVista] = useState<'semana' | 'lista'>('semana');
29
30   const labs: Evento[] = matriculasActivas(estado).map((m) => {
31     const t = estado.turnos.find((x) => x.id === m.turnoId)!;
32     return { ...t, tipo: 'lab', lugar: laboratorio(estado, t.laboratorioId).nombre };
33   });
34   const teoria: Evento[] = teoriaDelEstudiante().map((t) => ({ ...t, tipo: 'teoria', lugar: t.aula }));
35   const eventos = ordenarBloques([...labs, ...teoria]);
36
37   const disponible = width - espacio.l * 2 - CANAL_HORAS;
38   const anchoDia = Math.max(64, disponible / DIAS_VISIBLES.length);
39   const desborda = anchoDia * DIAS_VISIBLES.length > disponible + 1;
40   const horas = Array.from({ length: (FIN - INICIO) / 60 + 1 }, (_, i) => INICIO + i * 60);
41
42   const cuadricula = (
43     <View style={{ flexDirection: 'row' }}>
44       <View style={{ width: CANAL_HORAS, height: (FIN - INICIO) / 60 * ALTO_HORA + 28 }}>
45         {horas.map((h) => (
46           <Text key={h} style={[estilos.hora, { top: (h - INICIO) / 60 * ALTO_HORA + 28 - 7, color: c.textoTenue
       }]}>{aHora(h)}</Text>
47         ))}
48       </View>
49       {DIAS_VISIBLES.map((dia) => (
50         <View key={dia} style={{ width: anchoDia }}>
51           <View style={estilos.cabeceraDia}>
52             <Text style={[tipo.etiqueta, { color: c.textoSecundario }]}>{NOMBRE_DIA[dia].slice(0, 3)}</Text>
53           </View>
54           <View style={{ height: (FIN - INICIO) / 60 * ALTO_HORA, borderLeftWidth: StyleSheet.hairlineWidth,
       borderColor: c.borde }}>
55             {horas.map((h) => (
56               <View key={h} style={[estilos.lineaHora, { top: (h - INICIO) / 60 * ALTO_HORA, borderColor: c.borde
       }]} />
57             ))}
58             {eventos.filter((e) => e.dia === dia).map((e) => {
59               const cur = curso(e.cursoId);
60               const lab = e.tipo === 'lab';
61               return (
62                 <View
63                   key={e.id}
64                   accessibilityLabel={`${cur.nombre}, ${lab ? 'laboratorio' : 'teoría'}, ${NOMBRE_DIA[e.dia]} ${
       rango(e)}`}
65                   style={[
66                     estilos.bloque,
67                     {
68                       top: (e.inicio - INICIO) / 60 * ALTO_HORA + 1,
69                       height: (e.fin - e.inicio) / 60 * ALTO_HORA - 2,
70                       backgroundColor: lab ? cur.color : c.superficie,
71                       borderColor: cur.color,
72                       borderStyle: lab ? 'solid' : 'dashed',
73                     },
74                   ]}
75                 >
76                   <Text style={[estilos.textoBloque, { color: lab ? '#FFFFFF' : cur.color }]} numberOfLines={3}>{
       cur.codigo.split('-')[0]}{lab ? ` · ${e.grupo}` : ''}</Text>
77                   <Text style={[estilos.lugar, { color: lab ? '#FFFFFF' : c.textoSecundario }]} numberOfLines
       ={1}>{e.lugar}</Text>
78                 </View>
```

```
              );
            })}
          </View>
        </View>
      ))}
    </View>
  );

  return (
    <ScrollView style={{ backgroundColor: c.fondo }} contentContainerStyle={estilos.contenido}>
      <View style={estilos.selector}>
        <Chip texto="Semana" activo={vista === 'semana'} onPress={() => setVista('semana')} icono="grid-outline" />
        <Chip texto="Lista" activo={vista === 'lista'} onPress={() => setVista('lista')} icono="list-outline" />
      </View>

      {vista === 'semana' ? (
        <Tarjeta style={{ paddingHorizontal: espacio.s }}>
          {desborda ? <ScrollView horizontal showsHorizontalScrollIndicator>{cuadricula}</ScrollView> : cuadricula}

          <View style={estilos.leyenda}>
            <View style={[estilos.muestra, { backgroundColor: c.primario }]} />
            <Text style={[tipo.pie, { color: c.textoSecundario }]}>Laboratorio matriculado</Text>
            <View style={[estilos.muestra, { borderWidth: 1, borderStyle: 'dashed', borderColor: c.primario }]} />
            <Text style={[tipo.pie, { color: c.textoSecundario }]}>Teoría</Text>
          </View>
        </Tarjeta>
      ) : (
        DIAS_VISIBLES.map((dia) => {
          const del = eventos.filter((e) => e.dia === dia);
          if (!del.length) return null;
          return (
            <View key={dia}>
              <TituloSeccion titulo={NOMBRE_DIA[dia]} />
              <View style={{ gap: espacio.s }}>
                {del.map((e) => (
                  <Tarjeta key={e.id} style={estilos.itemLista}>
                    <Text style={[tipo.etiqueta, { color: c.texto, width: 92 }]}>{rango(e)}</Text>
                    <View style={{ flex: 1, gap: 2 }}>
                      <Text style={[tipo.cuerpo, { color: c.texto, fontWeight: '600' }]}>{curso(e.cursoId).nombre}</Text>
                      <Text style={[tipo.pie, { color: c.textoSecundario }]}>{e.lugar}</Text>
                    </View>
                    <Insignia estado="info" texto={e.tipo === 'lab' ? `Lab ${e.grupo}` : 'Teoría'} />
                  </Tarjeta>
                ))}
              </View>
            </View>
          );
        })
      )}
    </ScrollView>
  );
}

const estilos = StyleSheet.create({
  contenido: { padding: espacio.l, paddingBottom: espacio.xxl, gap: espacio.m },
  selector: { flexDirection: 'row', gap: espacio.s },
  hora: { position: 'absolute', fontSize: 10, fontVariant: ['tabular-nums'] },
  cabeceraDia: { height: 28, alignItems: 'center', justifyContent: 'center' },
  lineaHora: { position: 'absolute', left: 0, right: 0, borderTopWidth: StyleSheet.hairlineWidth },
  bloque: { position: 'absolute', left: 3, right: 3, borderRadius: radio.s, borderWidth: 1.5, padding: 4, overflow: 'hidden' },
  textoBloque: { fontSize: 11, fontWeight: '800' },
  lugar: { fontSize: 9, opacity: 0.9 },
  leyenda: { flexDirection: 'row', alignItems: 'center', flexWrap: 'wrap', gap: espacio.s, marginTop: espacio.m,
```

```
    paddingHorizontal: espacio.s },
  muestra: { width: 16, height: 12, borderRadius: 3 },
  itemLista: { flexDirection: 'row', alignItems: 'center', gap: espacio.m },
});
```

A.19 I7 · Panel docente

src/screens/docente/PanelDocenteScreen.tsx

```
/**
 * Interfaz 7 · Panel del docente / jefe de prácticas (RF-07 monitoreo de aforo).
 * Layout: FlatList con ListHeaderComponent (saludo + KPI) para que todo el contenido haga
 * scroll en una sola lista virtualizada, sin anidar ScrollView dentro de ScrollView.
 */
import { Ionicons } from '@expo/vector-icons';
import { FlatList, StyleSheet, Text, View } from 'react-native';
import { Avatar, BarraProgreso, Boton, Insignia, Kpi, Tarjeta, TituloSeccion } from '../../components/ui';
import { NOMBRE_DIA, cuposLibres, ordenarBloques, rango, semaforo } from '../../domain/reglas';
import type { PantallaDocente } from '../../navigation/tipos';
import { curso, laboratorio, useApp } from '../../state/AppStore';
import { useTema } from '../../theme/ThemeContext';
import { espacio, tipo } from '../../theme/tokens';

export default function PanelDocenteScreen({ navigation }: PantallaDocente<'Panel'>) {
  const { c } = useTema();
  const { estado } = useApp();
  const mios = ordenarBloques(estado.turnos.filter((t) => t.docenteId === estado.usuario?.id));
  const inscritos = mios.reduce((s, t) => s + t.cuposOcupados, 0);
  const capacidad = mios.reduce((s, t) => s + t.cuposTotales, 0);
  const presentes = Object.keys(estado.asistencias).filter((k) => mios.some((t) => k.startsWith(t.id) || estado.
     matriculas.find((m) => m.id === k)?.turnoId === t.id)).length;

  return (
    <FlatList
      style={{ backgroundColor: c.fondo }}
      contentContainerStyle={estilos.contenido}
      data={mios}
      keyExtractor={(t) => t.id}
      ItemSeparatorComponent={() => <View style={{ height: espacio.m }} />}
      ListHeaderComponent={
        <View>
          <View style={estilos.saludo}>
            <Avatar nombre={estado.usuario?.nombre ?? 'Docente'} tamano={52} />
            <View style={{ flex: 1 }}>
              <Text style={[tipo.pie, { color: c.textoSecundario }]}>Docente · Jefe de prácticas</Text>
              <Text style={[tipo.subtitulo, { color: c.texto }]} numberOfLines={1}>{estado.usuario?.nombre}</Text>
            </View>
          </View>
          <View style={estilos.kpis}>
            <Kpi valor={String(mios.length)} etiqueta="Grupos a cargo" icono="layers-outline" />
            <Kpi valor={`${inscritos}/${capacidad}`} etiqueta="Estudiantes inscritos" icono="people-outline" color
    ={c.marca} />
            <Kpi valor={`${Math.round((inscritos / Math.max(1, capacidad)) * 100)} %`} etiqueta="Ocupación
    promedio" icono="pie-chart-outline" color={c.alerta} />
            <Kpi valor={String(presentes)} etiqueta="Asistencias registradas hoy" icono="checkmark-done-outline"
    color={c.exito} />
          </View>
          <Boton titulo="Escanear pase de un estudiante" icono="scan-outline" variante="marca" onPress={() =>
    navigation.navigate('Escaner')} style={{ marginTop: espacio.l }} />
          <TituloSeccion titulo="Mis grupos de laboratorio" />
        </View>
      }
      renderItem={({ item }) => {
        const cur = curso(item.cursoId);
        const lab = laboratorio(estado, item.laboratorioId);
```

```
        return (
          <Tarjeta style={{ gap: espacio.m }}>
            <View style={estilos.cabecera}>
              <View style={{ flex: 1, gap: 2 }}>
                <Text style={[tipo.cuerpo, { color: c.texto, fontWeight: '700' }]}>{cur.nombre} · Grupo {item.grupo}</Text>
                <View style={estilos.meta}>
                  <Ionicons name="time-outline" size={14} color={c.textoSecundario} />
                  <Text style={[tipo.pie, { color: c.textoSecundario }]}>{NOMBRE_DIA[item.dia]} {rango(item)} · {lab.nombre}</Text>
                </View>
              </View>
              <Insignia estado={semaforo(item)} />
            </View>
            <BarraProgreso valor={item.cuposOcupados} total={item.cuposTotales} />
            <View style={estilos.cabecera}>
              <Text style={[tipo.pie, { color: c.textoSecundario }]}>{item.cuposOcupados} inscritos · {cuposLibres(item)} libres</Text>
              <Boton titulo="Nómina" icono="list-outline" variante="secundario" onPress={() => navigation.navigate('Nomina', { turnoId: item.id })} accessibilityLabel={`Nómina ${item.id}`} />
            </View>
          </Tarjeta>
        );
      }}
    />
  );
}

const estilos = StyleSheet.create({
  contenido: { padding: espacio.l, paddingBottom: espacio.xxl },
  saludo: { flexDirection: 'row', alignItems: 'center', gap: espacio.m, marginBottom: espacio.l },
  kpis: { flexDirection: 'row', flexWrap: 'wrap', gap: espacio.m },
  cabecera: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', gap: espacio.m },
  meta: { flexDirection: 'row', alignItems: 'center', gap: 4 },
});
```

A.20 I8 · Nómina

src/screens/docente/NominaScreen.tsx

```
/**
 * Interfaz 8 · Nómina del grupo con registro manual de asistencia y exportación (RF-07, RF-10).
 * Layout: FlatList virtualizada con un resumen fijo arriba (stickyHeaderIndices sobre el encabezado)
 * y filas de altura uniforme (ALTO_FILA) para que la lista se desplace sin saltos.
 */
import { Ionicons } from '@expo/vector-icons';
import { useLayoutEffect, useMemo, useState } from 'react';
import { FlatList, Pressable, Share, StyleSheet, Text, TextInput, View } from 'react-native';
import { BarraProgreso, Boton } from '../../components/ui';
import { NOMBRE_DIA, nominaCSV, rango } from '../../domain/reglas';
import type { PantallaRaiz } from '../../navigation/tipos';
import { curso, nomina, useApp } from '../../state/AppStore';
import { useTema } from '../../theme/ThemeContext';
import { espacio, radio, tipo } from '../../theme/tokens';

const ALTO_FILA = 68;

export default function NominaScreen({ route, navigation }: PantallaRaiz<'Nomina'>) {
  const { c } = useTema();
  const { estado, dispatch } = useApp();
  const [q, setQ] = useState('');
  const turno = estado.turnos.find((t) => t.id === route.params.turnoId)!;
  const cur = curso(turno.cursoId);
  const filas = useMemo(() => nomina(estado, turno.id), [estado, turno.id]);
  const visibles = filas.filter((f) => `${f.nombre} ${f.cui}`.toLowerCase().includes(q.trim().toLowerCase()));
```

```
26  const presentes = filas.filter((f) => estado.asistencias[f.id]).length;
27
28  useLayoutEffect(() => {
29    navigation.setOptions({ title: `${cur.codigo.split('-')[0]} · Grupo ${turno.grupo}` });
30  }, [navigation, cur.codigo, turno.grupo]);
31
32  const exportar = () =>
33    Share.share({
34      title: `Nómina ${cur.nombre} ${turno.grupo}`,
35      message: nominaCSV(filas.map((f) => ({ cui: f.cui, nombre: f.nombre, correo: f.correo, asistio: Boolean(
     estado.asistencias[f.id]) }))),
36    });
37
38  const marcar = (id: string) =>
39    dispatch({ tipo: 'MARCAR_ASISTENCIA', filaId: id, hora: new Date().toLocaleTimeString('es-PE', { hour: '2-
     digit', minute: '2-digit' }) });
40
41  return (
42    <View style={{ flex: 1, backgroundColor: c.fondo }}>
43      <FlatList
44        data={visibles}
45        keyExtractor={(f) => f.id}
46        stickyHeaderIndices={[0]}
47        ListHeaderComponent={
48          <View style={[estilos.resumen, { backgroundColor: c.superficie, borderBottomColor: c.borde }]}>
49            <Text style={[tipo.etiqueta, { color: c.textoSecundario }]}>{NOMBRE_DIA[turno.dia]} {rango(turno)} · {
     cur.nombre}</Text>
50            <View style={estilos.cifras}>
51              <Text style={[tipo.cifra, { color: c.texto }]}>{presentes}<Text style={[tipo.subtitulo, { color: c.
     textoTenue }]}> / {filas.length}</Text></Text>
52              <Text style={[tipo.pie, { color: c.textoSecundario }]}>presentes en la sesión de hoy</Text>
53            </View>
54            <BarraProgreso valor={presentes} total={filas.length} color={c.exito} />
55            <View style={[estilos.campo, { backgroundColor: c.fondo, borderColor: c.borde }]}>
56              <Ionicons name="search" size={16} color={c.textoTenue} />
57              <TextInput value={q} onChangeText={setQ} placeholder="Buscar por nombre o CUI" placeholderTextColor
     ={c.textoTenue} style={[estilos.input, { color: c.texto }]} accessibilityLabel="Buscar en la nómina" />
58            </View>
59          </View>
60        }
61        renderItem={({ item, index }) => {
62          const hora = estado.asistencias[item.id];
63          return (
64            <Pressable
65              onPress={() => marcar(item.id)}
66              disabled={Boolean(hora)}
67              accessibilityRole="checkbox"
68              accessibilityState={{ checked: Boolean(hora) }}
69              accessibilityLabel={`Asistencia de ${item.nombre}`}
70              android_ripple={{ color: c.primarioSuave }}
71              style={[estilos.fila, { borderBottomColor: c.borde, backgroundColor: c.fondo }]}
72            >
73              <Text style={[estilos.numero, { color: c.textoTenue }]}>{String(index + 1).padStart(2, '0')}</Text>
74              <View style={{ flex: 1 }}>
75                <Text style={[tipo.cuerpo, { color: c.texto, fontWeight: '600' }]} numberOfLines={1}>{item.nombre
     }</Text>
76                <Text style={[tipo.pie, { color: c.textoSecundario }]}>CUI {item.cui}{item.matriculaId ? ' ·
     matriculado en la app' : ''}</Text>
77              </View>
78              {hora ? (
79                <View style={[estilos.estado, { backgroundColor: c.exitoSuave }]}>
80                  <Ionicons name="checkmark" size={14} color={c.exito} />
81                  <Text style={[tipo.pie, { color: c.exito, fontWeight: '700' }]}>{hora}</Text>
82                </View>
83              ) : (
84                <View style={[estilos.estado, { borderColor: c.borde, borderWidth: 1 }]}>
```

```
                  <Text style={[tipo.pie, { color: c.textoTenue }]}>Pendiente</Text>
                </View>
              )}
            </Pressable>
          );
        }}
      />
      <View style={[estilos.pie, { backgroundColor: c.superficie, borderTopColor: c.borde }]}>
        <Boton titulo="Exportar CSV" icono="download-outline" variante="secundario" onPress={exportar} style={{ flex: 1 }} />
        <Boton titulo="Escanear" icono="scan-outline" onPress={() => navigation.navigate('Docente', { screen: 'Escaner' })} style={{ flex: 1 }} />
      </View>
    </View>
  );
}

const estilos = StyleSheet.create({
  resumen: { padding: espacio.l, gap: espacio.s, borderBottomWidth: StyleSheet.hairlineWidth },
  cifras: { flexDirection: 'row', alignItems: 'baseline', gap: espacio.s },
  campo: { flexDirection: 'row', alignItems: 'center', gap: espacio.s, borderWidth: 1, borderRadius: radio.m, paddingHorizontal: espacio.m, minHeight: 40, marginTop: espacio.xs },
  input: { flex: 1, fontSize: 14, paddingVertical: 6 },
  fila: { height: ALTO_FILA, flexDirection: 'row', alignItems: 'center', gap: espacio.m, paddingHorizontal: espacio.l, borderBottomWidth: StyleSheet.hairlineWidth },
  numero: { width: 22, fontSize: 12, fontWeight: '700', fontVariant: ['tabular-nums'] },
  estado: { flexDirection: 'row', alignItems: 'center', gap: 4, paddingHorizontal: 10, paddingVertical: 5, borderRadius: radio.pill },
  pie: { flexDirection: 'row', gap: espacio.s, padding: espacio.l, borderTopWidth: StyleSheet.hairlineWidth },
});
```

A.21 I9 · Escáner

src/screens/docente/EscanerScreen.tsx

```
/**
 * Interfaz 9 · Escáner de pases QR para el registro de asistencia (RF-08).
 * Layout: la vista de cámara ocupa el fondo y la guía de encuadre se superpone con
 * StyleSheet.absoluteFill; las esquinas del marco son Views con position: 'absolute'.
 * Si la cámara no está disponible, el docente puede digitar el código de verificación.
 */
import { Ionicons } from '@expo/vector-icons';
import { CameraView, useCameraPermissions } from 'expo-camera';
import { useRef, useState } from 'react';
import { KeyboardAvoidingView, Platform, ScrollView, StyleSheet, Text, TextInput, View } from 'react-native';
import { Boton, Tarjeta } from '../../components/ui';
import { NOMBRE_DIA, rango, verificarQR } from '../../domain/reglas';
import type { Matricula } from '../../domain/types';
import { codigoVerificacion, curso, useApp, usuario } from '../../state/AppStore';
import { useTema } from '../../theme/ThemeContext';
import { espacio, radio, tipo } from '../../theme/tokens';

type Resultado = { ok: true; matricula: Matricula; hora: string } | { ok: false; mensaje: string };

export default function EscanerScreen() {
  const { c } = useTema();
  const { estado, dispatch } = useApp();
  const [permiso, pedirPermiso] = useCameraPermissions();
  const [codigo, setCodigo] = useState('');
  const [resultado, setResultado] = useState<Resultado | null>(null);
  const bloqueado = useRef(false);

  const registrar = (m: Matricula | undefined) => {
    if (!m || m.estado !== 'confirmada') {
      setResultado({ ok: false, mensaje: 'El pase no corresponde a una matrícula vigente.' });
```

```
      return;
    }
    const turno = estado.turnos.find((t) => t.id === m.turnoId)!;
    if (turno.docenteId !== estado.usuario?.id) {
      setResultado({ ok: false, mensaje: `El pase es de ${curso(turno.cursoId).nombre} grupo ${turno.grupo}, que no está a tu cargo.` });
      return;
    }
    const hora = new Date().toLocaleTimeString('es-PE', { hour: '2-digit', minute: '2-digit' });
    dispatch({ tipo: 'MARCAR_ASISTENCIA', filaId: m.id, hora });
    setResultado({ ok: true, matricula: m, hora });
  };

  const alLeer = ({ data }: { data: string }) => {
    if (bloqueado.current) return;
    bloqueado.current = true;
    const v = verificarQR(data);
    registrar(v.valido ? estado.matriculas.find((m) => m.id === v.matriculaId) : undefined);
    setTimeout(() => (bloqueado.current = false), 2500);
  };

  const validarCodigo = () => {
    const limpio = codigo.trim().toUpperCase();
    registrar(estado.matriculas.find((m) => codigoVerificacion(m) === limpio));
  };

  return (
    <KeyboardAvoidingView style={{ flex: 1, backgroundColor: c.fondo }} behavior={Platform.OS === 'ios' ? 'padding' : undefined}>
      <ScrollView contentContainerStyle={estilos.contenido} keyboardShouldPersistTaps="handled">
        <View style={estilos.visor}>
          {permiso?.granted ? (
            <CameraView style={StyleSheet.absoluteFill} facing="back" barcodeScannerSettings={{ barcodeTypes: ['qr'] }} onBarcodeScanned={alLeer} />
          ) : (
            <View style={[StyleSheet.absoluteFill, estilos.sinPermiso]}>
              <Ionicons name="camera-outline" size={40} color="#FFFFFF" />
              <Text style={[tipo.cuerpo, { color: '#FFFFFF', textAlign: 'center' }]}>Se necesita la cámara para leer los pases QR.</Text>
              <Boton titulo="Permitir cámara" icono="lock-open-outline" variante="marca" onPress={pedirPermiso} />
            </View>
          )}
          <View style={[StyleSheet.absoluteFill, estilos.capa]} pointerEvents="none">
            <View style={estilos.marco}>
              {(['si', 'sd', 'ii', 'id'] as const).map((p) => (
                <View key={p} style={[estilos.esquina, estilos[p]]} />
              ))}
            </View>
            <Text style={estilos.ayuda}>Encuadra el código QR del pase</Text>
          </View>
        </View>

        {resultado && (
          <Tarjeta style={[estilos.resultado, { borderColor: resultado.ok ? c.exito : c.peligro, backgroundColor: resultado.ok ? c.exitoSuave : c.peligroSuave }]}>
            <Ionicons name={resultado.ok ? 'checkmark-circle' : 'close-circle'} size={32} color={resultado.ok ? c.exito : c.peligro} />
            <View style={{ flex: 1, gap: 2 }}>
              {resultado.ok ? (
                (() => {
                  const t = estado.turnos.find((x) => x.id === resultado.matricula.turnoId)!;
                  return (
                    <>
                      <Text style={[tipo.etiqueta, { color: c.exito }]}>Asistencia registrada · {resultado.hora}</Text>
                      <Text style={[tipo.cuerpo, { color: c.texto, fontWeight: '700' }]}>{usuario(resultado.
```

```
    matricula.estudianteId)?.nombre}</Text>
90                  <Text style={[tipo.pie, { color: c.textoSecundario }]}>{curso(t.cursoId).nombre} · Grupo {t.
    grupo} · {NOMBRE_DIA[t.dia]} {rango(t)}</Text>
91                </>
92              );
93            })()
94          ) : (
95            <>
96              <Text style={[tipo.etiqueta, { color: c.peligro }]}>Pase rechazado</Text>
97              <Text style={[tipo.cuerpo, { color: c.texto }]}>{resultado.mensaje}</Text>
98            </>
99          )}
100       </View>
101     </Tarjeta>
102   )}
103
104   <Tarjeta style={{ gap: espacio.m }}>
105     <Text style={[tipo.subtitulo, { color: c.texto }]}>Ingreso manual</Text>
106     <Text style={[tipo.cuerpo, { color: c.textoSecundario }]}>Si la cámara falla, digita el código de
    verificación de 8 caracteres que aparece bajo el QR del estudiante.</Text>
107     <TextInput
108       value={codigo}
109       onChangeText={(v) => setCodigo(v.toUpperCase().replace(/[^0-9A-F]/g, '').slice(0, 8))}
110       placeholder="Ej. 3FA92C1B"
111       placeholderTextColor={c.textoTenue}
112       autoCapitalize="none"
113       autoCorrect={false}
114       style={[estilos.input, { color: c.texto, borderColor: c.borde, backgroundColor: c.fondo }]}
115       accessibilityLabel="Código de verificación"
116       onSubmitEditing={validarCodigo}
117     />
118     <Boton titulo="Validar código" icono="shield-checkmark-outline" onPress={validarCodigo} deshabilitado={
    codigo.trim().length !== 8} />
119   </Tarjeta>
120  </ScrollView>
121 </KeyboardAvoidingView>
122 );
123 }
124
125 const LADO = 220;
126 const estilos = StyleSheet.create({
127   contenido: { padding: espacio.l, gap: espacio.l, paddingBottom: espacio.xxl },
128   visor: { height: 340, borderRadius: radio.l + 4, overflow: 'hidden', backgroundColor: '#0B1118' },
129   sinPermiso: { alignItems: 'center', justifyContent: 'center', gap: espacio.m, padding: espacio.xl },
130   capa: { alignItems: 'center', justifyContent: 'center', gap: espacio.l },
131   marco: { width: LADO, height: LADO },
132   esquina: { position: 'absolute', width: 36, height: 36, borderColor: '#FFFFFF' },
133   si: { top: 0, left: 0, borderTopWidth: 4, borderLeftWidth: 4, borderTopLeftRadius: 12 },
134   sd: { top: 0, right: 0, borderTopWidth: 4, borderRightWidth: 4, borderTopRightRadius: 12 },
135   ii: { bottom: 0, left: 0, borderBottomWidth: 4, borderLeftWidth: 4, borderBottomLeftRadius: 12 },
136   id: { bottom: 0, right: 0, borderBottomWidth: 4, borderRightWidth: 4, borderBottomRightRadius: 12 },
137   ayuda: { color: '#FFFFFF', fontWeight: '600', backgroundColor: 'rgba(0,0,0,0.45)', paddingHorizontal: espacio.m,
      paddingVertical: 6, borderRadius: radio.pill, overflow: 'hidden' },
138   resultado: { flexDirection: 'row', alignItems: 'center', gap: espacio.m, borderWidth: 1.5 },
139   input: { borderWidth: 1, borderRadius: radio.m, paddingHorizontal: espacio.m, minHeight: 48, fontSize: 20,
     letterSpacing: 4, fontWeight: '700', textAlign: 'center' },
140 });
```

A.22 I10 · Laboratorios

src/screens/admin/LaboratoriosScreen.tsx

```tsx
/**
 * Interfaz 10 · Gestión de laboratorios (RF-09).
 * Layout: FlatList con numColumns calculado con useWindowDimensions (1 columna en teléfonos,
 * 2 en tabletas pequeñas o teléfonos en horizontal y 3 en tabletas grandes). Al cambiar el número de columnas la lista
 * debe montarse de nuevo, por eso la prop key depende de él. La edición usa el Modal nativo.
 */
import { Ionicons } from '@expo/vector-icons';
import { useState } from 'react';
import { FlatList, Modal, Pressable, StyleSheet, Switch, Text, TextInput, View, useWindowDimensions } from 'react-native';
import { BarraProgreso, Boton, Insignia, Tarjeta } from '../../components/ui';
import type { Laboratorio } from '../../domain/types';
import { useApp } from '../../state/AppStore';
import { useTema } from '../../theme/ThemeContext';
import { espacio, radio, tipo } from '../../theme/tokens';

function Stepper({ valor, onCambiar, min, max }: { valor: number; onCambiar: (v: number) => void; min: number; max: number }) {
  const { c } = useTema();
  const boton = (icono: 'remove' | 'add', delta: number) => (
    <Pressable
      onPress={() => onCambiar(Math.min(max, Math.max(min, valor + delta)))}
      accessibilityRole="button"
      accessibilityLabel={delta > 0 ? 'Aumentar capacidad' : 'Disminuir capacidad'}
      style={[estilos.paso, { borderColor: c.borde, backgroundColor: c.fondo }]}
    >
      <Ionicons name={icono} size={16} color={c.primario} />
    </Pressable>
  );
  return (
    <View style={estilos.stepper}>
      {boton('remove', -1)}
      <Text style={[tipo.subtitulo, { color: c.texto, minWidth: 28, textAlign: 'center' }]}>{valor}</Text>
      {boton('add', 1)}
    </View>
  );
}

export default function LaboratoriosScreen() {
  const { c } = useTema();
  const { estado, dispatch } = useApp();
  const { width } = useWindowDimensions();
  const [edicion, setEdicion] = useState<Laboratorio | null>(null);
  const columnas = width < 600 ? 1 : width < 960 ? 2 : 3;

  const guardar = (l: Laboratorio) => dispatch({ tipo: 'ACTUALIZAR_LABORATORIO', laboratorio: l });
  const asignados = (id: string) => estado.turnos.filter((t) => t.laboratorioId === id);
  const totalPcs = estado.laboratorios.filter((l) => l.operativo).reduce((s, l) => s + l.capacidadPcs, 0);

  return (
    <View style={{ flex: 1, backgroundColor: c.fondo }}>
      <FlatList
        key={columnas}
        data={estado.laboratorios}
        numColumns={columnas}
        keyExtractor={(l) => l.id}
        contentContainerStyle={estilos.contenido}
        columnWrapperStyle={columnas > 1 ? { gap: espacio.m } : undefined}
        ItemSeparatorComponent={() => <View style={{ height: espacio.m }} />}
        ListHeaderComponent={
          <View style={[estilos.resumen, { backgroundColor: c.primario }]}>
```

```
          <Ionicons name="desktop-outline" size={28} color={c.sobrePrimario} />
          <View style={{ flex: 1 }}>
            <Text style={[tipo.cifra, { color: c.sobrePrimario }]}>{totalPcs} PC</Text>
            <Text style={[tipo.pie, { color: c.sobrePrimario, opacity: 0.85 }]}>operativas en {estado.laboratorios.filter((l) => l.operativo).length} de {estado.laboratorios.length} ambientes</Text>
          </View>
        </View>
      }
      renderItem={({ item }) => {
        const turnos = asignados(item.id);
        const ocupadas = turnos.reduce((s, t) => s + t.cuposOcupados, 0);
        const ofertadas = turnos.reduce((s, t) => s + t.cuposTotales, 0);
        return (
          <Tarjeta style={[estilos.tarjeta, !item.operativo && { opacity: 0.75 }]}>
            <View style={estilos.cabecera}>
              <Text style={[tipo.cuerpo, { color: c.texto, fontWeight: '700', flex: 1 }]} numberOfLines={1}>{item.nombre}</Text>
              <Insignia estado={item.operativo ? 'verde' : 'rojo'} texto={item.operativo ? 'Operativo' : 'Inhabilitado'} />
            </View>
            <Text style={[tipo.pie, { color: c.textoSecundario }]} numberOfLines={2}>{item.pabellon}</Text>
            <Text style={[tipo.pie, { color: c.textoTenue }]} numberOfLines={2}>{item.equipamiento}</Text>
            <View style={estilos.cabecera}>
              <Text style={[tipo.etiqueta, { color: c.textoSecundario }]}>Capacidad (PC)</Text>
              <Stepper valor={item.capacidadPcs} min={10} max={40} onCambiar={(v) => guardar({ ...item, capacidadPcs: v })} />
            </View>
            <BarraProgreso valor={ocupadas} total={Math.max(1, ofertadas)} />
            <Text style={[tipo.pie, { color: c.textoSecundario }]}>{turnos.length} grupo(s) · {ocupadas}/{ofertadas} puestos reservados</Text>
            <View style={estilos.cabecera}>
              <Text style={[tipo.etiqueta, { color: c.textoSecundario }]}>Habilitado para matrícula</Text>
              <Switch value={item.operativo} onValueChange={(v) => guardar({ ...item, operativo: v })} trackColor={{ true: c.primario, false: c.borde }} accessibilityLabel={`Habilitar ${item.nombre}`} />
            </View>
            <Boton titulo="Editar datos" icono="create-outline" variante="secundario" onPress={() => setEdicion(item)} accessibilityLabel={`Editar ${item.nombre}`} />
          </Tarjeta>
        );
      }}
    />

    <Modal visible={edicion !== null} animationType="slide" transparent onRequestClose={() => setEdicion(null)}>
      <Pressable style={estilos.velo} onPress={() => setEdicion(null)} accessibilityLabel="Cerrar edición" />
      {edicion && (
        <View style={[estilos.hoja, { backgroundColor: c.superficie }]}>
          <View style={[estilos.asa, { backgroundColor: c.borde }]} />
          <Text style={[tipo.subtitulo, { color: c.texto }]}>Editar {edicion.nombre}</Text>
          {([['nombre', 'Nombre'], ['pabellon', 'Ubicación'], ['equipamiento', 'Equipamiento']] as const).map(([campo, etiqueta]) => (
            <View key={campo} style={{ gap: espacio.xs }}>
              <Text style={[tipo.etiqueta, { color: c.textoSecundario }]}>{etiqueta}</Text>
              <TextInput
                value={edicion[campo]}
                onChangeText={(v) => setEdicion({ ...edicion, [campo]: v })}
                style={[estilos.input, { color: c.texto, borderColor: c.borde, backgroundColor: c.fondo }]}
                accessibilityLabel={etiqueta}
              />
            </View>
          ))}
          <View style={{ flexDirection: 'row', gap: espacio.s }}>
            <Boton titulo="Cancelar" variante="fantasma" onPress={() => setEdicion(null)} style={{ flex: 1 }} />
            <Boton titulo="Guardar" icono="save-outline" onPress={() => { guardar(edicion); setEdicion(null); }} style={{ flex: 1 }} />
          </View>
        </View>
```

```
        )}
      </Modal>
    </View>
  );
}

const estilos = StyleSheet.create({
  contenido: { padding: espacio.l, paddingBottom: espacio.xxl },
  resumen: { flexDirection: 'row', alignItems: 'center', gap: espacio.l, padding: espacio.l, borderRadius: radio.l,
      marginBottom: espacio.l },
  tarjeta: { flex: 1, gap: espacio.s },
  cabecera: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', gap: espacio.s },
  stepper: { flexDirection: 'row', alignItems: 'center', gap: espacio.s },
  paso: { width: 32, height: 32, borderRadius: radio.s, borderWidth: 1, alignItems: 'center', justifyContent: '
     center' },
  velo: { flex: 1, backgroundColor: 'rgba(0,0,0,0.4)' },
  hoja: { padding: espacio.xl, gap: espacio.m, borderTopLeftRadius: radio.l + 8, borderTopRightRadius: radio.l + 8
      },
  asa: { alignSelf: 'center', width: 40, height: 4, borderRadius: 2 },
  input: { borderWidth: 1, borderRadius: radio.m, paddingHorizontal: espacio.m, minHeight: 44, fontSize: 15 },
});
```

A.23 I11 · Configuración de matrícula

src/screens/admin/ConfiguracionScreen.tsx

```
/**
 * Interfaz 11 · Configuración de la ventana de matrícula (RF-09).
 * Layout: formulario en ScrollView con filas label/valor alineadas con flexDirection: 'row'
 * y justifyContent: 'space-between'; los grupos agotados se listan con map dentro de una Tarjeta.
 */
import { Ionicons } from '@expo/vector-icons';
import { useState } from 'react';
import { Alert, ScrollView, StyleSheet, Switch, Text, TextInput, View } from 'react-native';
import { Boton, Insignia, Tarjeta, TituloSeccion } from '../../components/ui';
import { NOMBRE_DIA, rango, semaforo, ventanaAbierta } from '../../domain/reglas';
import { curso, useApp } from '../../state/AppStore';
import { useTema } from '../../theme/ThemeContext';
import { espacio, radio, tipo } from '../../theme/tokens';

const aTexto = (iso: string) => {
  const d = new Date(iso);
  const p = (n: number) => String(n).padStart(2, '0');
  return `${p(d.getDate())}/${p(d.getMonth() + 1)}/${d.getFullYear()} ${p(d.getHours())}:${p(d.getMinutes())}`;
};

/** "dd/mm/aaaa hh:mm" -> ISO, o null si no es una fecha válida. */
const aISO = (texto: string): string | null => {
  const m = texto.trim().match(/^(\d{2})\/(\d{2})\/(\d{4}) (\d{2}):(\d{2})$/);
  if (!m) return null;
  const d = new Date(Number(m[3]), Number(m[2]) - 1, Number(m[1]), Number(m[4]), Number(m[5]));
  return Number.isNaN(d.getTime()) || d.getDate() !== Number(m[1]) ? null : d.toISOString();
};

export default function ConfiguracionScreen() {
  const { c } = useTema();
  const { estado, dispatch } = useApp();
  const [inicio, setInicio] = useState(aTexto(estado.config.inicio));
  const [fin, setFin] = useState(aTexto(estado.config.fin));
  const [maximo, setMaximo] = useState(String(estado.config.maxLaboratoriosPorEstudiante));
  const [anulacion, setAnulacion] = useState(estado.config.permitirAnulacion);
  const [aviso, setAviso] = useState(estado.config.notificarAforoCompleto);
  const [error, setError] = useState<string | null>(null);

  const abierta = ventanaAbierta(estado.config, new Date());
```

```
  const agotados = estado.turnos.filter((t) => semaforo(t) === 'rojo');

  const guardar = () => {
    const i = aISO(inicio);
    const f = aISO(fin);
    const max = Number(maximo);
    if (!i || !f) return setError('Usa el formato dd/mm/aaaa hh:mm en ambas fechas.');
    if (new Date(f) <= new Date(i)) return setError('El cierre debe ser posterior a la apertura.');
    if (!Number.isInteger(max) || max < 1 || max > 6) return setError('El máximo de laboratorios debe estar entre 1 y 6.');
    setError(null);
    dispatch({ tipo: 'ACTUALIZAR_CONFIG', config: { inicio: i, fin: f, maxLaboratoriosPorEstudiante: max, permitirAnulacion: anulacion, notificarAforoCompleto: aviso } });
    Alert.alert('Configuración guardada', 'Los cambios se aplican de inmediato a todos los estudiantes.');
  };

  const campo = (etiqueta: string, valor: string, set: (v: string) => void, icono: 'calendar-outline' | 'calendar-clear-outline', ayuda: string) => (
    <View style={{ gap: espacio.xs }}>
      <Text style={[tipo.etiqueta, { color: c.textoSecundario }]}>{etiqueta}</Text>
      <View style={[estilos.campo, { borderColor: c.borde, backgroundColor: c.fondo }]}>
        <Ionicons name={icono} size={18} color={c.textoTenue} />
        <TextInput value={valor} onChangeText={set} style={[estilos.input, { color: c.texto }]} keyboardType="numbers-and-punctuation" accessibilityLabel={etiqueta} />
      </View>
      <Text style={[tipo.pie, { color: c.textoTenue }]}>{ayuda}</Text>
    </View>
  );

  return (
    <ScrollView style={{ backgroundColor: c.fondo }} contentContainerStyle={estilos.contenido} keyboardShouldPersistTaps="handled">
      <Tarjeta style={[estilos.estado, { backgroundColor: abierta ? c.exitoSuave : c.peligroSuave, borderColor: 'transparent' }]}>
        <Ionicons name={abierta ? 'lock-open-outline' : 'lock-closed-outline'} size={26} color={abierta ? c.exito : c.peligro} />
        <View style={{ flex: 1 }}>
          <Text style={[tipo.etiqueta, { color: abierta ? c.exito : c.peligro }]}>{abierta ? 'Ventana abierta ahora' : 'Ventana cerrada ahora'}</Text>
          <Text style={[tipo.pie, { color: c.textoSecundario }]}>{aTexto(estado.config.inicio)} → {aTexto(estado.config.fin)}</Text>
        </View>
      </Tarjeta>

      <TituloSeccion titulo="Ventana de matrícula" />
      <Tarjeta style={{ gap: espacio.l }}>
        {campo('Apertura', inicio, setInicio, 'calendar-outline', 'Formato dd/mm/aaaa hh:mm (hora de Lima).')}
        {campo('Cierre', fin, setFin, 'calendar-clear-outline', 'Pasada esta hora el botón Confirmar se desactiva.')}
        <View style={estilos.fila}>
          <View style={{ flex: 1 }}>
            <Text style={[tipo.cuerpo, { color: c.texto, fontWeight: '600' }]}>Máximo de laboratorios</Text>
            <Text style={[tipo.pie, { color: c.textoSecundario }]}>Por estudiante y semestre</Text>
          </View>
          <TextInput value={maximo} onChangeText={setMaximo} keyboardType="number-pad" maxLength={1} style={[estilos.numero, { color: c.texto, borderColor: c.borde, backgroundColor: c.fondo }]} accessibilityLabel="Máximo de laboratorios" />
        </View>
      </Tarjeta>

      <TituloSeccion titulo="Reglas" />
      <Tarjeta style={{ gap: espacio.l }}>
        <View style={estilos.fila}>
          <View style={{ flex: 1 }}>
            <Text style={[tipo.cuerpo, { color: c.texto, fontWeight: '600' }]}>Permitir anular matrícula</Text>
            <Text style={[tipo.pie, { color: c.textoSecundario }]}>El cupo vuelve a la oferta al instante</Text>
```

```
          </View>
          <Switch value={anulacion} onValueChange={setAnulacion} trackColor={{ true: c.primario, false: c.borde }}
     accessibilityLabel="Permitir anular matrícula" />
        </View>
        <View style={estilos.fila}>
          <View style={{ flex: 1 }}>
            <Text style={[tipo.cuerpo, { color: c.texto, fontWeight: '600' }]}>Avisar aforo completo</Text>
            <Text style={[tipo.pie, { color: c.textoSecundario }]}>Notifica a coordinación cuando un grupo llega a
     0</Text>
          </View>
          <Switch value={aviso} onValueChange={setAviso} trackColor={{ true: c.primario, false: c.borde }}
    accessibilityLabel="Avisar aforo completo" />
        </View>
      </Tarjeta>

      {error && (
        <View style={[estilos.error, { backgroundColor: c.peligroSuave }]}>
          <Ionicons name="alert-circle" size={18} color={c.peligro} />
          <Text style={[tipo.etiqueta, { color: c.peligro, flex: 1 }]}>{error}</Text>
        </View>
      )}
      <Boton titulo="Guardar configuración" icono="save-outline" variante="marca" onPress={guardar} />

      <TituloSeccion titulo={`Grupos agotados (${agotados.length})`} />
      <Tarjeta style={{ gap: espacio.m }}>
        {agotados.length === 0 ? (
          <Text style={[tipo.cuerpo, { color: c.textoSecundario }]}>Todos los grupos tienen vacantes.</Text>
        ) : (
          agotados.map((t) => (
            <View key={t.id} style={estilos.fila}>
              <View style={{ flex: 1 }}>
                <Text style={[tipo.cuerpo, { color: c.texto, fontWeight: '600' }]}>{curso(t.cursoId).nombre} · {t.
    grupo}</Text>
                <Text style={[tipo.pie, { color: c.textoSecundario }]}>{NOMBRE_DIA[t.dia]} {rango(t)}</Text>
              </View>
              <Insignia estado="rojo" />
            </View>
          ))
        )}
        {agotados.length > 0 && (
          <Boton
            titulo="Evaluar grupo adicional"
            icono="add-circle-outline"
            variante="secundario"
            onPress={() => Alert.alert('Grupo adicional', 'Se registró la solicitud para evaluar la apertura de un
     grupo adicional con la dirección de escuela.')}
          />
        )}
      </Tarjeta>
    </ScrollView>
  );
}

const estilos = StyleSheet.create({
  contenido: { padding: espacio.l, paddingBottom: espacio.xxl },
  estado: { flexDirection: 'row', alignItems: 'center', gap: espacio.m },
  campo: { flexDirection: 'row', alignItems: 'center', gap: espacio.s, borderWidth: 1, borderRadius: radio.m,
     paddingHorizontal: espacio.m, minHeight: 46 },
  input: { flex: 1, fontSize: 16, paddingVertical: espacio.s, fontVariant: ['tabular-nums'] },
  fila: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', gap: espacio.m },
  numero: { width: 56, height: 44, borderWidth: 1, borderRadius: radio.m, textAlign: 'center', fontSize: 18,
     fontWeight: '700' },
  error: { flexDirection: 'row', gap: espacio.s, alignItems: 'center', padding: espacio.m, borderRadius: radio.m,
     marginVertical: espacio.m },
});
```

A.24 I12 · Perfil

src/screens/PerfilScreen.tsx

```tsx
/**
 * Interfaz 12 · Perfil y ajustes (común a los tres roles).
 * Layout: lista de ajustes con filas icono / texto / control a la derecha; la sección de
 * dispositivo muestra Platform y useWindowDimensions para evidenciar el entorno de ejecución.
 */
import { CommonActions, useNavigation } from '@react-navigation/native';
import { useState } from 'react';
import { Platform, ScrollView, StyleSheet, Switch, Text, View, useWindowDimensions } from 'react-native';
import { Avatar, Boton, Chip, FilaAjuste, Tarjeta, TituloSeccion } from '../components/ui';
import { useApp } from '../state/AppStore';
import { useTema } from '../theme/ThemeContext';
import { espacio, tipo } from '../theme/tokens';

const ROL = { estudiante: 'Estudiante', docente: 'Docente / Jefe de prácticas', admin: 'Administración de laboratorios' };

export default function PerfilScreen() {
  const { c, preferencia, setPreferencia } = useTema();
  const { estado, dispatch } = useApp();
  const navigation = useNavigation();
  const { width, height, fontScale } = useWindowDimensions();
  const [avisos, setAvisos] = useState(true);
  const [recordatorios, setRecordatorios] = useState(true);
  const u = estado.usuario;

  const salir = () => {
    dispatch({ tipo: 'CERRAR_SESION' });
    navigation.dispatch(CommonActions.reset({ index: 0, routes: [{ name: 'Login' }] }));
  };

  const interruptor = (valor: boolean, set: (v: boolean) => void, etiqueta: string) => (
    <Switch value={valor} onValueChange={set} trackColor={{ true: c.primario, false: c.borde }} accessibilityLabel={etiqueta} />
  );

  return (
    <ScrollView style={{ backgroundColor: c.fondo }} contentContainerStyle={estilos.contenido}>
      <Tarjeta style={estilos.cabecera}>
        <Avatar nombre={u?.nombre ?? '?'} tamano={64} />
        <View style={{ flex: 1, gap: 2 }}>
          <Text style={[tipo.subtitulo, { color: c.texto }]}>{u?.nombre}</Text>
          <Text style={[tipo.pie, { color: c.textoSecundario }]}>{u?.correo}</Text>
          <Text style={[tipo.etiqueta, { color: c.marca }]}>{u ? ROL[u.rol] : ''}{u?.cui ? ` · CUI ${u.cui}` : ''}</Text>
        </View>
      </Tarjeta>

      <TituloSeccion titulo="Apariencia" />
      <Tarjeta style={{ gap: espacio.m }}>
        <Text style={[tipo.cuerpo, { color: c.textoSecundario }]}>Tema de la aplicación</Text>
        <View style={estilos.chips}>
          <Chip texto="Sistema" icono="phone-portrait-outline" activo={preferencia === 'sistema'} onPress={() => setPreferencia('sistema')} />
          <Chip texto="Claro" icono="sunny-outline" activo={preferencia === 'claro'} onPress={() => setPreferencia('claro')} />
          <Chip texto="Oscuro" icono="moon-outline" activo={preferencia === 'oscuro'} onPress={() => setPreferencia('oscuro')} />
        </View>
      </Tarjeta>

      <TituloSeccion titulo="Notificaciones" />
      <Tarjeta style={{ paddingVertical: 0 }}>
```

```
        <FilaAjuste icono="notifications-outline" titulo="Cambios de vacantes" detalle="Aviso cuando se libera un cupo" derecha={interruptor(avisos, setAvisos, 'Cambios de vacantes')} />
        <FilaAjuste icono="alarm-outline" titulo="Recordatorio de sesión" detalle="30 minutos antes del laboratorio" derecha={interruptor(recordatorios, setRecordatorios, 'Recordatorio de sesión')} />
      </Tarjeta>

      <TituloSeccion titulo="Dispositivo" />
      <Tarjeta style={{ paddingVertical: 0 }}>
        <FilaAjuste icono="hardware-chip-outline" titulo="Plataforma" detalle={`${Platform.OS} · versión ${String(Platform.Version)}`} />
        <FilaAjuste icono="resize-outline" titulo="Ventana" detalle={`${Math.round(width)} × ${Math.round(height)} dp · escala de texto ${fontScale.toFixed(2)}`} />
        <FilaAjuste icono="information-circle-outline" titulo="Acerca de MatriLab" detalle="Versión 1.0.0 · React Native + Expo SDK 58" />
      </Tarjeta>

      <Boton titulo="Cerrar sesión" icono="log-out-outline" variante="fantasma" onPress={salir} style={{ marginTop: espacio.xl }} accessibilityLabel="Cerrar sesión" />
    </ScrollView>
  );
}

const estilos = StyleSheet.create({
  contenido: { padding: espacio.l, paddingBottom: espacio.xxl },
  cabecera: { flexDirection: 'row', alignItems: 'center', gap: espacio.l },
  chips: { flexDirection: 'row', flexWrap: 'wrap', gap: espacio.s },
});
```

A.25 Pruebas unitarias

__tests__/reglas.test.ts

```
import { CURSOS, MATRICULAS, TEORIA, TURNOS, configInicial, nominaDemo } from '../src/domain/datos';
import {
  aHora, aMinutos, cuposLibres, haySolape, huella, liberarCupo, nominaCSV, ordenarBloques, payloadQR, reservarCupo, semaforo, validarMatricula, ventanaAbierta, verificarQR,
} from '../src/domain/reglas';
import type { Turno } from '../src/domain/types';
import { codigoVerificacion, estadoInicial, nomina, reductor } from '../src/state/AppStore';

const turno = (id: string) => TURNOS.find((t) => t.id === id)!;
const nombreCurso = (id: string) => CURSOS.find((c) => c.id === id)!.nombre;
const AHORA = new Date();
const contexto = (t: Turno, extra: Partial<Parameters<typeof validarMatricula>[0]> = {}) => ({
  turno: t, turnos: TURNOS, teoria: TEORIA, matriculasActivas: MATRICULAS, config: configInicial(AHORA), ahora: AHORA, nombreCurso, ...extra,
});

describe('horas', () => {
  it('convierte entre texto y minutos', () => {
    expect(aMinutos('07:00')).toBe(420);
    expect(aHora(640)).toBe('10:40');
  });
});

describe('RN-02 · cruce de horario', () => {
  const b = (dia: 'LUN' | 'MAR', ini: string, fin: string) => ({ dia, inicio: aMinutos(ini), fin: aMinutos(fin) });
  it('detecta solape en el mismo día', () => expect(haySolape(b('LUN', '07:00', '08:40'), b('LUN', '08:00', '09:00'))).toBe(true));
  it('bloques contiguos no se cruzan', () => expect(haySolape(b('LUN', '07:00', '08:40'), b('LUN', '08:40', '10:30'))).toBe(false));
  it('días distintos no se cruzan', () => expect(haySolape(b('LUN', '07:00', '08:40'), b('MAR', '07:00', '08:40'))).toBe(false));
});
```

```
28
29  describe('semáforo de vacantes', () => {
30    it('rojo sin cupos, ámbar con menos del 30 % y verde en otro caso', () => {
31      expect(semaforo(turno('PE-A'))).toBe('rojo');
32      expect(semaforo(turno('GPS-A'))).toBe('ambar');
33      expect(semaforo(turno('GPS-B'))).toBe('verde');
34    });
35  });
36
37  describe('validarMatricula', () => {
38    it('acepta un grupo libre y sin cruces', () => expect(validarMatricula(contexto(turno('PE-B')))).toEqual({ ok:
        true }));
39    it('rechaza el cruce con la teoría ya inscrita', () => {
40      const r = validarMatricula(contexto(turno('PE-C')));
41      expect(r.ok).toBe(false);
42      if (!r.ok) expect(r.motivo).toBe('CRUCE');
43    });
44    it('rechaza un segundo grupo del mismo curso', () => {
45      const r = validarMatricula(contexto(turno('CS-A')));
46      expect(!r.ok && r.motivo).toBe('MISMO_CURSO');
47    });
48    it('rechaza un grupo agotado', () => {
49      const r = validarMatricula(contexto(turno('PE-A'), { teoria: [] }));
50      expect(!r.ok && r.motivo).toBe('SIN_CUPO');
51    });
52    it('rechaza fuera de la ventana de matrícula', () => {
53      const r = validarMatricula(contexto(turno('PE-B'), { ahora: new Date(AHORA.getTime() + 10 * 86400000) }));
54      expect(!r.ok && r.motivo).toBe('VENTANA_CERRADA');
55    });
56    it('rechaza volver a matricularse en el mismo grupo', () => {
57      const r = validarMatricula(contexto(turno('CS-B')));
58      expect(!r.ok && r.motivo).toBe('YA_MATRICULADO');
59    });
60    it('rechaza el cruce con otro laboratorio ya matriculado', () => {
61      // Grupo hipotético de GSTI el miércoles 09:00, encima del laboratorio CS-B (08:50-10:30).
62      const nuevo: Turno = { ...turno('GSTI-A'), id: 'GSTI-X', inicio: aMinutos('09:00'), fin: aMinutos('10:40') };
63      const r = validarMatricula(contexto(nuevo, { teoria: [] }));
64      expect(!r.ok && r.motivo).toBe('CRUCE');
65      if (!r.ok) expect(r.detalle).toContain('laboratorio de Calidad de Software');
66    });
67    it('respeta el máximo de laboratorios por estudiante', () => {
68      const config = { ...configInicial(AHORA), maxLaboratoriosPorEstudiante: 1 };
69      const r = validarMatricula(contexto(turno('PE-B'), { config }));
70      expect(!r.ok && r.motivo).toBe('LIMITE');
71    });
72  });
73
74  describe('RF-05 · reserva atómica', () => {
75    it('incrementa en uno los cupos ocupados sin mutar el arreglo original', () => {
76      const antes = turno('PE-B').cuposOcupados;
77      const { turnos, reservado } = reservarCupo(TURNOS, 'PE-B');
78      expect(reservado).toBe(true);
79      expect(turnos.find((t) => t.id === 'PE-B')!.cuposOcupados).toBe(antes + 1);
80      expect(turno('PE-B').cuposOcupados).toBe(antes);
81    });
82    it('liberar un cupo nunca deja valores negativos', () => {
83      const vacio = TURNOS.map((t) => (t.id === 'PE-B' ? { ...t, cuposOcupados: 0 } : t));
84      expect(liberarCupo(vacio, 'PE-B').find((t) => t.id === 'PE-B')!.cuposOcupados).toBe(0);
85    });
86    it('no reserva si el cupo ya se agotó', () => {
87      const { turnos, reservado } = reservarCupo(TURNOS, 'PE-A');
88      expect(reservado).toBe(false);
89      expect(turnos).toBe(TURNOS);
90    });
91    it('nunca supera el aforo físico aunque lleguen más solicitudes que vacantes', () => {
92      let turnos = TURNOS;
```

```
    const libres = cuposLibres(turno('GPS-A'));
    let aceptadas = 0;
    for (let i = 0; i < libres + 5; i++) {
      const r = reservarCupo(turnos, 'GPS-A');
      turnos = r.turnos;
      if (r.reservado) aceptadas++;
    }
    expect(aceptadas).toBe(libres);
    expect(turnos.find((t) => t.id === 'GPS-A')!.cuposOcupados).toBe(turno('GPS-A').cuposTotales);
  });
});

describe('pase QR', () => {
  const m = { id: 'M-0009', turnoId: 'PE-B', estudianteId: 'E01' };
  it('el contenido generado se verifica y devuelve la matrícula', () => expect(verificarQR(payloadQR(m))).toEqual
     ({ valido: true, matriculaId: 'M-0009' }));
  it('un QR alterado se rechaza', () => expect(verificarQR(payloadQR(m).replace('E01', 'E02')).valido).toBe(false)
     );
  it('la huella es estable y de 8 caracteres hexadecimales', () => {
    expect(huella('MATRILAB')).toBe(huella('MATRILAB'));
    expect(huella('MATRILAB')).toMatch(/^[0-9A-F]{8}$/);
  });
});

describe('horario', () => {
  it('ordena los bloques por día y luego por hora de inicio', () => {
    const orden = ordenarBloques([turno('GPS-B'), turno('PE-B'), turno('GPS-A'), turno('PE-A')]).map((t) => t.id);
    expect(orden).toEqual(['PE-A', 'GPS-A', 'PE-B', 'GPS-B']);
  });
});

describe('ventana y nómina', () => {
  it('la ventana de demostración está abierta hoy', () => expect(ventanaAbierta(configInicial(AHORA), AHORA)).toBe
     (true));
  it('la nómina sintética tiene tantas filas como cupos ocupados', () => expect(nominaDemo(turno('PE-B'))).
     toHaveLength(turno('PE-B').cuposOcupados));
  it('el CSV usa punto y coma y cabecera en español', () => {
    const csv = nominaCSV([{ cui: '1', nombre: 'A, B', correo: 'a@unsa.edu.pe', asistio: true }]);
    expect(csv.split('\n')[0]).toBe('CUI;Apellidos y nombres;Correo;Asistencia');
    expect(csv).toContain('Presente');
  });
});

describe('reductor', () => {
  const conSesion = reductor(estadoInicial, { tipo: 'INICIAR_SESION', usuario: { id: 'E01', nombre: 'Lopez Arela,
     Ower Frank', correo: 'olopeza@unsa.edu.pe', rol: 'estudiante', cui: '20213412' } });
  it('matricular agrega la matrícula y ocupa un cupo', () => {
    const e = reductor(conSesion, { tipo: 'MATRICULAR', turnoId: 'PE-B', ahora: AHORA.toISOString() });
    expect(e.matriculas).toHaveLength(2);
    expect(e.turnos.find((t) => t.id === 'PE-B')!.cuposOcupados).toBe(turno('PE-B').cuposOcupados + 1);
    expect(nomina(e, 'PE-B').some((f) => f.matriculaId === 'M-0002')).toBe(true);
  });
  it('anular libera el cupo', () => {
    const e1 = reductor(conSesion, { tipo: 'MATRICULAR', turnoId: 'PE-B', ahora: AHORA.toISOString() });
    const e2 = reductor(e1, { tipo: 'ANULAR', matriculaId: 'M-0002' });
    expect(e2.turnos.find((t) => t.id === 'PE-B')!.cuposOcupados).toBe(turno('PE-B').cuposOcupados);
  });
  it('sin sesión iniciada no se registra ninguna matrícula', () => {
    expect(reductor(estadoInicial, { tipo: 'MATRICULAR', turnoId: 'PE-B', ahora: AHORA.toISOString() })).toBe(
     estadoInicial);
  });
  it('cerrar sesión limpia el usuario sin perder las matrículas', () => {
    const e = reductor(conSesion, { tipo: 'CERRAR_SESION' });
    expect(e.usuario).toBeNull();
    expect(e.matriculas).toBe(conSesion.matriculas);
  });
```

```
  it('el administrador actualiza un laboratorio y la configuración', () => {
    const lab = { ...estadoInicial.laboratorios[0], capacidadPcs: 30 };
    const e1 = reductor(conSesion, { tipo: 'ACTUALIZAR_LABORATORIO', laboratorio: lab });
    expect(e1.laboratorios[0].capacidadPcs).toBe(30);
    const e2 = reductor(e1, { tipo: 'ACTUALIZAR_CONFIG', config: { ...e1.config, permitirAnulacion: false } });
    expect(e2.config.permitirAnulacion).toBe(false);
  });
  it('el código de verificación es la huella de 8 caracteres del pase', () => {
    expect(codigoVerificacion(MATRICULAS[0])).toMatch(/^[0-9A-F]{8}$/);
    expect(codigoVerificacion(MATRICULAS[0])).toBe(huella('MATRILAB|M-0001|CS-B|E01'));
  });
  it('una asistencia solo se marca una vez', () => {
    const e1 = reductor(conSesion, { tipo: 'MARCAR_ASISTENCIA', filaId: 'X', hora: '07:05' });
    const e2 = reductor(e1, { tipo: 'MARCAR_ASISTENCIA', filaId: 'X', hora: '07:30' });
    expect(e2.asistencias.X).toBe('07:05');
  });
});
```

A.26 Recorrido automatizado de capturas

herramientas/capturas-android.mjs

```
/**
 * Recorre la app en un emulador Android y guarda una captura de cada interfaz.
 * Localiza los controles por su accessibilityLabel (content-desc) o texto visible con
 * `uiautomator dump`, así el recorrido no depende de coordenadas fijas.
 *
 * Uso: node herramientas/capturas-android.mjs <carpeta-destino> [--oscuro]
 * Requisitos: emulador encendido y Metro sirviendo la app (npx expo start). EXP_URL apunta al servidor.
 */
import { execFileSync } from 'node:child_process';
import { mkdirSync, writeFileSync } from 'node:fs';
import { join } from 'node:path';

const ADB = process.env.ADB ?? join(process.env.LOCALAPPDATA ?? '', 'Android', 'Sdk', 'platform-tools', 'adb.exe')
    ;
const destino = process.argv[2] ?? 'capturas';
mkdirSync(destino, { recursive: true });

// MSYS_NO_PATHCONV evita que Git Bash convierta rutas del dispositivo como /sdcard.
const adb = (...args) => execFileSync(ADB, args, { maxBuffer: 64 * 1024 * 1024, env: { ...process.env,
    MSYS_NO_PATHCONV: '1' } });
const dormir = (ms) => new Promise((r) => setTimeout(r, ms));

function nodos() {
  adb('shell', 'uiautomator', 'dump', '/sdcard/ui.xml');
  const xml = adb('shell', 'cat', '/sdcard/ui.xml').toString('utf8');
  return [...xml.matchAll(/<node [^>]*?>/g)].map(([n]) => {
    const attr = (k) => (n.match(new RegExp(`${k}="([^"]*)"`)) ?? [])[1] ?? '';
    const [x1, y1, x2, y2] = attr('bounds').match(/\d+/g)?.map(Number) ?? [0, 0, 0, 0];
    return { texto: attr('text'), desc: attr('content-desc'), x: (x1 + x2) / 2, y: (y1 + y2) / 2, alto: y2 - y1 };
  });
}

async function buscar(etiqueta, { exacto = true, intentos = 20, desplazar = false } = {}) {
  for (let i = 0; i < intentos; i++) {
    const lista = nodos();
    const coincide = (v) => (exacto ? v === etiqueta : v.includes(etiqueta));
    // Primero la etiqueta de accesibilidad (el control) y luego el texto visible (p. ej. un rótulo).
    const n = lista.find((x) => coincide(x.desc)) ?? lista.find((x) => coincide(x.texto));
    if (n && n.y > 0 && n.y < 2380) return n;
    if (desplazar) adb('shell', 'input', 'swipe', '540', '1700', '540', '800', '350');
    await dormir(700);
  }
  throw new Error(`No se encontró «${etiqueta}» en pantalla`);
```

```
}

async function tocar(etiqueta, opciones) {
  const n = await buscar(etiqueta, opciones);
  adb('shell', 'input', 'tap', String(Math.round(n.x)), String(Math.round(n.y)));
  await dormir(1200);
}

async function deslizar(desdeY, hastaY) {
  adb('shell', 'input', 'swipe', '540', String(desdeY), '540', String(hastaY), '400');
  await dormir(900);
}

/** Escribe en el campo y pulsa Enter: el TextInput de una línea pierde el foco y cierra el teclado. */
async function escribir(etiqueta, texto) {
  await tocar(etiqueta);
  adb('shell', 'input', 'text', texto.replace(/ /g, '%s'));
  await dormir(500);
  adb('shell', 'input', 'keyevent', 'KEYCODE_ENTER');
  await dormir(900);
}

async function reiniciarApp() {
  const url = `${process.env.EXP_URL ?? 'exp://192.168.1.2:8081'}?__expo_disable_fab=1&__expo_disable_onboarding=1
    `;
  adb('shell', 'am', 'force-stop', 'host.exp.exponent');
  await dormir(1500);
  adb('shell', 'am', 'start', '-a', 'android.intent.action.VIEW', '-d', `'${url}'`, 'host.exp.exponent');
  await buscar('Ingresar', { intentos: 60 });
  await dormir(1500);
  // Expo Go puede abrir su menú de desarrollo al iniciar: se cierra para que no tape la app.
  if (nodos().some((n) => n.texto === 'Go home')) {
    adb('shell', 'input', 'keyevent', 'KEYCODE_BACK');
    await dormir(1000);
  }
}

async function captura(nombre) {
  await dormir(900);
  writeFileSync(join(destino, `${nombre}.png`), adb('exec-out', 'screencap', '-p'));
  console.log('captura', nombre);
}

const atras = async () => { adb('shell', 'input', 'keyevent', 'KEYCODE_BACK'); await dormir(1000); };

// ---------------- recorrido ----------------
await reiniciarApp();
await captura('01_login');
await escribir('Correo institucional', 'correo@gmail.com');
await escribir('Contraseña', 'abc12345');
await buscar('Usa tu correo institucional @unsa.edu.pe.');
await captura('01b_login_error');

await tocar('Demo estudiante');
await buscar('Mis laboratorios');
await captura('02_dashboard');

await tocar('Catálogo');
await buscar('Buscar en el catálogo');
await captura('03_catalogo');
await tocar('Sin cruce de horario');
await captura('03b_catalogo_filtro');
await tocar('Todos');

await tocar('Grupo PE-C');
await buscar('Puestos del laboratorio');
```

```
await captura('04b_detalle_cruce');
await atras();
await tocar('Grupo PE-B');
await buscar('Puestos del laboratorio');
await captura('04_detalle_grupo');
await tocar('Confirmar matrícula');
await buscar('Código de verificación', { intentos: 30 });
await captura('05_ficha_qr');
await atras();

await tocar('Horario');
await buscar('Semana');
await captura('06_horario');
await tocar('Lista');
await captura('06b_horario_lista');

await tocar('Perfil');
await buscar('Apariencia');
await captura('12_perfil');
await tocar('Oscuro');
await tocar('Inicio');
await captura('02b_dashboard_oscuro');
await tocar('Perfil');
await tocar('Claro');
await tocar('Cerrar sesión', { desplazar: true });

await tocar('Demo docente');
await buscar('Mis grupos de laboratorio');
await captura('07_panel_docente');
await deslizar(1900, 900);
await tocar('Nómina PE-B');
await buscar('presentes en la sesión de hoy', { exacto: false });
await tocar('Asistencia de Lopez Arela, Ower Frank');
await tocar('Asistencia de Apaza Mamani, Lucía Fernanda').catch(() => {});
await captura('08_nomina');
await atras();
await tocar('Escáner');
await buscar('Ingreso manual');
// Expo Go concede la cámara por aplicación: se acepta el diálogo del sistema si aparece.
if (nodos().some((n) => n.desc === 'Permitir cámara')) {
  await tocar('Permitir cámara');
  for (let i = 0; i < 6; i++) {
    const boton = nodos().find((n) => ['While using the app', 'Only this time', 'Allow', 'ALLOW', 'Mientras la app
      está en uso', 'Permitir'].includes(n.texto));
    if (boton) { adb('shell', 'input', 'tap', String(Math.round(boton.x)), String(Math.round(boton.y))); await
     dormir(1500); }
    else await dormir(800);
  }
}
await dormir(2500);
await captura('09_escaner');
// Código de verificación del pase emitido en la matrícula del estudiante (pantalla 05).
await escribir('Código de verificación', process.env.CODIGO_PASE ?? '25E350D3');
await tocar('Validar código');
await buscar('Asistencia registrada', { exacto: false });
await captura('09b_escaner_validado');

await tocar('Perfil');
await tocar('Cerrar sesión', { desplazar: true });
await tocar('Demo admin');
await buscar('Capacidad (PC)', { exacto: false });
await captura('10_laboratorios');
await tocar('Editar Laboratorio 1');
await captura('10b_laboratorio_modal');
await atras();
await tocar('Matrícula');
```

```
await buscar('Ventana de matrícula');
await captura('11_configuracion');

// Orientación horizontal: useWindowDimensions entrega ~915 dp de ancho y la lista pasa a 2 columnas.
adb('shell', 'settings', 'put', 'system', 'accelerometer_rotation', '0');
adb('shell', 'settings', 'put', 'system', 'user_rotation', '1');
await dormir(2500);
await tocar('Laboratorios');
await captura('13_laboratorios_horizontal');
adb('shell', 'settings', 'put', 'system', 'user_rotation', '0');
await dormir(2000);
console.log('recorrido completo');
```

8. REFERENCIAS

[1] Meta Platforms, Inc., *Layout with Flexbox*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/docs/flexbox

[2] Meta Platforms, Inc., *Core Components and Native Components*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/docs/intro-react-native-components

[3] Meta Platforms, Inc., *ScrollView*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/docs/scrollview

[4] Meta Platforms, Inc., *FlatList*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/docs/flatlist

[5] Meta Platforms, Inc., *SectionList*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/docs/sectionlist

[6] Meta Platforms, Inc., *StyleSheet*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/docs/stylesheet

[7] Meta Platforms, Inc., *Pressable*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/docs/pressable

[8] Meta Platforms, Inc., *KeyboardAvoidingView*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/docs/keyboardavoidingview

[9] Meta Platforms, Inc., *Modal*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/docs/modal

[10] Meta Platforms, Inc., *useWindowDimensions*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/docs/usewindowdimensions

[11] Meta Platforms, Inc., *Platform*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/docs/platform

[12] Meta Platforms, Inc., *Height and Width*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/docs/height-and-width

[13] Meta Platforms, Inc., *About the New Architecture*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/architecture/landing-page

[14] Expo, *Introduction to Expo*, Expo Documentation, 2026. [En línea]. Disponible en: https://docs.expo.dev/get-started/introduction/

[15] Meta Platforms, Inc., *Yoga: A portable layout engine*, 2026. [En línea]. Disponible en: https://www.yogalayout.dev/

[16] Meta Platforms, Inc., *SafeAreaView (deprecated)*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/docs/safeareaview

[17] E. Ortegón, J. F. Pacheco y A. Prieto, *Metodología del marco lógico para la planificación, el seguimiento y la evaluación de proyectos y programas*, Serie Manuales n.° 42. Santiago de Chile: CEPAL, 2005.

[18] Meta Platforms, Inc., *Optimizing FlatList Configuration*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/docs/optimizing-flatlist-configuration

[19] Expo, *Camera (expo-camera)*, Expo SDK Reference, 2026. [En línea]. Disponible en: https://docs.expo.dev/versions/latest/sdk/camera/

[20] React Navigation, *Native Stack Navigator*, React Navigation 7 Documentation, 2026. [En línea]. Disponible en: https://reactnavigation.org/docs/native-stack-navigator

[21] React Navigation, *Bottom Tabs Navigator*, React Navigation 7 Documentation, 2026. [En línea]. Disponible en: https://reactnavigation.org/docs/bottom-tab-navigator

[22] Expo, *Unit testing with Jest*, Expo Documentation, 2026. [En línea]. Disponible en: https://docs.expo.dev/develop/unit-testing/

[23] OpenJS Foundation, *Getting Started*, Jest Documentation, 2026. [En línea]. Disponible en: https://jestjs.io/docs/getting-started

[24] T. Atkins Jr., E. J. Etemad, R. Atanassov y O. Oyekunle (eds.), *CSS Flexible Box Layout Module Level 1*, W3C Candidate Recommendation, 2018. [En línea]. Disponible en: https://www.w3.org/TR/css-flexbox-1/

[25] International Organization for Standardization, *Ergonomics of human-system interaction — Part 210: Human-centred design for interactive systems*, ISO 9241-210:2019, 2019.

[26] B. Eisenman, *Learning React Native: Building Native Mobile Apps with JavaScript*, 2.ª ed. Sebastopol, CA, EE. UU.: O'Reilly Media, 2017.